

An Automated Model to Search For Differential Meet-In-The-Middle Attack: Applications to AndRX Ciphers

Debasmita Chakraborty¹ , Soumya Sahoo² , Phuong Hoa Nguyen³ 
and Santanu Sarkar² 

¹ Graz University of Technology, Graz, Austria
debasmitachakraborty1@gmail.com

² Department of Mathematics, Indian Institute of Technology Madras, Chennai, Tamil Nadu
600036

soumyasahoo078@gmail.com, sarkar.santanu.biri@gmail.com

³ Telecom Paris, Institut Polytechnique de Paris, France
phunguyen@telecom-paris.fr

Abstract. Differential meet-in-the-middle (MITM) cryptanalysis, recently introduced by Boura et al., has emerged as a powerful and versatile technique for assessing the security of modern block cipher designs. Since its introduction, this method has been effectively applied to a variety of block ciphers, including different variants of SKINNY, CRAFT, and AES. However, identifying such attacks manually—especially on bit-oriented ciphers with large block sizes—can be a complex and error-prone process, which underscores the growing importance of automated solutions in this domain. In this work, we present, for the first time to the best of our knowledge, a novel and efficient automated tool for constructing optimized differential MITM attacks on bit-oriented block ciphers, with a particular focus on AndRX designs. Our framework begins by modeling an efficient constraint programming (CP) model to search for single-key optimal differential trails in AndRX ciphers. Building on this, we propose a unified bitwise CP model to automatically construct optimized differential MITM attacks within the same design framework. Furthermore, we incorporate two dedicated optimization strategies—namely, the *equivalent subkey technique* and the *selective key guessing technique*—both of which are tailored to the structural properties of AndRX ciphers and significantly enhance key recovery efficiency. Additionally, we also apply two additional optimization techniques: the *parallel partitioning technique* and *reducing data with imposed conditions techniques* to further enhance the differential MITM attack on AndRX ciphers. To demonstrate the practical effectiveness of our tool, we apply it to all versions of SIMON and Simeck, two widely studied representatives of the AndRX family, and report improved cryptanalytic results. Specifically, we present differential MITM attacks on SIMON-32-64, SIMON-48-96, SIMON-64-128, and SIMON-96-144, covering 23, 25, 32, and 38 rounds, respectively. All of these results represent improvements in the number of attacked rounds compared to the best known differential attacks, classical meet-in-the-middle (MITM), and Demirci-Selçuk MITM ($\mathcal{DS}$ -MITM) attacks on the corresponding versions of SIMON. For instance, we present a 37-round differential MITM attack on SIMON-96-144, which extends the best known differential, classical MITM, and $\mathcal{DS}$ -MITM attacks by 1, 17, and 18 rounds, respectively. In the case of Simeck, we report a 29-round differential MITM attack on Simeck-48-96, improving the previous best differential attack by one round. These results demonstrate the strength and versatility of our automated tool. Importantly, our automated method for constructing differential

MITM attacks operates at the bit level and is generic in nature, making it applicable to a broad class of bit-oriented block ciphers beyond the AndRX family.

Keywords: AndRX · CP · SIMON · Simeck · Differential MITM attack · Selective Key Guessing · Equivalent Subkey · Parallel Partition.

1 Introduction

The first modern symmetric cryptanalysis techniques were developed in the late 1980s. Among these, the most notable attacks include the meet-in-the-middle attack [DH77], differential cryptanalysis [BS91], linear cryptanalysis [Mat94a], differential-linear [LH94], impossible differential [BBS99], higher-order differential [Knu95], rectangular [BDK01], and boomerang attacks [Wag99]. Over time, these attacks have been improved and adapted to analyze new cipher designs.

Differential cryptanalysis, pioneered by Biham and Shamir [BS91], stands as one of the most potent methods for evaluating block cipher security. Its core concept involves identifying high-probability input-output difference patterns (differentials) that persist across a significant number of cipher rounds. Following its inception, this cryptanalytic technique has been the subject of extensive research, resulting in a wide array of proposed methodologies and refinements [Mat94b, MWGP12, MP13, SHW⁺14]. Once a high-probability differential is identified, a few additional outer rounds can be incorporated to reduce the possible key bits. The correct key associated with these rounds enables the attacker to detect non-randomness in the differential. Over the years, the attack has been refined, and several improvements have been made: the use of structures for generating plaintext or ciphertext pairs [BS93], truncated differentials [Knu95], conditional differentials [KMNP10], probabilistic neutral bits [AFK⁺08], the early abort technique [LKKD08], and enhancements in the key recovery process.

Another older yet highly effective technique is the meet-in-the-middle (MITM) attack [DH77]. Since its introduction, MITM has undergone several enhancements. Notable extensions of this attack include the partial matching technique [DSP07], the splice-and-cut technique [AS09, GLRW10], the all subkeys approach [IS13a], the sieve-in-the-middle method [CNPV13], and the biclique method [BKR11]. Most of these approaches aim to extend the MITM attack to cover more rounds.

In [BDD⁺23], Boura et al. pioneered the differential meet-in-the-middle (differential MITM) attack, a hybrid technique integrating concepts from both MITM and differential cryptanalysis. This method extends the classical MITM approach and presents a novel strategy for key recovery within differential cryptanalysis frameworks. Its effectiveness is demonstrated in attacks on prominent block ciphers. For instance, the technique compromised 25 rounds out of 56 for SKINNY-128-384 (a 128-bit block cipher with a 384-bit tweak) in the single-tweakey model, surpassing prior results by two rounds. Similarly, against AES-256 under a related-key model, the differential MITM attack broke 12 rounds using only two related keys, exceeding the previous maximum of 10 rounds. Later, the work by Song et al. [SYL23] revisited the foundational differential MITM approach, developing several variants and generalizing the core methodology. Their refinements led to further improvements in the attacks on both 24-round SKINNY-128-384 and 12-round AES-256.

In a recent study, Ahmadian et al. [AKM⁺24] expanded upon the basic differential MITM attack by developing a truncated MITM attack. They also introduced several innovative techniques to enhance the effectiveness of this method. Their work demonstrated how to extend the attack using longer trails and incorporate probabilistic elements to improve its success rate. Furthermore, they created a MILP-based model designed to optimize the overall complexity of the truncated differential MITM attack. As a practical application,

their approach yielded the best-known attack on 23-round CRAFT, introduced a novel attack on 23-round SKINNY-64-192, and enhanced the existing best attacks on SKINNY-128-384.

Executing a differential MITM attack against a block cipher comprises two primary stages: initially, discovering a distinguisher valid for intermediate rounds, followed by mounting a key recovery procedure that expands the attack path over several additional rounds both before and after this distinguisher. Generally, identifying the optimal differential MITM attack configuration represents a complex combinatorial optimization challenge. Manual exploration techniques can become prohibitively inefficient, particularly for large block sizes where the potential solution space is vast. Consequently, creating automated methods for systematically assessing block cipher resistance against differential MITM attacks is essential.

One solution to this optimization challenge involves developing specialized algorithms. For example, a dedicated algorithm introduced at CRYPTO’2016 [DF16] was designed to find $\mathcal{DS}$ -MITM (Demirci-Selçuk-type MITM) and impossible differential (ID) attacks. However, implementing an efficient algorithm remains a complex task. Recently, several CP-based models have been developed to address these optimization and satisfaction problems, outperforming previous manual methods in terms of both accuracy and efficiency [MWGP12, ST17, SSD⁺18, XZBL16, HNE22, HSE23, HGSE24, CHNE24].

To the best of our knowledge, the differential MITM attack has mostly been applied to word-oriented ciphers such as SKINNY, CRAFT, and AES. While Ahmadian et al. [AKM⁺24] introduced an automatic MILP-aided approach for finding truncated differential MITM attacks, no automated tool exists for applying the original differential MITM attack to bit-oriented ciphers. Hence, developing a bit-wise automated tool for discovering optimized differential MITM attacks is a promising area for future exploration.

1.1 Our Contributions.

In this paper, we introduce a new CP-based, easy-to-use automated tool for identifying bit-wise differential meet-in-the-middle (MITM) attacks on AndRX ciphers. The key advantage of our tool lies in its ability to formulate a unified optimization problem that simultaneously considers all attack parameters, enabling the generation of optimized differential MITM attacks for AndRX ciphers. Furthermore, our tool can efficiently produce optimized differential MITM attacks even for block ciphers with large block sizes. Our automated framework starts by constructing an efficient CP model to search for optimal single-key differential trails in AndRX ciphers. We then demonstrate how to extend this CP model to identify optimized differential MITM attacks for bit-wise ciphers, particularly AndRX designs, based on these optimal differential trails. In addition, we incorporate two specialized optimization techniques—namely, the equivalent subkey technique and the selective key guessing technique—into our tool to improve the key recovery phase within the differential MITM attack framework, taking advantage of the structural properties of AndRX ciphers. Once the differential MITM attack is obtained using our novel and efficient automated tool, we further refine it by applying two additional optimization techniques: the parallel partitioning technique and the reducing data with imposed conditions technique. These two techniques, originally introduced in [BDD⁺23], were proposed to enhance traditional differential MITM attacks. By applying them to AndRX ciphers, we are able to further improve the differential MITM attacks discovered through our tool. To validate the effectiveness of our approach, we apply it to find differential MITM attacks on AndRX ciphers—specifically, all variants of SIMON and Simeck—for the first time, to the best of our knowledge. In particular, we present differential MITM attacks on both SIMON and Simeck using not only single optimal differential characteristics but also optimal differentials. We provide comprehensive technical details for each cipher variant

in Section 4.

In summary, our main contributions are as follows:

- We present improved differential MITM attacks on several variants of **SIMON**: a 23-round attack on **SIMON-32-64**, a 25-round attack on **SIMON-48-96**, a 32-round attack on **SIMON-64-128**, and a 38-round attack on **SIMON-96-144**. These results improve all previously known differential, original MITM, and $\mathcal{DS}$ -MITM attacks in terms of covered rounds.
- For **SIMON-48-72**, **SIMON-64-96**, and **SIMON-128-256**, we introduce 23-, 30-, and 51-round attacks, respectively. While these attacks significantly improve upon existing original MITM and $\mathcal{DS}$ -MITM results, they match the best-known differential attacks in terms of the number of rounds.
- We also present a 29-round differential MITM attack on **Simeck-48-96**, improving upon the previous best differential attack by one round. Additionally, we propose a 35-round attack on **Simeck-64-128**, matching the best-known differential results on this variant.
- Notably, most of our attacks require less data than the full codebook, highlighting their practical efficiency.

Table 1 summarizes the details of our best differential MITM attacks across various variants of **SIMON** and **Simeck**. It is important to note that we are not claiming here that the differential MITM attack is the strongest attack we find on **SIMON**, and **Simeck** across all existing cryptanalytic techniques. However, here our main goal is to create an efficient, easy-to-use, bit-wise automatic tool specifically for finding optimized differential MITM attacks on AndRX ciphers, as identifying such attacks manually on bit-oriented ciphers with large block sizes can be a complex and error-prone procedure. One significant benefit of our tool is that, while our bit-wise automatic tool is applied to AndRX ciphers, it can be extended to other bit-oriented ciphers (e.g., SPN) to construct differential MITM attacks. Additionally, our tool offers the flexibility to integrate with various state-of-the-art solvers, including CP, SAT/SMT, and MILP solvers, further enhancing its efficiency and applicability.

1.2 Organization.

We begin by providing an overview of differential MITM attacks and a concise explanation of how to find optimal distinguishers in **SIMON** and **Simeck** in Section 2. This section also outlines our CP model used to identify optimal distinguishers for these ciphers. Following this, in Section 3, we present a comprehensive description of our bit-wise automated model for identifying differential MITM attacks, along with a detailed explanation of how we built the automated tool for **SIMON**. In Section 4, we discuss the practical application of our methods. Lastly, Section 5 summarizes our findings and highlights potential future research directions.

2 Background

This section concisely outlines the differential MITM attack methodology and its computational complexity. We further examine techniques for generating optimal differential trails in **SIMON** and **Simeck**.

Table 1: Summary of our attacks on SIMON and Simeck in the single-key model.

Cipher	Attack	(Dist., p)	#R	Time	Data	Mem.	Ref.
SIMON-32-64	Differential	-	22	$2^{58.76}$	2^{32}	-	[QHS16]
	MITM	-	18	$2^{62.57}$	2^3	-	[SHMS14]
	$\mathcal{DS}$ -MITM	-	16	$2^{56.29}$	2^{30}	-	[LSG ⁺ 23]
	Diff-MITM [*]	(11, 31)	20	2^{63}	2^{32}	$2^{52.55}$	Appendix B
	Diff-MITM [†]	(14, 30.81)	23	$2^{62.81}$	2^{32}	$2^{43.409}$	Appendix B
SIMON-48-72	Differential	-	23	$2^{63.25}$	2^{47}	-	[WWJZ18]
	MITM	-	17	$2^{71.75}$	2^3	-	[SHMS14]
	$\mathcal{DS}$ -MITM	-	16	$2^{63.24}$	2^{44}	-	[LSG ⁺ 23]
	Diff-MITM [*]	(15, 46)	22	2^{70}	2^{47}	$2^{52.571}$	Appendix B
	Diff-MITM [†]	(17, 46.32)	23	$2^{70.32}$	2^{48}	$2^{36.286}$	Appendix B
SIMON-48-96	Differential	-	24	$2^{78.99}$	2^{48}	-	[WWJZ18]
	MITM	-	19	$2^{95.26}$	2^3	-	[SHMS14]
	$\mathcal{DS}$ -MITM	-	18	$2^{91.62}$	2^{45}	-	[LSG ⁺ 23]
	Diff-MITM [*]	(15, 46)	24	2^{94}	2^{47}	$2^{74.128}$	Appendix B
	Diff-MITM [†]	(17, 46.32)	25	$2^{94.32}$	2^{48}	$2^{57.946}$	Appendix B
SIMON-64-96	Differential	-	30	2^{88}	$2^{63.3}$	-	[WWJZ18]
	MITM	-	17	$2^{94.05}$	2^3	-	[SHMS14]
	$\mathcal{DS}$ -MITM	-	18	$2^{95.94}$	2^{58}	-	[LSG ⁺ 23]
	Diff-MITM [*]	(18, 62)	25	2^{94}	2^{63}	$2^{48.107}$	Appendix B
	Diff-MITM [†]	(23, 61.93)	30	$2^{93.93}$	2^{63}	$2^{59.22}$	Appendix B
SIMON-64-128	Differential	-	31	2^{120}	$2^{63.3}$	-	[WWJZ18]
	MITM	-	19	$2^{126.01}$	2^3	-	[SHMS14]
	$\mathcal{DS}$ -MITM	-	19	$2^{100.94}$	2^{58}	-	[LSG ⁺ 23]
	Diff-MITM [*]	(18, 62)	28	2^{126}	2^{63}	$2^{81.419}$	Appendix B
	Diff-MITM [†]	(23, 61.93)	32	$2^{125.93}$	2^{63}	$2^{90.493}$	Appendix B
SIMON-96-144	Differential	-	37	$2^{130.75}$	2^{95}	-	[WWJZ18]
	MITM	-	21	$2^{141.27}$	2^4	-	[SHMS14]
	$\mathcal{DS}$ -MITM	-	20	$2^{111.90}$	2^{36}	-	[LSG ⁺ 23]
	Diff-MITM [*]	(27, 90)	34	2^{138}	2^{93}	$2^{77.763}$	Appendix B
	Diff-MITM [†]	(31, 95.34)	38	$2^{143.34}$	2^{96}	$2^{80.763}$	Appendix B
SIMON-128-256	Differential	-	51	$2^{247.17}$	2^{127}	-	[WWJZ18]
	MITM	-	25	$2^{253.94}$	2^3	-	[SHMS14]
	$\mathcal{DS}$ -MITM	-	26	$2^{250.08}$	2^{53}	-	[LSG ⁺ 23]
	Diff-MITM [*]	(36, 126)	48	2^{254}	2^{127}	$2^{188.542}$	Appendix B
	Diff-MITM [†]	(41, 123.74)	51	$2^{251.74}$	2^{126}	$2^{148.542}$	Appendix B
Simeck-48-96	Differential	-	28	$2^{68.3}$	2^{46}	-	[QHS16]
	Diff-MITM [*]	(18, 46)	27	2^{94}	2^{47}	2^{35}	Appendix B
	Diff-MITM [†]	(21, 45.28)	29	$2^{93.28}$	2^{47}	2^{36}	Appendix B
Simeck-64-128	Differential	-	35	$2^{116.3}$	2^{63}	-	[QHS16]
	Diff-MITM [*]	(24, 62)	33	2^{126}	2^{63}	2^{61}	Appendix B
	Diff-MITM [†]	(27, 61.49)	35	$2^{125.49}$	2^{63}	2^{39}	Appendix B

Dist.: Distinguisher length. **p :** Distinguisher probability (2^{-p}). **#R:** Total attacked rounds.

^{*}: Based on the optimal single differential trail (Table 4).

[†]: Based on optimal differentials (Table 5).

2.1 Revisited Description of Differential MITM Attack

Building upon foundational cryptanalytic methods, Boura et al. [BDD⁺23] introduced a hybrid approach that integrates differential cryptanalysis with conventional MITM techniques. This differential MITM framework extends classical MITM attacks while demonstrating superior cryptanalytic efficiency.

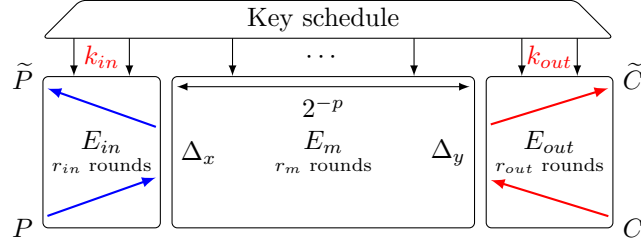


Figure 1: Overview and parameters of differential MITM attacks.

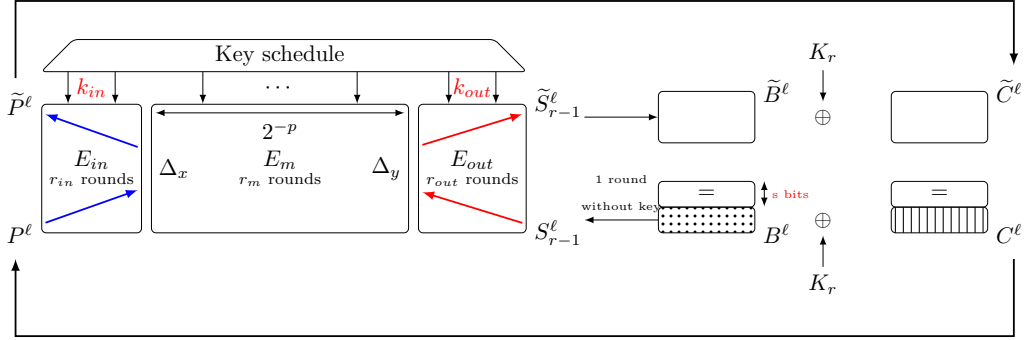


Figure 2: Overview and parameters of differential MITM attacks with parallel partitioning technique.

Attack Structure and Core Principle The n -bit cipher E is decomposed into three consecutive sub-ciphers: $E_{out} \circ E_m \circ E_{in}$, where r_{in} , r_m , and r_{out} represent round counts for each segment (illustrated in Figure 1). Let Δ_x and Δ_y denote input and output differences for E_m , with the differential $\Delta_x \rightarrow \Delta_y$ holding over r_m rounds with probability 2^{-p} . The attack is initiated by:

- Selecting plaintext-ciphertext pair (P, C)
- Generating pair $(\tilde{P}, \tilde{C})$ satisfying the differential trail

Key Recovery Mechanism The attack comprises three coordinated phases:

- *Upper Path Processing (E_{in}):*
 - Guess minimal key material k_{in}
 - For each guess i , compute $\tilde{P}^i$ satisfying, $E_{in}(P) \oplus E_{in}(\tilde{P}^i) = \Delta_x$
 - Compute $\tilde{C}^i = E(\tilde{P}^i)$ for all $2^{|k_{in}|}$ candidates
 - Store $(\tilde{C}^i, i)$ in hash table H
- *Lower Path Processing (E_{out}):*
 - For ciphertext C , guess key material k_{out}
 - For each guess j , derive $\tilde{C}^j$ satisfying $E^{-1}out(C) \oplus E^{-1}out(\tilde{C}^j) = \Delta_y$
 - Generate $2^{|k_{out}|}$ candidates $\tilde{C}^j$
- *Matching Phase:*
 - Execute upper/lower paths 2^p times using distinct (P, C) pairs

- Check collisions where $\widehat{C}_\ell^i = \widetilde{C}_\ell^j$ via table H
- Valid collisions suggest candidate keys ($k_{in} = i, k_{in} = j$)
- Expected collisions: $2^{|k_{in}|+|k_{out}|-n-|k_{in}\cap k_{out}|}$

Complexity Analysis The attack’s efficiency is quantified through three critical metrics:

- *Time Complexity* ($\mathcal{T}$): The time complexity comprises three components:
 - The complexity of the upper and lower paths are $2^p(2^{|k_{in}|} + 2^{|k_{out}|})$
 - $2^{|k_{in}|+|k_{out}|-|k_{in}\cap k_{out}|-n+p}$ is the expected number of candidate keys after matching phase
 - When $|k_{in}|+|k_{out}|-n-|k_{in}\cap k_{out}|+p > 0$, additional cost 2^{k-n+p} for exhaustive verification of partial keys against remaining key space.

Then, the total time complexity:

$$\mathcal{T} = 2^p \times \left(2^{|k_{in}|} + 2^{|k_{out}|} \right) + 2^{|k_{in}\cup k_{out}|-n+p} + 2^{k-n+p}$$

- *Data Complexity* ($\mathcal{D}$):

$$\mathcal{D} = \min \left(2^n, 2^{p+\min(|k_{in}|, |k_{out}|)} \right)$$

- Lower bound $2^{p+\min(|k_{in}|, |k_{out}|)}$ ensures sufficient pairs satisfy the differential
- Upper bound 2^n reflects block size limitations
- Dictates plaintext-ciphertext requirements

- *Memory Complexity* ($\mathcal{M}$):

$$\mathcal{M} = 2^{\min(|k_{in}|-|k_{in}\cap k_{out}|, |k_{out}|-|k_{in}\cap k_{out}|)}$$

- Determined by the storage of intermediate states
- Optimization by selecting a minimal independent key set:
 - * Upper path: $2^{|k_{in}|-|k_{in}\cap k_{out}|}$
 - * Lower path: $2^{|k_{out}|-|k_{in}\cap k_{out}|}$
- Represents the dominant memory component

Optimization Advantage This approach reduces effective key space through simultaneous path computation and probabilistic matching, with complexity advantages becoming pronounced when:

- $|k_{in} \cap k_{out}|$ is maximized (involved key bits)
- Differential probability 2^{-p} is sufficiently high

The complete framework provides cryptanalysts with a systematic methodology for evaluating cipher security against advanced combined attacks.

2.2 Improvement to General Framework of Differential MITM Attack

In this section, we briefly revisit two key ideas originally proposed in [BDD⁺23] to enhance the general framework of differential MITM attacks. Specifically, these techniques are known as the *parallel partitioning method* and the *data reduction method using imposed conditions*.

2.2.1 Parallel Partitioning Technique [BDD⁺23]

In 2023, Boura et al.'s work [BDD⁺23] introduced a parallel partitioning approach to enhance differential MITM attacks. This technique enables the inclusion of an extra round, either at the start or at the end of the attack, under specific cipher design constraints. It is particularly applicable to block ciphers where the round key is combined (e.g., via XOR) with only a subset of the internal state bits during key addition. For such ciphers, like SIMON [BSS⁺15], Simeck [YZS⁺15], and GIFT [BPP⁺17], the differential MITM attack can effectively gain coverage of one additional round.

Consider a block cipher with an n -bit block size where the secret key affects only $s < n$ bits of the internal state during each key addition (typically via XOR operation). In this configuration, an existing differential MITM attack targeting $(r-1)$ rounds can be extended by one final round. Applying this technique thus elevates the attack to encompass r rounds of the cipher. The structure of this enhanced differential MITM attack is depicted in Figure 2. Let S_{r-1} denote the final state after the $(r-1)$ -th round in a basic differential MITM attack. Let B represent the intermediate state obtained after applying the r -th round function to S_{r-1} , excluding the XOR with the round subkey. Finally, let C denote the state resulting from xoring B with the r th round subkey K_r . As the subkey is XORed only in the s bits of the state, then the remaining $(n-s)$ bits of the states B and C should be the same. The core methodology involves selecting ciphertexts that share a fixed constant value across the $(n-s)$ bits unaffected by key addition. For these texts, we generate the full set of 2^s possible values for both C (each of which we denote it as C^ℓ , $1 \leq \ell \leq 2^s$) and B (each of which we denote it as B^ℓ , $1 \leq \ell \leq 2^s$). Utilizing these values, one can construct hash tables to facilitate collision detection. To implement this enhancement without raising the time complexity (when $p > s$), the core attack process must be executed 2^{p-s} times. However, it should be noted that this repetition could lead to higher memory requirements.

From the state portion impacted by key addition, we consider all 2^s possible values of B (denoted B^ℓ). Each B^ℓ value allows direct computation of S_{r-1}^ℓ , representing the final state from the $(r-1)$ -round differential MITM attack. For each state S_{r-1}^ℓ (the output of the $(r-1)$ -round attack) and every subkey guess j of the $|k_{out}|$ bits in k_{out} , we compute a modified state $\tilde{S}_{r-1}^{\ell,j}$. This state satisfies the difference propagation condition:

$$E_{out}^{-1}(S_{r-1}^\ell) \oplus E_{out}^{-1}(\tilde{S}_{r-1}^{\ell,j}) = \Delta_y$$

where E_{out} represents the lower part of the core attack. This process generates $2^{|k_{out}|+s}$ tuples $(B^\ell, \tilde{B}^{\ell,j}, j)$.

Similarly, each ciphertext C^ℓ is decrypted to plaintext P^ℓ . For every P^ℓ and subkey guess i of the $|k_{in}|$ bits in k_{in} , we derive associated plaintext $\tilde{P}^{\ell,i}$ satisfying:

$$E_{in}(P^\ell) \oplus E_{in}(\tilde{P}^{\ell,i}) = \Delta_x$$

where E_{in} is the inner transformation, yielding $2^{k_{in}+s}$ tuples $(C^\ell, \tilde{C}^{\ell,i}, i)$. The initial collision space has size $2^{|k_{in}|+|k_{out}|+2s}$, but three filtering conditions apply:

- *Key-Addition Consistency (s -bit filter)*: $C^\ell = B^\ell \oplus K_r$
- *State-Matching Consistency (n -bit filter)*: $\tilde{C}^{\ell,i} = \tilde{B}^{\ell,j} \oplus K_r$
- *Subkey Overlap ($|k_{in} \cap k_{out}|$ -bit filter)*: Matching shared bits between k_{in} and k_{out}

For one structure, the filtered candidate keys number $2^{|k_{in}|+|k_{out}|-|k_{in} \cap k_{out}|+s-n}$. When repeated 2^{p-s} times, the total candidate count simplifies to:

$$2^{|k_{in}|+|k_{out}|-|k_{in} \cap k_{out}|-n+p}$$

This matches the $(r - 1)$ -round core differential MITM attack's (without the parallel partitioning technique) time complexity. Thus, the parallel partitioning technique extends the $(r - 1)$ -round differential MITM attack to r rounds without increasing time complexity.

2.2.2 Reducing Data with Imposed Condition [BDD⁺23]

In [BDD⁺23], the authors presented an approach to lower the data complexity in differential MITM attacks. Specifically, this technique is applied in scenarios where access to the full codebook is necessary. The core technique restricts P and $\tilde{P}$ to 2^{n-x} possible plaintexts (instead of the full codebook) by predefining specific values for x bits in each. The plaintext P can be selected at will, but the associated plaintext $\tilde{P}$ satisfies the x -bit constraint with probability 2^{-x} . The attack probability, therefore, becomes 2^{-p-x} , given the need to repeatedly execute the method until finding a pair $(P, \tilde{P})$ that matches the differential trail. Importantly, tuples $(P, \tilde{P}, i)$ and $(C, \tilde{C}, j)$ that fail the conditions—constituting 2^x instances—are discarded in upper/lower attack phases. Consequently, both data and memory complexities experience a reduction factor of 2^x . By integrating this approach with the standard differential MITM attack parameters, we obtain the following two constraints governing the number of fixed bits x :

$$\begin{cases} p + x \leq n - x \\ 2^{p+x}(2^{|k_{in}|} + 2^{|k_{out}|}) < 2^k \end{cases}$$

A key benefit of incorporating this idea into the differential MITM attack is the reduction in data and memory complexity, achieved without adversely affecting the time complexity. By efficiently reducing the number of potential keys for the input and output parts to $2^{|k_{in}|-x}$ and $2^{|k_{out}|-x}$ respectively, and assuming these reduced sets of candidates (along with their corresponding plaintexts $\tilde{P}$ or ciphertexts $\tilde{C}$) can be found with computational effort proportional to their size, the overall time complexity of the attack becomes:

$$2^{p+x}(2^{|k_{in}|-x} + 2^{|k_{out}|-x}) = 2^p(2^{|k_{in}|} + 2^{|k_{out}|})$$

This demonstrates that the data complexity can be lowered to 2^{n-x} without increasing the time complexity compared to the original scenario. Crucially, this balance between data requirements, computational time, and memory usage remains intact when applying parallel partitioning to the differential MITM attack.

2.3 Brief Specification of AndRX Ciphers

2.3.1 Specification of SIMON

Designed for resource-constrained environments, the SIMON [BSS⁺15] block cipher employs a balanced Feistel structure optimized for hardware efficiency. Following the ARX paradigm (combining **Addition**, **Rotation**, and **XOR** operations) seen in designs like ThreeFish and BLAKE, SIMON's round function utilizes only three elementary operations: bitwise **AND**, **XOR**, and circular bit **Rotation**. This minimalist approach ensures exceptional hardware performance while maintaining cryptographic strength.

The cipher supports multiple configurations parameterized by word size (n bits) and key expansion ratio (m). With a block size of $2n$ bits where $n \in 16, 24, 32, 48, 64$, and key sizes of $m \times n$ bits for $m \in 2, 3, 4$, each variant is denoted as **SIMON2n/mn**. The round count for every configuration is determined through security analysis based on both block and key dimensions, with specific values detailed in Table 2. Each round starts with the state represented by two words: (L_i, R_i) . The left word L_i is passed through a round function F , defined as:

$$F(L_i) = (L_i \lll 1) \wedge (L_i \lll 8) \oplus (L_i \lll 2),$$

Table 2: SIMON parameters

Variant	Block size	Key size	Word size	Key words	Round
SIMON32	32	64	16	4	32
SIMON48	48	72 96	24	3 4	36 36
SIMON64	64	96 128	32	3 4	42 44
SIMON96	96	96 144	48	2 3	52 54
SIMON128	128	128 192 256	64	2 3 4	68 69 72

where $\lll$ denotes left rotation. The result of $F(L_i)$ is XORed with R_i and the subkey K_i , and then the two words are swapped:

$$L_{i+1} = R_i \oplus F(L_i) \oplus K_i,$$

$$R_{i+1} = L_i.$$

After all rounds are completed, the final state (L_r, R_r) forms the ciphertext. The round function of SIMON is illustrated in Figure 3a. SIMON uses a linear feedback shift register (LFSR)-like process to generate the required r subkeys $K_0, K_1, \dots, K_{r-1}$. Depending on the length of the secret key (which can consist of 2, 3, or 4 words), Simon employs slightly different key schedule procedures.

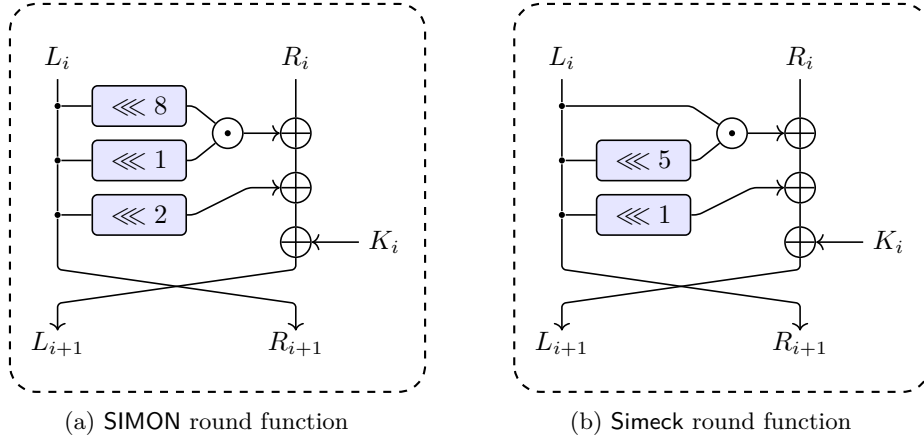


Figure 3: Round Functions of AndRX Ciphers

The key expansion process initializes the first w words (K_0 through K_{w-1} , where $w \in \{2, 3, 4\}$) directly from the master key. Subsequent round keys K_i for $i = w$ to $r - 1$ are then derived through the recurrence relation:

- For $w = 2$:

$$K_{i+2} = K_i \oplus (K_{i+1} \ggg 3) \oplus (K_{i+1} \ggg 1) \oplus c \oplus (z_j)_i,$$

- For $w = 3$:

$$K_{i+3} = K_i \oplus (K_{i+1} \ggg 3) \oplus (K_{i+2} \ggg 1) \oplus c \oplus (z_j)_i,$$

- For $w = 4$:

$$K_{i+4} = K_i \oplus (K_{i+1} \ggg 3) \oplus (K_{i+3} \ggg 1) \oplus c \oplus (z_j)_i,$$

where $c = 0xf f \dots f c$ is a fixed constant, and $(z_j)_i$ refers to the i -th bit of one of five predefined constant sequences z_0, z_1, z_2, z_3, z_4 .

2.3.2 Specification of Simeck

The Simeck [YZS⁺15] family of lightweight block ciphers was designed to be hardware-efficient while maintaining strong security. Simeck adopts a simple and compact structure inspired by the SIMON cipher and employs bitwise operations for both encryption and key scheduling. Its primary goal is to provide a small footprint in hardware, making it suitable for constrained environments like IoT devices and embedded systems. The Simeck family includes variants with different block and key sizes: Simeck-32-64, Simeck-48-96, and Simeck-64-128.

Simeck is structured as a balanced Feistel network, where the state is divided into two equal-sized words (L_i, R_i) at the beginning of each round. The round function of Simeck is illustrated in Figure 3b. The round function is designed to be simple and hardware-friendly, operating as follows:

$$f(x) = (x \wedge (x \lll 5)) \oplus (x \lll 1),$$

where $\lll$ denotes a left cyclic shift, $\wedge$ represents bitwise AND, and $\oplus$ is the XOR operation. This function is applied to the left half of the state (L_i) , after which the right half (R_i) is XORed with the result of $f(L_i)$ and the subkey K_i . The round operation can be described by:

$$L_{i+1} = R_i \oplus f(L_i) \oplus K_i,$$

$$R_{i+1} = L_i.$$

This process is repeated over T rounds, where T depends on the block size of the Simeck variant in use. The final state after the last round provides the ciphertext.

Simeck uses a compact and efficient key schedule that reuses the round function to generate the required subkeys. The master key is divided into multiple words depending on the key size. A linear feedback shift register (LFSR) structure is used to update the keywords and generate new subkeys for each round. The update rule for the key schedule is given by:

$$K_{i+t} = K_i \oplus (K_{i+t-1} \ggg 3) \oplus (K_{i+t-1} \ggg 1) \oplus z_j,$$

where $\ggg$ denotes a right shift, and z_j is a constant derived from a predefined sequence. This key schedule ensures that subkeys are generated efficiently, minimizing hardware requirements.

2.4 Optimal Differential Trails in SIMON-like Ciphers

In this section, we recall the automatic search algorithm for identifying optimal differential trails in SIMON-like ciphers. SIMON [BSS⁺15] and Simeck [YZS⁺15] both based on the Feistel construction and their round functions utilize three primary operations: bitwise AND ($x \odot y$), bitwise rotation ($x \lll n$), and XOR ($x \oplus y$). Therefore, they are often referred to as AndRX ciphers; their detailed descriptions can be found in Subsection 2.3.

Differential cryptanalysis stands as a foundational method in symmetric-key cryptanalysis. Numerous cryptanalytic findings for SIMON have been established through this approach [BRV15, ALLW15, KLT15, LLW17, WWHL18]. Interestingly, Kölbl et al. [KLT15] derived a closed-form expression for the differential probability of SIMON-like round functions, subsequently employing SAT/SMT solvers to discover optimal differential trails. This approach was later extended to Simeck in [KR16]. However, they did not present optimal differential trails for larger variants, such as SIMON-64, SIMON-96, and SIMON-128.

Additionally, finding optimal trails for SIMON-like ciphers using SAT/SMT solvers can be time-intensive, particularly for larger block sizes like 96 and 128 bits.

Beierle [Bei16] established foundational security bounds for SIMON-like ciphers by deriving upper probability limits for differential trails and presenting the first theoretical security proofs for multiple cipher variants. Extending this research, Liu et al. [LLW17] developed tighter differential probability bounds for the SIMON round function through analytical refinements incorporating insights from [KLT15, Bei16]. Their work additionally introduced an optimized branch-and-bound algorithm [Mat94b] for automated discovery of optimal differential trails in SIMON-like designs, significantly advancing search efficiency for these cryptographic primitives.

By focusing on input differences with low Hamming weight, the algorithm efficiently prunes branches that do not meet the criteria, significantly reducing the search space. Using their algorithm, the authors in [LLW17] reported optimal differential trails on 12-, 16-, 19-, 28-, and 37- rounds of SIMON-32, SIMON-48, SIMON-64, SIMON-96, and SIMON-128, respectively.

To analyze a differential characteristic, it is important to calculate the differential’s overall probability, which can be significantly higher than the probability of an individual trail. Building upon their discovery of optimal differential trails for SIMON and Simeck, Liu et al. [LLW17] extended their analysis to differential (composite trail) identification. Their automated framework first generated multiple differential trails, then systematically explored additional trails exhibiting identical input-output difference pairs. By aggregating the probabilities of all compatible trails, they computed the composite differential probability as follows:

$$\Pr(\Delta_x \xrightarrow{f^r} \Delta_y) = \sum_{i=w_{min}}^{w_{max}} s_i \cdot 2^{-i}$$

where s_i is the number of characteristics with a probability of 2^{-i} . Moreover, w_{min} and w_{max} are the lower and upper bounds of the probability of the differential characteristics, and f^r is the r -round function of SIMON-like ciphers.

Wang et al. [WWHL18] developed precise MILP models for SIMON-like ciphers through detailed analysis of the **Rotation-AND** operation. Their research revealed how input difference configurations—specifically active bit positions—directly influence both output differences and differential characteristic probabilities. By exhaustively mapping input-output difference relationships, they derived exact probability distributions for all possible output differences given any input difference. Leveraging this framework, they replicated prior optimal single-key differential trail results [LLW17] while additionally discovering new optimal related-key differential trails for SIMON and Simeck variants.

2.4.1 Constructing CP Models for SIMON and Simeck.

Building on the model described in [WWHL18], we construct a CP model to find single-key optimal differential trails in SIMON and Simeck. The following theorem establishes the relationship between the input and output difference of the **Rotation-AND** operations.

Theorem 1 ([WWHL18]). *Let $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be such that $f(x) = (x \lll a) \odot (x \lll b)$, and $\gcd(n, a - b) = 1$. Let $\delta x, \delta y \in \mathbb{F}_2^n$ be the input and output difference of f , respectively, with $wt(\delta x) = m$, $m < n$, and $R = \{\delta x_{i_0}, \delta x_{i_1}, \dots, \delta x_{i_{m-1}}\}$ be the set of all active bits in δx . If there exists*

1. p_1 pairs of $\{i_j, i_k\}$ such that $|i_j - i_k| \equiv |a - b| \pmod{n}$; and

2. p_2 pairs of $\{i_j, i_k\}$ such that $|i_j - i_k| \equiv 2|a - b| \pmod{n}$ and there exists some h such that $|h - i_j| \equiv |a - b| \pmod{n}$, $|h - i_k| \equiv |a - b| \pmod{n}$, $\delta x_h \notin R$;

then there are $2^{2m-p_1-p_2}$ possible values for δy , and each has the same probability of $2^{-2m+p_1+p_2}$.

Now, based on **Theorem 1**, we can state the following proposition to model the differential trails through the **Rotation-AND** operation.

Proposition 1 (Rotation-AND). Let $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be such that $f(x) = (x \lll a) \odot (x \lll b)$, and $\gcd(n, a - b) = 1$. Let $\mathbf{x}, \mathbf{y} \in \mathbb{F}_2^n$ be the n -bit binary variables to encode the input and output difference with respect to the function f . Furthermore, we divide the n bits input difference and n bits output difference of f into n groups, where group i consists of three input difference bits at positions $(i, i + t \pmod{n}, i + 2t \pmod{n})$ and two output difference bits at positions $(i - b \pmod{n}, i + t - b \pmod{n})$, where $t = |a - b|$, and $i \in \{0, 1, \dots, n-1\}$. For each group index i satisfying $0 \leq i \leq n-1$, the constraint programming formulation defines all admissible differential transitions through the following CP constraints:

$$\text{group}_i(\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2}, \mathbf{y}_{i_3}, \mathbf{y}_{i_4}) :=$$

Condition	Assignment
$(\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2}) = (0, 0, 0)$	$\mathbf{y}_{i_3}, \mathbf{y}_{i_4} = (0, 0)$
$\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2} = (0, 0, 1)$	$(\mathbf{y}_{i_3}, \mathbf{y}_{i_4}) = (0, 0) \vee (0, 1)$
$\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2} = (0, 1, 0)$	$\mathbf{y}_{i_3}, \mathbf{y}_{i_4} = (0, 0) \vee (0, 1) \vee (1, 0) \vee (1, 1)$
$(\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2}) = (0, 1, 1)$	$(\mathbf{y}_{i_3}, \mathbf{y}_{i_4}) = (0, 0) \vee (0, 1) \vee (1, 0) \vee (1, 1)$
$(\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2}) = (1, 0, 0)$	$(\mathbf{y}_{i_3}, \mathbf{y}_{i_4}) = (0, 0) \vee (1, 0)$
$\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2} = (1, 0, 1)$	$(\mathbf{y}_{i_3}, \mathbf{y}_{i_4}) = (0, 0) \vee (1, 1)$
$\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2} = (1, 1, 0)$	$(\mathbf{y}_{i_3}, \mathbf{y}_{i_4}) = (0, 0) \vee (0, 1) \vee (1, 0) \vee (1, 1)$
otherwise	$(\mathbf{y}_{i_3}, \mathbf{y}_{i_4}) = (0, 0) \vee (0, 1) \vee (1, 0) \vee (1, 1)$

For notational simplicity, we denote the indices $i + t \pmod{n}$, $i + 2t \pmod{n}$, $i - b \pmod{n}$, and $i + t - b \pmod{n}$ as i_1 , i_2 , i_3 , and i_4 , respectively. Consequently, differential propagation across the **Rotation-AND** operation is governed by the following constraint system:

$$\text{Rotation-AND}(\mathbf{x}, \mathbf{y}) := \left(\bigwedge_{i=0}^{n-1} \text{group}_i(\mathbf{x}_i, \mathbf{x}_{i_1}, \mathbf{x}_{i_2}, \mathbf{y}_{i_3}, \mathbf{y}_{i_4}) \right)$$

The propagation rules for differential trails through **XOR** operations are governed by the following model:

Proposition 2 (XOR). Let $f : \mathbb{F}_2^3 \rightarrow \mathbb{F}_2$ be such that $f(x_0, x_1, x_2) = y$ where $y = x_0 \oplus x_1 \oplus x_2$. Let, $\mathbf{x} = (x_0, x_1, x_2) \in \mathbb{F}_2^3$, and $\mathbf{y} \in \mathbb{F}_2$ be the corresponding variables to encode the difference in x_i and y . The admissible differential propagation through the **XOR** operation is constrained

by the following conditions:

$$\text{XOR}(x_0, x_1, x_2, y) := \left\{ \begin{array}{c|c} \text{Condition} & y \\ \hline \mathbf{x}_0 = 0 \wedge \mathbf{x}_1 = 0 \wedge \mathbf{x}_2 = 0 & 0 \\ \mathbf{x}_0 = 0 \wedge \mathbf{x}_1 = 0 \wedge \mathbf{x}_2 = 1 & 1 \\ \mathbf{x}_0 = 0 \wedge \mathbf{x}_1 = 1 \wedge \mathbf{x}_2 = 0 & 1 \\ \mathbf{x}_0 = 0 \wedge \mathbf{x}_1 = 1 \wedge \mathbf{x}_2 = 1 & 0 \\ \mathbf{x}_0 = 1 \wedge \mathbf{x}_1 = 0 \wedge \mathbf{x}_2 = 0 & 1 \\ \mathbf{x}_0 = 1 \wedge \mathbf{x}_1 = 0 \wedge \mathbf{x}_2 = 1 & 0 \\ \mathbf{x}_0 = 1 \wedge \mathbf{x}_1 = 1 \wedge \mathbf{x}_2 = 1 & 0 \\ \text{otherwise} & 1 \end{array} \right.$$

We formulate the search for an optimal differential trail across r_m rounds of a SIMON-like cipher E_m as a constraint satisfaction problem. The cipher's internal state after round r ($0 \leq r \leq r_m$) is represented as $x_{r,0} \parallel x_{r,1}$, with difference patterns modeled by binary variables $\mathbf{x}_{r,0} \parallel \mathbf{x}_{r,1}$. Each variable $\mathbf{x}_{r,i}[j]$ ($i \in \{0, 1\}$, $0 \leq j < n$) encodes the difference at bit position j of state component $x_{r,i}$. Differential propagation through E_m is governed by the rules in Proposition [Proposition 1](#) and Proposition [Proposition 2](#), formalized in the constraint model $\text{CSP}_m(\mathbf{x}_{0,0}, \mathbf{x}_{0,1}, \dots, \mathbf{x}_{m,0}, \mathbf{x}_{m,1})$.

Furthermore, let the probability of the differential characteristics be 2^{-p} , and our goal is to maximize the probability. Hence, from [Theorem 1](#), we can write the following objective function:

$$\min. \left(\sum_{r=0}^{r_m-1} \sum_{i=0}^{n-1} (2\mathbf{x}_{r,0}[i] - \mathbf{x}_{r,0}[i]\mathbf{x}_{r,0}[i+t] - \mathbf{x}_{r,0}[i]\mathbf{x}_{r,0}[i+2t] + \mathbf{x}_{r,0}[i]\mathbf{x}_{r,0}[i+t]\mathbf{x}_{r,0}[i+2t]) \right)$$

In our CP model, we declare the variable $\mathbf{p}$ such that

$$\mathbf{p} = \left(\sum_{r=0}^{r_m-1} \sum_{i=0}^{n-1} (2\mathbf{x}_{r,0}[i] - \mathbf{x}_{r,0}[i]\mathbf{x}_{r,0}[i+t] - \mathbf{x}_{r,0}[i]\mathbf{x}_{r,0}[i+2t] + \mathbf{x}_{r,0}[i]\mathbf{x}_{r,0}[i+t]\mathbf{x}_{r,0}[i+2t]) \right).$$

The cubic degree of this objective function complicates model solving. To address this challenge, we adopt the grouping strategy from [\[WWHL18\]](#): For each index $0 \leq i \leq n-1$, construct group i containing three input difference bits ($\mathbf{x}_{r,0}[i]$, $\mathbf{x}_{r,0}[i+t]$, $\mathbf{x}_{r,0}[i+2t]$) and auxiliary variable $\mathbf{p}_{r,0}[i]$. This formulation reduces the objective function's degree to linear, enabling efficient solving. The transformed objective function is expressed as:

$$\min. \left(\sum_{r=0}^{r_m-1} \sum_{i=0}^{n-1} (2\mathbf{x}_{r,0}[i] - \mathbf{p}_{r,0}[i]) \right)$$

where the auxiliary variables $\mathbf{p}_{r,0}[i]$, for all $r \in \{0, \dots, r_m-1\}$, and $0 \leq i \leq (n-1)$ satisfy

the following constraints:

$$\text{AUX}(\mathbf{x}_{r,0}[i], \mathbf{x}_{r,0}[i+t], \mathbf{x}_{r,0}[i+2t], \mathbf{p}_{r,0}[i]) :=$$

Condition	$\mathbf{p}_{r,0}[i]$
$\mathbf{x}_{r,0}[i] = 0 \wedge \mathbf{x}_{r,0}[i+t] = 0 \wedge \mathbf{x}_{r,0}[i+2t] = 0$	0
$\mathbf{x}_{r,0}[i] = 0 \wedge \mathbf{x}_{r,0}[i+t] = 0 \wedge \mathbf{x}_{r,0}[i+2t] = 1$	0
$\mathbf{x}_{r,0}[i] = 0 \wedge \mathbf{x}_{r,0}[i+t] = 1 \wedge \mathbf{x}_{r,0}[i+2t] = 0$	0
$\mathbf{x}_{r,0}[i] = 0 \wedge \mathbf{x}_{r,0}[i+t] = 1 \wedge \mathbf{x}_{r,0}[i+2t] = 1$	1
$\mathbf{x}_{r,0}[i] = 1 \wedge \mathbf{x}_{r,0}[i+t] = 0 \wedge \mathbf{x}_{r,0}[i+2t] = 0$	0
$\mathbf{x}_{r,0}[i] = 1 \wedge \mathbf{x}_{r,0}[i+t] = 0 \wedge \mathbf{x}_{r,0}[i+2t] = 1$	1
$\mathbf{x}_{r,0}[i] = 1 \wedge \mathbf{x}_{r,0}[i+t] = 1 \wedge \mathbf{x}_{r,0}[i+2t] = 0$	0
otherwise	1

Application of the aforementioned technique enables successful identification of optimal differential trails for SIMON and Simeck.

2.5 Bit-wise CP Model for Deterministic Trail

In this section, we recall the propagation rules for deterministic trails in the AndRX design, which are essential for constructing the CP model. Specifically, we focus on the propagation rules for the three core operations in AndRX: Branching, XOR, and AND. The propagation rules for these three operations are described below, as presented in [HGSE24, HDE24, CHNE24].

Proposition 3 (Branching (Deterministic) [HGSE24, HDE24, CHNE24]). *For $f : \mathbb{F}_2 \rightarrow \mathbb{F}_2^n$, $f(x) = (y_0, y_1, \dots, y_{n-1})$ where $y_0 = y_1 = \dots = x$, the valid transitions for deterministic differential trails satisfy*

$$\text{Branch}_D(\mathbf{x}, \mathbf{y}_0, \dots, \mathbf{y}_{n-1}) := \bigwedge_{i=0}^{n-1} (\mathbf{y}_i = \mathbf{x}),$$

where the integer variables $\mathbf{x}, \mathbf{y}_i \in \{-1, 0, 1\}$ encode the difference in x, y_i for $0 \leq i \leq n-1$.

Proposition 4 (XOR (Deterministic) [HGSE24, HDE24, CHNE24]). *For $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$, $f(x_0, x_1, \dots, x_{n-1}) = y$, where $y = x_0 \oplus x_1 \oplus \dots \oplus x_{n-1}$, the valid deterministic differential transitions satisfy*

$$\text{XOR}_D(\mathbf{y}, \mathbf{x}_0, \dots, \mathbf{x}_{n-1}) := \begin{cases} \text{if } \bigvee_{i=0}^{n-1} (\mathbf{x}_i = -1) \text{ then } \mathbf{y} = -1 \\ \text{else } \mathbf{y} = \mathbf{x}_0 + \dots + \mathbf{x}_{n-1} \pmod{2} \end{cases}$$

where $\mathbf{y}$, and $\mathbf{x}_i$ are integer variables with the domain of $\{-1, 0, 1\}$ for all $0 \leq i \leq n-1$.

Proposition 5 (AND (Deterministic) [CHNE24]). *Let $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ be such that $y = f(x_0, x_1, \dots, x_{n-1}) = \bigwedge_{i=0}^{n-1} x_i$. Let $\mathbf{x}_i$ and $\mathbf{y}$ be the corresponding integer variables with domain $\{-1, 0, 1\}$ to encode the difference in x_i and y . Then, the valid transitions for deterministic differential trails satisfy*

$$\text{AND}_D(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{n-1}, \mathbf{y}) := \begin{cases} \text{if } \mathbf{x}_0 = \mathbf{x}_1 = \dots = \mathbf{x}_{n-1} = 0 \text{ then } \mathbf{y} = 0 \\ \text{else } \mathbf{y} = -1 \end{cases}$$

In Section 3, we explain how to construct a CP model for finding differential MITM attack in SIMON-like ciphers.

3 Modeling Differential MITM Attack

In this section, we describe the automated CP-based model for searching for the optimized differential MITM attack in bit-oriented ciphers for the first time, best known to us. We first introduce the general framework of the modeling in any bit-oriented cipher. Then, we explain the detailed attack model for AndRX ciphers (SIMON, Simeck), incorporating some optimization techniques to improve the differential MITM attack in AndRX ciphers. The framework is structured around three parameters: r_{in} , r_m , and r_{out} , where r_m represents the length of the distinguisher, while r_{in} and r_{out} denote the lengths of the outer sections, as shown in Figure 1. The goal is to generate an optimal differential MITM attack targeting $r_{tot} = r_{in} + r_m + r_{out}$ rounds. As established in the complexity analysis (Subsection 2.1), the parameters p , $|k_{in}|$, $|k_{out}|$, and $|k_{in} \cup k_{out}|$ fundamentally govern the efficiency of the differential MITM attack. Determining the subkey set k_{out} involves identifying which state bits influence difference propagation during partial decryption through E_{out}^{-1} . Similarly, the set k_{in} is derived by analyzing state bit dependencies during partial encryption via E_{in} . Furthermore, accurately computing $|k_{in} \cup k_{out}|$ necessitates applying specialized optimization techniques: the *equivalent subkey method* for Feistel structure or the *key bridging approach* for other bit-oriented ciphers.

3.1 Constructing Automated Model to find Differential MITM Attack: A Step-by-Step Approach

Our CP-based automated framework systematically models all critical aspects of differential MITM attacks through five interconnected components:

- *Differential Distinguisher Formulation.* We construct a probabilistic differential distinguisher $\Delta_x \rightarrow \Delta_y$ covering r_m rounds with probability 2^{-p} . For AndRX ciphers (e.g., SIMON, Simeck), this is modeled using the specialized approach detailed in Section Subsection 2.4, where binary variables model the differential path through the cipher’s linear and nonlinear components.
- *Deterministic Difference Propagation in Outer Rounds.* We model probability-1 truncated differential propagation in the outer rounds:
 - *Backward propagation:* From Δ_x to plaintext differences
 - *Forward propagation:* From Δ_y to ciphertext differences

Unlike the distinguisher model, which uses binary variables, here we use integer variables with domain $\{-1, 0, 1\}$ to model the difference propagation in outer parts. More precisely,

- 0/1: Fixed difference values
- -1: Unknown difference (0 or 1)

Following the terms used in previous literature [HGSE24, CHNE24], we can call the truncated differential propagation with probability 1 as deterministic differential propagation, and model the deterministic propagation rules through elementary operations (XOR, AND, Branching, S-box) using constraint satisfaction techniques.

- *Subkey Involvement Identification.* We systematically determine and model which key bits affect the differential transitions:
 - *For E_{in} :* Identify state bits whose difference or data values are required to compute $\tilde{P}$ from P such that $E_{in}(P) \oplus E_{in}(\tilde{P}) = \Delta_x$

- For E_{out} : Identify state bits needed to derive $\tilde{C}$ from C satisfying $E_{out}^{-1}(C) \oplus E_{out}^{-1}(\tilde{C}) = \Delta_y$

This process outputs the involved subkey sets k_{in} and k_{out} that directly impact attack complexity.

- *Key Schedule Bridging.* To optimize subkey dependencies, we can model the cipher’s key schedule using:
 - A generalized CP-based key bridging framework [HE22] that discovers linear relationships between subkeys.
 - Custom constraint templates for cipher-specific key schedules

This critical phase computes $|k_{in} \cup k_{out}|$ by revealing shared bits between k_{in} and k_{out} , significantly reducing the effective key space.

- *Complexity Analysis and Objective Function Formulation.* This component models the complexity formulas described in [Subsection 2.1](#) with the following constraints:

$$\begin{cases} d_0 &:= \min\{k_{in}, k_{out}\} + p \\ D &:= \min\{d_0, n\} \\ M &:= \min\{k_{in} - k_{in \cap out}, k_{out} - k_{in \cap out}\}, \quad M < k \\ t_0 &:= \max\{k_{in}, k_{out}\} + p \\ t_1 &:= k_{in \cup out} - n + p \\ t_2 &:= k - n + p \\ T &:= \max\{t_0, t_1, t_2\}, \quad T < k \end{cases} \quad (1)$$

The objective is to minimize the time complexity T . The CP variables k_{in} , k_{out} , $k_{in \cup out}$, and $k_{in \cap out}$ are integer variables corresponding to the key bits involved in the attack, while D , T , and M are real numbers representing data, time, and memory complexity, respectively.

Our tool operates solely on three integer inputs, representing the lengths of $E_{in}(r_{in})$, $E_m(r_m)$, and $E_{out}(r_{out})$. Users are free to explore different length configurations for these components to identify the most effective differential MITM attack on the bit-oriented ciphers.

3.2 Modeling Bit-Wise Differential MITM Attack on SIMON

This section details our bit-wise constraint model for differential MITM attacks on AndRX ciphers, with SIMON serving as the primary case study. We establish an attack framework for $r_{tot} = r_{in} + r_m + r_{out}$ rounds, where:

- r_m : The length of the distinguisher (core probabilistic component)
- r_{in}/r_{out} : Backward/forward extensions on both sides of the distinguisher
- *Internal state representation:* $x_{r,0} \parallel x_{r,1}$ (left/right n -bit words at round r , where $0 \leq r \leq r_{tot}$)

Our methodology constructs an automated constraint system to optimize differential MITM attacks over r_{tot} rounds, as visualized in [Figure 1](#).

3.2.1 Modeling the Distinguisher in SIMON.

We begin by modeling the differential trail through SIMON’s round function (see [Subsubsection 2.3.1](#) for structure details). For each internal state $x_{r_{in}+r,0} \parallel x_{r_{in}+r,1}$, we define

Algorithm 1: CSP_m model of differential propagation through E_m for SIMON

Input: The integer number r_m
Output: CSP_m

```

1 Declare an empty CSP model  $\mathcal{M}$ ;
2 /* Declaration of CP variables */
3  $\mathcal{M}.var \leftarrow \{x_{r,0}[i] \in \{0,1\}, x_{r,1}[i] \in \{0,1\} : 0 \leq r \leq r_m, 0 \leq i \leq (n-1)\};$ 
4  $\mathcal{M}.var \leftarrow \{y_{r,0}[i] \in \{0,1\} : 0 \leq r \leq r_m - 1, 0 \leq i \leq (n-1)\};$ 
5  $\mathcal{M}.var \leftarrow \{z_{r,0}[i] \in \{0,1\} : 0 \leq r \leq r_m - 1, 0 \leq i \leq (n-1)\};$ 
6  $\mathcal{M}.var \leftarrow \{p_{r,0}[i] \in \{0,1\} : 0 \leq r \leq r_m - 1, 0 \leq i \leq (n-1)\};$ 
7  $\mathcal{M}.var \leftarrow 0 \leq p \leq (2n-1);$ 
8 /* Rotation and AND operation */
9 for  $r = 0, \dots, r_m - 1$  do
10    $\mathcal{M}.con \leftarrow z_{r,0} = x_{r,0} \lll 2;$ 
11 end
12 for  $r = 0, \dots, r_m - 1$  do
13    $\mathcal{M}.con \leftarrow \text{Rotation-AND}(x_{r,0}, y_{r,0});$ 
14 end
15 /* XOR operation */
16 for  $r = 0, \dots, r_m - 1, i = 0, \dots, (n-1)$  do
17    $\mathcal{M}.con \leftarrow \text{XOR}(y_{r,0}[i], z_{r,0}[i], x_{r,1}[i], x_{r+1,0}[i]);$ 
18 end
19 /* Equality condition */
20 for  $r = 0, \dots, r_m - 1, i = 0, \dots, (n-1)$  do
21    $\mathcal{M}.con \leftarrow (x_{r,0}[i] = x_{r+1,1}[i]);$ 
22 end
23 /* Probability calculation */
24 for  $r = 0, \dots, r_m - 1, i = 0, \dots, (n-1)$  do
25    $\mathcal{M}.con \leftarrow \text{AUX}(x_{r,0}[i], x_{r,0}[i + t \pmod n], x_{r,0}[i + 2t \pmod n], p_{r,0}[i]);$ 
26 end
27  $\mathcal{M}.con \leftarrow p = \sum_{r=0}^{r_m-1} \sum_{i=0}^{n-1} (2 * x_{r,0}[i] - p_{r,0}[i]);$ 
28 /* Constraints on input-output difference */
29  $\mathcal{M}.con \leftarrow \sum_{i=0}^{n-1} (x_{0,0}[i] + x_{0,1}[i]) \neq 0;$ 
30  $\mathcal{M}.con \leftarrow \sum_{i=0}^{n-1} (x_{r_m,0}[i] + x_{r_m,1}[i]) \neq 0;$ 
31 return  $\mathcal{M}$ ;

```

CP variables $x_{r,0} \parallel x_{r,1}$, where $0 \leq r \leq r_m$. More precisely, each bit position i ($0 \leq i < n$) in these states is represented by binary CP variables $x_{r,0}[i]/x_{r,1}[i]$ (i -th bit corresponding to left/right n -bit words for SIMON) and with domain $\{0,1\}$. This builds upon our existing CP model for optimal differential trails in SIMON-like ciphers (Subsection 2.4), enabling the construction of the core distinguisher model as formalized in algorithm 1.

3.2.2 Deterministic Difference Propagation in Outer Rounds for SIMON

We implement probability-1 difference propagation through the outer rounds using ternary state variables. For each bit in the cipher's state during E_{in}^{-1} (backward from Δ_x to plaintext) and E_{out} (forward from Δ_y to ciphertext), we assign an integer variable with domain $\{-1, 0, 1\}$ where:

- 0: Fixed difference of 0
- 1: Fixed difference of 1
- -1: Unknown difference (0 or 1)

Therefore, we define $\mathbf{dx}_{r,0}^b \parallel \mathbf{dx}_{r,1}^b$ as the CP variables corresponding to the deterministic difference pattern of the internal state $x_{r,0} \parallel x_{r,1}$, where $0 \leq r \leq r_{in}$. Here, $\mathbf{dx}_{r,0}^b[i]$ and $\mathbf{dx}_{r,1}^b[i]$ have a domain of $\{-1, 0, 1\}$, for all $0 \leq r \leq r_{in}$ and $0 \leq i \leq (n-1)$. Similarly, we define $\mathbf{dx}_{r,0}^f \parallel \mathbf{dx}_{r,1}^f$ for the deterministic difference pattern of $x_{r_{in}+r_m+r,0} \parallel x_{r_{in}+r_m+r,1}$,

where $0 \leq r \leq r_{out}$. For these variables, $\mathbf{dx}_{r,0}^f[i]$, and $\mathbf{dx}_{r,1}^f[i]$ also have a domain $\{-1, 0, 1\}$, for all $0 \leq r \leq r_{out}$ and $0 \leq i \leq (n-1)$. From the description, it is clear that the CP variables $\mathbf{dx}_{rin,0}^b \parallel \mathbf{dx}_{rin,1}^b$, and $\mathbf{dx}_{0,0}^f \parallel \mathbf{dx}_{0,1}^f$ depict the input difference Δ_x and output difference Δ_y , respectively.

We apply deterministic propagation rules for core operations—**Branching** (Proposition 3), **XOR** (Proposition 4), and **AND** (Proposition 5)—to compute probability-1 truncated difference propagation paths:

- *Backward propagation:* From Δ_x through E_{in}^{-1} to plaintext differences
- *Forward propagation:* From Δ_y through E_{out} to ciphertext differences

Our CSP_{out}^{dp} model (Algorithm 2) formalizes this propagation for E_{out} . Using identical principles, we construct CSP_{in}^{dp} for E_{in}^{-1} . These models are then unified into a comprehensive constraint system capturing deterministic difference propagation through both the outer rounds of SIMON.

3.2.3 Modeling the Steps to Find Involved Subkey Bits in Outer Parts in SIMON.

In this part, we detect the state bits whose difference or data values need to be determined to construct the plaintext $\tilde{P}$ from P such that $E_{in}(P) \oplus E_{in}(\tilde{P}) = \Delta_x$, and to construct the ciphertext $\tilde{C}$ from C such that $E_{out}^{-1}(C) \oplus E_{out}^{-1}(\tilde{C}) = \Delta_y$. For each state bit, we define two variables with domain $\{0, 1\}$ to indicate whether its difference and data values need to be determined or not. Specifically, the values “0” and “1” represent that the difference or data value does not need to be determined and needs to be determined, respectively. Here, we explain the method to find the involved key bits k_{out} in the lower part through the function E_{out} .

Therefore, we define $\mathbf{kx}_{r,0}^f \parallel \mathbf{kx}_{r,1}^f$ to be the CP variables for the value determination pattern (“0” indicates that the data value does not need to be determined, and “1” indicates that it does) for the internal state $x_{rin+r_m+r,0} \parallel x_{rin+r_m+r,1}$, where $0 \leq r \leq r_{out}$. The variables $\mathbf{kx}_{r,0}^f[i]$ and $\mathbf{kx}_{r,1}^f[i]$ are CP variables with a domain of $\{0, 1\}$, for all $0 \leq r \leq r_{out}$ and $0 \leq i \leq (n-1)$. In a similar manner, we define $\mathbf{kdx}_{r,0}^f \parallel \mathbf{kdx}_{r,1}^f$ for the difference determination pattern (“0” indicates that the difference does not need to be determined and “1” indicates that it does) of $x_{rin+r_m+r,0} \parallel x_{rin+r_m+r,1}$ for $0 \leq r \leq r_{out}$ where $\mathbf{kdx}_{r,0}^f[i]$ and $\mathbf{kdx}_{r,1}^f[i]$ also have a domain of $\{0, 1\}$, for all $0 \leq r \leq r_{out}$ and $0 \leq i \leq (n-1)$.

To identify the state bits whose differences or data values need to be determined, it is essential to trace how these state bits propagate through the basic operations of SIMON, such as XOR and AND. It is important to note that the value determination pattern in round r of E_{out} ($\mathbf{kx}_{r,0}^f \parallel \mathbf{kx}_{r,1}^f$) depends on the difference determination pattern and the value determination pattern of the state bits in round $(r-1)$ (i.e., $\mathbf{kdx}_{r-1,0}^f \parallel \mathbf{kdx}_{r-1,1}^f$ and $\mathbf{kx}_{r-1,0}^f \parallel \mathbf{kx}_{r-1,1}^f$). In contrast, the difference determination patterns of the state bits in round r (i.e., $\mathbf{kdx}_{r,0}^f \parallel \mathbf{kdx}_{r,1}^f$) are determined solely by the deterministic difference patterns of the state bits in round $(r-1)$ (i.e., $\mathbf{dx}_{r-1,0}^f \parallel \mathbf{dx}_{r-1,1}^f$).

Constraint Propagation Rules for XOR Operations in Difference and Data Determination

Consider the XOR operation $z = x \oplus y$ where $x, y, z \in \mathbb{F}_2$. We model differences and data value requirements for each state using the following constraint variables:

- *Difference patterns:* Integer variables $\mathbf{dx}, \mathbf{dy}, \mathbf{dz} \in \{-1, 0, 1\}$ represent truncated differences:
 - 0/1: Fixed difference value

Algorithm 2: CSP_{out}^{dp} model of deterministic differential transition through E_{out} for SIMON

Input: $\text{CSP}_m.\text{var}$, and the integer number r_{out}
Output: CSP_{out}^{dp}

```

1 Declare an empty CSP model  $\mathcal{M}$ ;
2 /* Declaration of CP variables */
3  $\mathcal{M}.\text{var} \leftarrow \{\text{dx}_{r,0}^f[i] \in \{-1, 0, 1\}, \text{dx}_{r,0}^f[i] \in \{-1, 0, 1\} : 0 \leq r \leq r_{out}, 0 \leq i \leq (n-1)\}$ ;
4  $\mathcal{M}.\text{var} \leftarrow \{\text{dy}_{r,0}^f[i] \in \{-1, 0, 1\} : 0 \leq r \leq r_{out} - 1, 0 \leq i \leq (n-1)\}$ ;
5  $\mathcal{M}.\text{var} \leftarrow \{\text{dz}_{r,0}^f[i] \in \{-1, 0, 1\} : 0 \leq r \leq r_{out} - 1, 0 \leq i \leq (n-1)\}$ ;
6  $\mathcal{M}.\text{var} \leftarrow \{\text{dw}_{r,0}^f[i] \in \{-1, 0, 1\} : 0 \leq r \leq r_{out} - 1, 0 \leq i \leq (n-1)\}$ ;
7  $\mathcal{M}.\text{var} \leftarrow \{\text{dp}_{r,0}^f[i] \in \{-1, 0, 1\}, \text{dq}_{r,0}^f[i] \in \{-1, 0, 1\} : 0 \leq r \leq r_{out} - 1, 0 \leq i \leq (n-1)\}$ ;
8 /* Boundary conditions */
9 for  $i = 0, \dots, (n-1)$  do
10    $\mathcal{M}.\text{con} \leftarrow (\text{dx}_{0,0}^f[i] = \text{x}_{rm,0}[i]);$ 
11 end
12 for  $i = 0, \dots, (n-1)$  do
13    $\mathcal{M}.\text{con} \leftarrow (\text{dx}_{0,1}^f[i] = \text{x}_{rm,1}[i]);$ 
14 end
15 /* Rotation */
16 for  $r = 0, \dots, r_{out} - 1$  do
17    $\mathcal{M}.\text{con} \leftarrow \text{dy}_{r,0}^f = \text{dx}_{r,0}^f \lll 8;$ 
18    $\mathcal{M}.\text{con} \leftarrow \text{dz}_{r,0}^f = \text{dx}_{r,0}^f \lll 1;$ 
19    $\mathcal{M}.\text{con} \leftarrow \text{dw}_{r,0}^f = \text{dx}_{r,0}^f \lll 2;$ 
20 end
21 /* AND operation */
22 for  $r = 0, \dots, r_{out} - 1, i = 0, \dots, (n-1)$  do
23    $\mathcal{M}.\text{con} \leftarrow \text{AND}_D(\text{dy}_{r,0}^f[i], \text{dz}_{r,0}^f[i], \text{dp}_{r,0}^f[i]);$ 
24 end
25 /* XOR operation */
26 for  $r = 0, \dots, r_{out} - 1, i = 0, \dots, (n-1)$  do
27    $\mathcal{M}.\text{con} \leftarrow \text{XOR}_D(\text{dp}_{r,0}^f[i], \text{dw}_{r,0}^f[i], \text{dq}_{r,0}^f[i]);$ 
28 end
29 for  $r = 0, \dots, r_{out} - 1, i = 0, \dots, (n-1)$  do
30    $\mathcal{M}.\text{con} \leftarrow \text{XOR}_D(\text{dq}_{r,0}^f[i], \text{dx}_{r,1}^f[i], \text{dx}_{r+1,0}^f[i]);$ 
31 end
32 /* Equality condition */
33 for  $r = 0, \dots, r_{out} - 1, i = 0, \dots, (n-1)$  do
34    $\mathcal{M}.\text{con} \leftarrow (\text{dx}_{r,0}^f[i] = \text{dx}_{r+1,1}^f[i]);$ 
35 end
36 return  $\mathcal{M}$ ;
```

– -1: Unknown difference (0 or 1)

- *Difference determination flags:* Binary variables $\text{kdx}, \text{kdy}, \text{kdz}$ indicate whether actual difference values must be determined
- *Data determination flags:* Binary variables $\text{kx}, \text{ky}, \text{kz}$ indicate whether data values must be determined

Now, we are ready to define the necessary constraints to model whether the difference or data values or both need to be determined through the **XOR** operation.

- *Difference value determination constraints.* The need to determine the difference values of a state depends on the difference patterns of that particular state. For instance, if the difference pattern of a state bit is unknown (denoted by -1), the difference needs to be determined. Otherwise, the value of the difference in that

particular state bit is already known. The constraints determine when difference values must be computed for bits involved in XOR operations:

$$\text{XOR}_{\text{Diff}}(\text{dx}, \text{dy}, \text{dz}, \text{kdx}, \text{kdy}, \text{kdz}) :=$$

Condition	kdx	kdy	kdz
$\text{dz} \neq -1 \wedge \text{dx} = -1 \wedge \text{dy} \geq 0$	1	0	0
$\text{dz} \neq -1 \wedge \text{dx} \geq 0 \wedge \text{dy} = -1$	0	1	0
$\text{dz} \neq -1 \wedge \text{dx} = -1 \wedge \text{dy} = -1$	1	1	0
$\text{dz} \neq -1 \wedge \text{dx} \geq 0 \wedge \text{dy} \geq 0$	0	0	0
$\text{dz} = -1 \wedge \text{dx} = -1 \wedge \text{dy} \geq 0$	1	0	1
$\text{dz} = -1 \wedge \text{dx} \geq 0 \wedge \text{dy} = -1$	0	1	1
$\text{dz} = -1 \wedge \text{dx} = -1 \wedge \text{dy} = -1$	1	1	1
otherwise	0	0	1

- *Data value determination constraints.* In this scenario, if the data value of the output state bit from an XOR operation needs to be determined, it is necessary to determine the data values of both input state bits involved in the XOR operation.

$$\text{XOR}_{\text{Value}}(\text{kz}, \text{kx}, \text{ky}) :=$$

Condition	kx	ky
$\text{kz} = 0$	0	0
otherwise	1	1

Constraint Propagation Rules for AND Operations in Difference and Data Determination

Following similar methodology, we derive constraints to identify state bits requiring difference or data value determination through the AND operation. For the AND operation $z = f(x, y) = x \cdot y$ where $x, y, z \in \mathbb{F}_2$:

- Define integer variables $\text{dx}, \text{dy}, \text{dz} \in \{-1, 0, 1\}$ for the deterministic difference
- Define binary variable kdz indicating whether z 's difference value must be determined
- Define binary variable kz indicating whether z 's data value must be determined

The differential propagation through the AND operation adheres to the fundamental nonlinear relationship:

$$dz = x \cdot dy \oplus y \cdot dx \oplus dx \cdot dy \quad (2)$$

To determine the difference in z , it is evident from Equation 2 that the values of x and y , along with their respective differences, must be known. However, whether the data value of x (resp. y) needs to be determined or not depends on the difference in y (resp. x). For example, if $dy = 0$, the attacker can determine dz without knowing the value of x . Hence, in this scenario, the value of x does not need to be determined. Similarly, if $dx = 0$, knowing the value of y is unnecessary to determine dz implies the value of y does not need to be determined. Additionally, whether the difference in x (resp. y) needs to be determined depends on the known difference pattern in x (resp. y). As previously mentioned, if a state bit's difference pattern is unknown (denoted by -1), the difference must be computed. Furthermore, if the data value in z needs to be determined, the data values in both x and y must also be determined.

More precisely, we systematically examine potential scenarios through the following case analysis:

- If neither the difference nor the data value needs to be determined (i.e., $\mathbf{kdz} = 0$ and $\mathbf{kz} = 0$), then the differences and data values of x and y also do not need to be determined.
- If the difference in z needs to be determined, but the data value in z does not (i.e., $\mathbf{kdz} = 1$ and $\mathbf{kz} = 0$), then whether the differences and data values in x and y need to be determined depends on the difference patterns of x and y (i.e., $\mathbf{dx}$ and $\mathbf{dy}$).
- If the data value in z needs to be determined (i.e., $\mathbf{kz} = 1$), then the data values in x and y must also be determined (i.e., $\mathbf{kx} = 1$, $\mathbf{ky} = 1$). Additionally, whether the differences in x and y need to be determined depends on their respective difference patterns (i.e., $\mathbf{dx}$ and $\mathbf{dy}$).

The following constraints specify state bits necessitating value determination through the AND operation:

$$\text{AND}_{\text{Diff}\cup\text{Value}}(\mathbf{kdz}, \mathbf{kz}, \mathbf{dx}, \mathbf{dy}, \mathbf{kx}, \mathbf{ky}, \mathbf{kdx}, \mathbf{kdy}) :=$$

Condition	$\mathbf{kx}$	$\mathbf{ky}$	$\mathbf{kdx}$	$\mathbf{kdy}$
$\mathbf{kdz} = 0 \wedge \mathbf{kz} = 0$	0	0	0	0
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 0 \wedge \mathbf{dx} = 0 \wedge \mathbf{dy} = 1$	1	0	0	0
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 0 \wedge \mathbf{dx} = 0 \wedge \mathbf{dy} = -1$	1	0	0	1
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 0 \wedge \mathbf{dx} = 1 \wedge \mathbf{dy} = 0$	0	1	0	0
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 0 \wedge \mathbf{dx} = -1 \wedge \mathbf{dy} = 0$	0	1	1	0
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 0 \wedge \mathbf{dx} = 1 \wedge \mathbf{dy} = 1$	1	1	0	0
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 0 \wedge \mathbf{dx} = 1 \wedge \mathbf{dy} = -1$	1	1	0	1
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 0 \wedge \mathbf{dx} = -1 \wedge \mathbf{dy} = 1$	1	1	1	0
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 0 \wedge \mathbf{dx} = -1 \wedge \mathbf{dy} = -1$	1	1	1	1
$\mathbf{kdz} = 0 \wedge \mathbf{kz} = 1 \wedge \mathbf{dx} = -1 \wedge \mathbf{dy} \geq 0$	1	1	1	0
$\mathbf{kdz} = 0 \wedge \mathbf{kz} = 1 \wedge \mathbf{dx} \geq 0 \wedge \mathbf{dy} = -1$	1	1	0	1
$\mathbf{kdz} = 0 \wedge \mathbf{kz} = 1 \wedge \mathbf{dx} = -1 \wedge \mathbf{dy} = -1$	1	1	1	1
$\mathbf{kdz} = 0 \wedge \mathbf{kz} = 1 \wedge \mathbf{dx} \geq 0 \wedge \mathbf{dy} \geq 0$	1	1	0	0
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 1 \wedge \mathbf{dx} = -1 \wedge \mathbf{dy} \geq 0$	1	1	1	0
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 1 \wedge \mathbf{dx} \geq 0 \wedge \mathbf{dy} = -1$	1	1	0	1
$\mathbf{kdz} = 1 \wedge \mathbf{kz} = 1 \wedge \mathbf{dx} \geq 0 \wedge \mathbf{dy} \geq 0$	1	1	0	0
otherwise	1	1	1	1

Optimizing value determination through AND operation In this section, we explain how the number of bits that need to be determined through the AND operation can be further reduced. Consider the scenario where $\mathbf{kdz} = 1$, $\mathbf{kz} = 0$, and $\mathbf{dx} = \mathbf{dy} = 1$. Based on Equation 2, we can express this as

$$dz = x \oplus y \oplus 1$$

In this case, if the attacker aims to determine the difference in z , they would typically need to know the values of both x and y , implying $\mathbf{kx} = 1$ and $\mathbf{ky} = 1$, as outlined in the $\text{AND}_{\text{Diff}\cup\text{Value}}$ function discussed earlier. However, instead of resolving the values at both x and y , it suffices to determine the value of $x \oplus y$ to calculate the difference in z . This allows us to reduce the bit determination from two positions to one, shifting from the basic model to the advanced model.

Although this reduction may lower the number of subkey bits involved in both E_{in} and E_{out} , it offers a way to minimize the key bits required in the attack using this method.

Furthermore, this optimization can be applied to differential MITM attacks on any bit-oriented cipher. It is worth mentioning that we leave the integration of this technique into our automated model as an open challenge for future research.

Identification of involved subkey bits We establish a systematic approach to determine the involved subkey bits k_{in} and k_{out} critical to the attack. For E_{out} :

- Define binary variables $\mathbf{ikf}_r[i] \in \{0, 1\}$ ($0 \leq r \leq r_{out} - 1$, $0 \leq i \leq n - 1$) indicating involvement of the i -th subkey bit in round r
- Introduce value determination variables $\mathbf{kx}_{r,0}^f[i]$ and $\mathbf{kx}_{r,1}^f[i]$ tracking whether the i -th input bits to r -th round function require value determination

Therefore, the subkey involvement follows the constraint:

$$\left\{ \begin{array}{c|c} \text{Condition} & \text{Assignment} \\ \hline \mathbf{kx}_{r,1}^f[i] = 1 & \mathbf{ikf}_r[i] = 1 \\ \text{otherwise} & \mathbf{ikf}_r[i] = 0 \end{array} \right.$$

Symmetrically for E_{in} , we define $\mathbf{ikb}_r[i]$ with identical semantics. Hence, [algorithm 3](#) formalizes this relationship through constraint model CSP_{out}^{isk} , which propagates bit-level requirements to identify all relevant subkey bits.

3.2.4 Modeling Equivalent Subkey Technique in SIMON

The technique of using equivalent subkeys has become a key strategy in various key-recovery attacks. This strategy optimizes key recovery by minimizing guessed subkey bits through equivalent subkey substitution. Initially formalized by Isobe et al. [[IS13b](#)] for Feistel network cryptanalysis, the technique was later adapted by Sun et al. [[SFW15](#)] for zero-correlation attacks on SIMON. Subsequent research extended its application to impossible differential and zero-correlation cryptanalysis of Simeck [[SB18](#), [ZGHL18](#)]. In a recent work [[CHNE24](#)], the authors presented complete impossible differential attacks on SIMON and Simeck, incorporating the equivalent subkey technique into their attack model. Motivated by this fact, we incorporate the idea of an equivalent subkey technique in our model to reduce the number of involved subkey bits ($|k_{in}|$ and $|k_{out}|$).

We begin by explaining the fundamental concept behind the equivalent subkey technique. Then, we outline the steps for integrating this technique into our framework for constructing a differential MITM attack. Lastly, through an example, we demonstrate how the equivalent subkey technique reduces the number of subkeys involved, ultimately lowering the attack's complexities and resulting in a more efficient differential MITM attack. A strategy to reduce the number of guessed subkey bits in key recovery involves shifting the subkey K_i from the i -th round, where $0 \leq i \leq (r_{in} - 1)$, to the $(i+1)$ -th round, creating an equivalent subkey K_{i+1}^e ([Figure 4a](#), and [Figure 4b](#)). Likewise, for $(r_{in} + r_m) \leq i \leq (r_{in} + r_m + r_{out} - 1) = (r_{tot} - 1)$, the subkey K_i can be moved to the $(i-1)$ -th round, producing the equivalent subkey K_{i-1}^e ([Figure 4c](#), and [Figure 4d](#)). For example, the relationships between equivalent subkeys and original subkeys¹ for SIMON ([Figure 4](#)) as presented in [[CHNE24](#)] are as follows:

¹In [Appendix A](#), the expressions of equivalent subkeys in Simeck can be found.

$$\left\{ \begin{array}{l} K_1^e = K_0 \\ K_2^e = (K_1^e \lll 2) \oplus K_1 \\ K_3^e = K_1^e \oplus (K_2^e \lll 2) \oplus K_2 \\ K_4^e = K_2^e \oplus (K_3^e \lll 2) \oplus K_3 \\ \vdots \\ K_{r_{in}-1}^e = K_{r_{in}-3}^e \oplus K_{r_{in}-2}^e \\ \quad \oplus (K_{r_{in}-2}^e \lll 2) \\ K_{r_{in}}^e = K_{r_{in}-2}^e \oplus K_{r_{in}-1}^e \\ \quad \oplus (K_{r_{in}-1}^e \lll 2) \end{array} \right. \quad \left\{ \begin{array}{l} K_{r_{tot}-2}^e = K_{r_{tot}-1}^e \\ K_{r_{tot}-3}^e = (K_{r_{tot}-2}^e \lll 2) \oplus K_{r_{tot}-2}^e \\ K_{r_{tot}-4}^e = K_{r_{tot}-2}^e \oplus K_{r_{tot}-3}^e \\ \quad \oplus (K_{r_{tot}-3}^e \lll 2) \\ K_{r_{tot}-5}^e = K_{r_{tot}-3}^e \oplus K_{r_{tot}-4}^e \\ \quad \oplus (K_{r_{tot}-4}^e \lll 2) \\ \vdots \\ K_{r_{in}+r_m}^e = K_{r_{in}+r_m+2}^e \oplus K_{r_{in}+r_m+1}^e \\ \quad \oplus (K_{r_{in}+r_m+1}^e \lll 2) \\ K_{r_{in}+r_m-1}^e = K_{r_{in}+r_m+1}^e \oplus K_{r_{in}+r_m}^e \\ \quad \oplus (K_{r_{in}+r_m}^e \lll 2) \end{array} \right.$$

In SIMON's E_{out} component, the left state word $x_{r_{in}+r_m+r,0}$ ($0 \leq r \leq r_{out}-2$) designates the equivalent subkey XOR location. The state variables after performing (8,1)-bit left rotations are defined as $y_{r_{in}+r_m+r,0}$ and $z_{r_{in}+r_m+r,0}$, respectively. Additionally, we introduce the binary variables $\mathbf{ky}_{r,0}^f[i]$ and $\mathbf{kz}_{r,0}^f[i]$, which indicate the value determination patterns for the state variables $y_{r_{in}+r_m+r,0}[i]$ and $z_{r_{in}+r_m+r,0}[i]$, for $0 \leq i \leq n-1$. Finally, $\mathbf{ikf}_r^{eq}[i]$ signifies whether the i -th bit of the equivalent subkey in the r -th round of E_{out} is involved, applicable for $0 \leq r \leq r_{out}-2$ and $0 \leq i \leq n-1$. Consequently, the equivalent subkey methodology is formalized through the following constraint system:

$$\left\{ \begin{array}{c|c} \text{Condition} & \text{Assignment} \\ \hline \mathbf{ky}_{r,0}^f[i] = \mathbf{kz}_{r,0}^f[i] = 0 & \mathbf{ikf}_r^{eq}[i] = 0 \\ \text{otherwise} & \mathbf{ikf}_r^{eq}[i] = 1 \end{array} \right.$$

The variables $\mathbf{ky}_{r,0}^f$ and $\mathbf{kz}_{r,0}^f$ represent the state after applying an 8-bit right rotation to $\mathbf{ky}_{r,0}^f$ and a 1-bit right rotation to $\mathbf{kz}_{r,0}^f$, respectively, for all $0 \leq r \leq r_{out}-2$. Similarly, the binary indicator $\mathbf{ikb}_r^{eq}[i]$ denotes activation status for the i -th equivalent subkey bit in round r of E_{in} , where $1 \leq r \leq r_{in}-1$ and $0 \leq i \leq n-1$. Using this, we can calculate the number of involved subkeys (k_{in}) in E_{in} ². The comparison between our model of the differential MITM attack using the equivalent subkey technique and the basic differential MITM attack framework with original subkeys is provided in detail in [Subsubsection 4.1.1](#).

Selective Key Guessing (SKG) Technique in Equivalent Subkey Model For the r -th round of E_{out} , let the state variables corresponding to the two inputs of the AND operation be $y_{r_{in}+r_m+r,0}$ and $z_{r_{in}+r_m+r,0}$, respectively, and the state variables corresponding to the output of AND operation be $p_{r_{in}+r_m+r,0}$ where $0 \leq r \leq r_{out}-1$. Therefore, we can write

$$p_{r_{in}+r_m+r,0}[i] = y_{r_{in}+r_m+r,0}[i] \cdot z_{r_{in}+r_m+r,0}[i], \quad 0 \leq i \leq (n-1)$$

Since the state variable corresponding to the input of the r -th round function of E_{out} is given by $x_{r_{in}+r_m+r,0} \parallel x_{r_{in}+r_m+r,1}$, we can leverage the structure of the SIMON round function to express the following, for all $0 \leq r \leq r_{out}-2$, and $0 \leq i \leq (n-1)$.

$$\left\{ \begin{array}{l} y_{r_{in}+r_m+r,0}[i] = x_{r_{in}+r_m+r,0}[i+8 \pmod n] \oplus K_{r_{in}+r_m+r}^e[i+8 \pmod n] \\ z_{r_{in}+r_m+r,0}[i] = x_{r_{in}+r_m+r,0}[i+1 \pmod n] \oplus K_{r_{in}+r_m+r}^e[i+1 \pmod n] \end{array} \right. \quad (3)$$

²The approach explained here is similar to the approach described in [ZLW⁺23, CHNE24]. Previously, this technique was mainly applied in impossible differential attacks and zero-correlation attacks on Feistel ciphers in the literature. For the first time, we apply this technique in a differential MITM attack framework.

Algorithm 3: CSP_{out}^{isk} model to find k_{out} through E_{out} for SIMON

Input: $\text{CSP}_m.\text{var}$, CSP_{out}^{dp} , and the integer number r_{out}
Output: CSP_{out}^{isk}

```

1 Declare an empty CSP model  $\mathcal{M}$ ;
2 /* Declaration of CP variables */
3  $\mathcal{M}.\text{var} \leftarrow \{\text{kdx}_{r,0}^f[i] \in \{0,1\}, \text{kdxu}_{r,1}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}, 0 \leq i \leq (n-1)\};$ 
4  $\mathcal{M}.\text{var} \leftarrow \{\text{kdy}_{r,0}^f[i], \text{kdz}_{r,0}^f[i], \text{kdw}_{r,0}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}-1, 0 \leq i \leq (n-1)\};$ 
5  $\mathcal{M}.\text{var} \leftarrow \{\text{kdp}_{r,0}^f[i], \text{kq}_{r,0}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}-1, 0 \leq i \leq (n-1)\};$ 
6  $\mathcal{M}.\text{var} \leftarrow \{\text{kdy1}_{r,0}^f[i], \text{kz1}_{r,0}^f[i], \text{kdw1}_{r,0}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}-1, 0 \leq i \leq (n-1)\};$ 
7  $\mathcal{M}.\text{var} \leftarrow \{\text{kx}_{r,0}^f[i] \in \{0,1\}, \text{kx}_{r,1}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}, 0 \leq i \leq (n-1)\};$ 
8  $\mathcal{M}.\text{var} \leftarrow \{\text{ky}_{r,0}^f[i], \text{kz}_{r,0}^f[i], \text{kw}_{r,0}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}-1, 0 \leq i \leq (n-1)\};$ 
9  $\mathcal{M}.\text{var} \leftarrow \{\text{kp}_{r,0}^f[i] \in \{0,1\}, \text{kq}_{r,0}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}-1, 0 \leq i \leq (n-1)\};$ 
10  $\mathcal{M}.\text{var} \leftarrow \{\text{ky1}_{r,0}^f[i], \text{kz1}_{r,0}^f[i], \text{kw1}_{r,0}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}-1, 0 \leq i \leq (n-1)\};$ 
11  $\mathcal{M}.\text{var} \leftarrow \{\text{ikf}_{r,0}^f[i] \in \{0,1\} : 0 \leq r \leq r_{out}-1, 0 \leq i \leq (n-1)\};$ 
12 /* Boundary conditions */
13 for  $i = 0, \dots, (n-1)$  do
14    $\mathcal{M}.\text{con} \leftarrow (\text{if } \text{dx}_{0,0}^f[i] \geq 0 \text{ then } \text{kdx}_{0,0}^f[i] = 0 \text{ else } \text{kdx}_{0,0}^f[i] = 1);$ 
15 end
16 for  $i = 0, \dots, (n-1)$  do
17    $\mathcal{M}.\text{con} \leftarrow (\text{if } \text{dx}_{0,1}^f[i] \geq 0 \text{ then } \text{kdx}_{0,1}^f[i] = 0 \text{ else } \text{kdx}_{0,1}^f[i] = 1);$ 
18 end
19 for  $i = 0, \dots, (n-1)$  do
20    $\mathcal{M}.\text{con} \leftarrow (\text{kx}_{0,0}^f[i] = 0) \cup (\text{kx}_{0,1}^f[i] = 0);$ 
21 end
22 /* Difference and value determination through XOR operation */
23 for  $r = 0, \dots, r_{out}-1, i = 0, \dots, (n-1)$  do
24    $\mathcal{M}.\text{con} \leftarrow \text{XOR}_{Diff}(\text{dx}_{r+1,0}^f[i], \text{dq}_{r,0}^f[i], \text{dx}_{r,1}^f[i], \text{kdx}_{r+1,0}^f[i], \text{kq}_{r,0}^f[i], \text{kdx}_{r,1}^f[i]);$ 
25 end
26 for  $r = 0, \dots, r_{out}-1, i = 0, \dots, (n-1)$  do
27    $\mathcal{M}.\text{con} \leftarrow \text{XOR}_{Diff}(\text{dp}_{r,0}^f[i], \text{dw}_{r,0}^f[i], \text{dq}_{r,0}^f[i], \text{kdp}_{r,0}^f[i], \text{kdw}_{r,0}^f[i], \text{kq}_{r,0}^f[i]);$ 
28 end
29 for  $r = 0, \dots, r_{out}-1, i = 0, \dots, (n-1)$  do
30    $\mathcal{M}.\text{con} \leftarrow \text{XOR}_{Value}(\text{kx}_{r,1}^f[i], \text{kx}_{r+1,0}^f[i], \text{kq}_{r,0}^f[i]) \cup \text{XOR}_{Value}(\text{kq}_{r,0}^f[i], \text{kp}_{r,0}^f[i], \text{kw}_{r,0}^f[i]);$ 
31 end
32 /* Difference and value determination through AND operation */
33 for  $r = 0, \dots, r_{out}-1, i = 0, \dots, (n-1)$  do
34    $\mathcal{M}.\text{con} \leftarrow \text{AND}_{Diff \cup Value}(\text{kdp}_{r,0}^f[i], \text{kp}_{r,0}^f[i], \text{dy}_{r,0}^f[i], \text{dz}_{r,0}^f[i], \text{ky}_{r,0}^f[i], \text{kz}_{r,0}^f[i], \text{kdy}_{r,0}^f[i], \text{kz}_{r,0}^f[i]);$ 
35 end
36 /* Difference and value determination through Rotation operation */
37 for  $r = 0, \dots, r_{out}-1$  do
38    $\mathcal{M}.\text{con} \leftarrow (\text{kdy1}_{r,0}^f = \text{kdy}_{r,0}^f \ggg 8) \cup (\text{kz1}_{r,0}^f = \text{kz}_{r,0}^f \ggg 1) \cup (\text{kdw1}_{r,0}^f = \text{kdw}_{r,0}^f \ggg 2);$ 
39 end
40 for  $r = 0, \dots, r_{out}-1$  do
41    $\mathcal{M}.\text{con} \leftarrow (\text{ky1}_{r,0}^f = \text{ky}_{r,0}^f \ggg 8) \cup (\text{kz1}_{r,0}^f = \text{kz}_{r,0}^f \ggg 1) \cup (\text{kw1}_{r,0}^f = \text{kw}_{r,0}^f \ggg 2);$ 
42 end
43 for  $r = 0, \dots, r_{out}-1, i = 0, \dots, (n-1)$  do
44    $\mathcal{M}.\text{con} \leftarrow (\text{if } \text{kdy1}_{r,0}^f[i] = \text{kz1}_{r,0}^f[i] = \text{kdw1}_{r,0}^f[i] = \text{kdx}_{r,0}^f[i] = 0 \text{ then } \text{kdx}_{r+1,1}^f[i] =$ 
45      $0 \text{ else } \text{kdx}_{r+1,1}^f[i] = 1);$ 
46 end
47 for  $r = 0, \dots, r_{out}-1, i = 0, \dots, (n-1)$  do
48    $\mathcal{M}.\text{con} \leftarrow (\text{if } \text{ky1}_{r,0}^f[i] = \text{kz1}_{r,0}^f[i] = \text{kw1}_{r,0}^f[i] = \text{kx}_{r,0}^f[i] = 0 \text{ then } \text{kx}_{r+1,1}^f[i] = 0 \text{ else } \text{kx}_{r+1,1}^f[i] = 1);$ 
49 end
50 /* Finding involved subkey bits */
51 for  $r = 0, \dots, r_{out}-1, i = 0, \dots, (n-1)$  do
52    $\mathcal{M}.\text{con} \leftarrow (\text{if } \text{kx}_{r,1}^f[i] = 1 \text{ then } \text{ikf}_r[i] = 1 \text{ else } \text{ikf}_r[i] = 0);$ 
53 end
54 return  $\mathcal{M}$ ;

```

Now, the CP variables corresponding to the deterministic difference pattern of the state variables $x_{rin+r_m+r,0}$, $y_{rin+r_m+r,0}$, $z_{rin+r_m+r,0}$, and $p_{rin+r_m+r,0}$ are $\text{dx}_{r,0}^f$, $\text{dy}_{r,0}^f$, $\text{dz}_{r,0}^f$, and $\text{dp}_{r,0}^f$, respectively, for all $0 \leq r \leq r_{out}-1$. Additionally, the CP variables corresponding to the value determination pattern of the state variables $x_{rin+r_m+r,0}$, $y_{rin+r_m+r,0}$, $z_{rin+r_m+r,0}$, and $p_{rin+r_m+r,0}$ are $\text{kx}_{r,0}^f$, $\text{ky}_{r,0}^f$, $\text{kz}_{r,0}^f$, and $\text{kp}_{r,0}^f$, respectively, for all $0 \leq r \leq r_{out}-1$. We

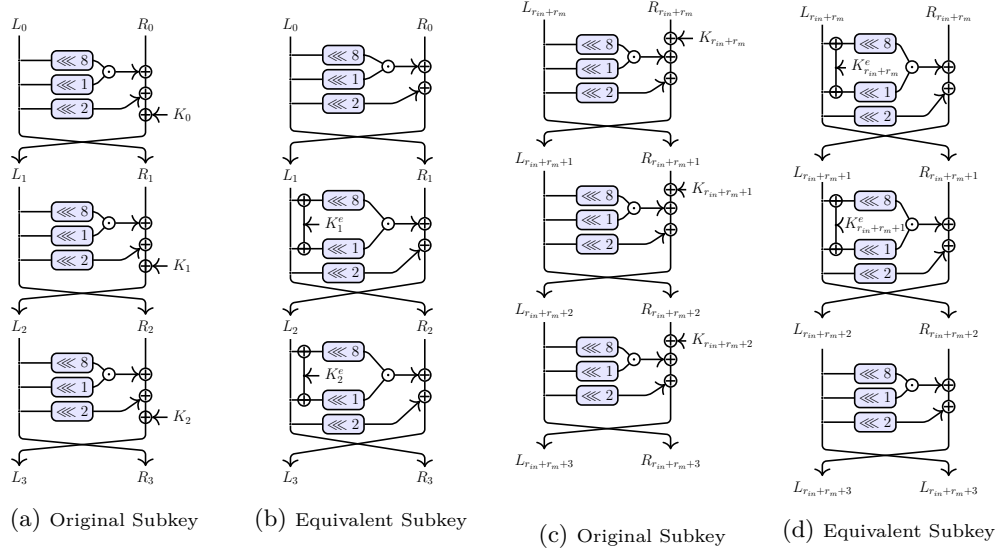


Figure 4: Relationship between original subkey and equivalent subkey in SIMON

define two variables $\tilde{y}_{r_{in}+r_m+r,0}$, and $\tilde{z}_{r_{in}+r_m+r,0}$, such that for all $0 \leq i \leq (n-1)$, the domain of $\tilde{y}_{r_{in}+r_m+r,0}[i]$, and $\tilde{z}_{r_{in}+r_m+r,0}[i]$ is $\{0,1\}$. For all $0 \leq i \leq (n-1)$, suppose $\tilde{y}_{r_{in}+r_m+r,0}[i]$, and $\tilde{z}_{r_{in}+r_m+r,0}[i]$ satisfy the following conditions,

$$\begin{cases} K_{r_{in}+r_m+r}^e[i+8 \pmod n] = \tilde{y}_{r_{in}+r_m+r,0}[i] \\ K_{r_{in}+r_m+r}^e[i+1 \pmod n] = \tilde{z}_{r_{in}+r_m+r,0}[i] \end{cases} \quad (4)$$

Hence, from Equation 3, we can easily conclude that,

$$\begin{cases} y_{r_{in}+r_m+r,0}[i] = x_{r_{in}+r_m+r,0}[i+8 \pmod n] \oplus \tilde{y}_{r_{in}+r_m+r,0}[i] \\ z_{r_{in}+r_m+r,0}[i] = x_{r_{in}+r_m+r,0}[i+1 \pmod n] \oplus \tilde{z}_{r_{in}+r_m+r,0}[i] \end{cases} \quad (5)$$

Moreover, from the specification of SIMON round function, using Equation 4, and Equation 5, we can easily conclude that

$$\begin{cases} \tilde{y}_{r_{in}+r_m+r,0}[i] = \tilde{z}_{r_{in}+r_m+r,0}[i+7 \pmod n] = K_{r_{in}+r_m+r}^e[i+8 \pmod n] \\ \tilde{y}_{r_{in}+r_m+r,0}[i-7 \pmod n] = \tilde{z}_{r_{in}+r_m+r,0}[i] = K_{r_{in}+r_m+r}^e[i+1 \pmod n] \end{cases} \quad (6)$$

We now identify conditions enabling further reduction of key bits in E_{out} . For any bit position $0 \leq i \leq n-1$, when the following conditions hold simultaneously:

- $\mathbf{dy}_{r,0}^f[i] \geq 0$ and $\mathbf{dz}_{r,0}^f[i] \geq 0$
- $\mathbf{ky}_{r,0}^f[i] = \mathbf{kz}_{r,0}^f[i] = 1$

The attacker must determine the value of the state variables $y_{r_{in}+r_m+r,0}[i]$ and $z_{r_{in}+r_m+r,0}[i]$. In the basic model, this requires guessing two equivalent subkey bits:

- $K_{r_{in}+r_m+r}^e[i+8 \pmod n]$ (in other notation $\tilde{y}_{r_{in}+r_m+r,0}[i]$)
- $K_{r_{in}+r_m+r}^e[i+1 \pmod n]$ (in other notation $\tilde{z}_{r_{in}+r_m+r,0}[i]$)

This requirement follows directly from the key generation relationships in Equation 3 and Equation 4. However, we analyze this scenario more closely in the following:

- When the attacker guesses the key bit $\tilde{y}_{r_{in}+r_m+r,0}[i]$ such that the value of $y_{r_{in}+r_m+r,0}[i]$ turns out as 0 (Equation 5), it follows that $p_{r_{in}+r_m+r,0}[i] = 0$. In this case, there is no need to determine the value of $\tilde{z}_{r_{in}+r_m+r,0}[i]$, since the value of $z_{r_{in}+r_m+r,0}[i]$ becomes irrelevant (as we already determine the value of $p_{r_{in}+r_m+r,0}[i]$).
- When the attacker guesses the key bit $\tilde{y}_{r_{in}+r_m+r,0}[i]$ such that the value of $y_{r_{in}+r_m+r,0}[i]$ turns out to be 1 (Equation 5), this directly sets $p_{r_{in}+r_m+r,0}[i] = z_{r_{in}+r_m+r,0}[i]$. This linear relationship absorbs the key bit $\tilde{z}_{r_{in}+r_m+r,0}[i]$ (controlling $z_{r_{in}+r_m+r,0}[i]$) into $p_{r_{in}+r_m+r,0}[i]$. Since this absorption occurs linearly, $\tilde{z}_{r_{in}+r_m+r,0}[i]$ can be integrated with the next key xoring operation affecting $p_{r_{in}+r_m+r,0}[i]$, avoiding the need for separate guessing.

The previous analysis reveals a significant optimization: only one key bit needs to be guessed per position - either $\tilde{y}_{r_{in}+r_m+r,0}[i]$ or $\tilde{z}_{r_{in}+r_m+r,0}[i]$, not both as required in the basic model. This reduces the number of key bits required for E_{out} . Now, to maximize efficiency, we implement a strategic choice:

- For each bit position $0 \leq i \leq n-1$, determine which key bit need to choose for guessing ($\tilde{y}_{r_{in}+r_m+r,0}[i]$ or $\tilde{z}_{r_{in}+r_m+r,0}[i]$)
- Select the optimal option for attack efficiency

We formalize this optimization using constraint programming variables:

- The ternary decision (CP) variables $\tilde{y}_{r,0}^f[i] \in \{0, 1, 2\}$ (for the key bit $\tilde{y}_{r_{in}+r_m+r,0}[i]$)
- The ternary decision (CP) variables $\tilde{z}_{r,0}^f[i] \in \{0, 1, 2\}$ (for the key bit $\tilde{z}_{r_{in}+r_m+r,0}[i]$)

The domain interpretation of these two ternary variables is as follows:

Table 3: Interpretation of ternary CP variables for key guessing

Value	Meaning	Action
0	Key bit <i>not guessed</i>	Excluded from guessing
1	Key bit <i>must be guessed</i>	Included in key recovery
2	Guessing status <i>undetermined</i>	Requires further resolution

Therefore, we first construct the following constraints to illustrate the above scenario and identify key bits that may or may not need to be guessed:

$$\text{SKG}_1(\text{dy}_{r,0}^f[i], \text{dz}_{r,0}^f[i], \text{ky}_{r,0}^f[i], \text{kz}_{r,0}^f[i], \tilde{y}_{r,0}^f[i], \tilde{z}_{r,0}^f[i]) :=$$

$$\left\{ \begin{array}{c|c} \text{Condition} & \text{Assignment} \\ \hline \text{dy}_{r,0}^f[i] \geq 0 \wedge \text{dz}_{r,0}^f[i] \geq 0 \wedge \text{ky}_{r,0}^f[i] = 1 \wedge \text{kz}_{r,0}^f[i] = 1 & \tilde{y}_{r,0}^f[i] = 2 \wedge \tilde{z}_{r,0}^f[i] = 2 \\ \text{otherwise} & \tilde{y}_{r,0}^f[i] = \text{ky}_{r,0}^f[i] \wedge \tilde{z}_{r,0}^f[i] = \text{kz}_{r,0}^f[i] \end{array} \right.$$

Now, due to the **Rotation-AND** operation of **SIMON** round function, the key bit $K_{r_{in}+r_m+r}^e[i+8 \pmod n]$ is actually shared in two positions $\tilde{y}_{r_{in}+r_m+r,0}[i]$, and $\tilde{z}_{r_{in}+r_m+r,0}[i+7 \pmod n]$, for any $0 \leq i \leq (n-1)$. Similarly, the key bit $K_{r_{in}+r_m+r}^e[i+1 \pmod n]$ is actually shared in two positions $\tilde{z}_{r_{in}+r_m+r,0}[i]$, and $\tilde{y}_{r_{in}+r_m+r,0}[i-7 \pmod n]$, for any $0 \leq i \leq (n-1)$. Therefore, the following constraints actually illustrate which key bits the attacker needs to guess and which key bits the attacker does not have to guess

from the aforementioned analysis.

$$\text{SKG}_2(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i]) :=$$

Condition	$\tilde{\mathbf{y}}_{r,0}^f[i]$	$\tilde{\mathbf{z}}_{r,0}^f[i]$
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 0, 0)$	1	0
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 0, 1)$	0	1
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 1, 0)$	1	0
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 1, 1)$	1	1
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 0, 2)$	0	1
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 2, 0)$	1	0
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 2, 2)$	1 (or 0)	0 (or 1)
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 1, 2)$	1	0
$(\tilde{\mathbf{y}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i], \tilde{\mathbf{z}}_{r,0}^f[i+7 \pmod n], \tilde{\mathbf{y}}_{r,0}^f[i-7 \pmod n]) = (2, 2, 2, 1)$	0	1
otherwise	$= \tilde{\mathbf{y}}_{r,0}^f[i]$	$= \tilde{\mathbf{z}}_{r,0}^f[i]$

For all $0 \leq i \leq (n-1)$, the CP variables $\tilde{\mathbf{y}}_{r,0}^f[i]$, and $\tilde{\mathbf{z}}_{r,0}^f[i]$ with domain $\{0, 1\}$, which indicate whether the key bits in the states $y_{r_{in}+r_m+r,0}[i]$, and $z_{r_{in}+r_m+r,0}[i]$ (Equation 3) need to be guessed or not, respectively, utilizing the SKG technique in equivalent subkey model. Finally, from the variables $\tilde{\mathbf{y}}_{r,0}^f$, and $\tilde{\mathbf{z}}_{r,0}^f$, we are able to find the involved subkey bits using the following constraints:

$$\mathbf{ikf}_r^{eq \cup skg} = (\tilde{\mathbf{y}}_{r,0}^f \ggg 8) \vee (\tilde{\mathbf{z}}_{r,0}^f \ggg 1)$$

where the CP variable $\mathbf{ikf}_r^{eq \cup skg}[i]$ indicates whether the i -th bit of the equivalent subkey in the r -th round of E_{out} is involved or not in differential MITM attack, for all $0 \leq i \leq (n-1)$. algorithm 4 outlines our $\text{CSP}_{out}^{eq \cup skg}$ model for identifying the accurate involved subkey bits k_{out} through E_{out} using selective key guessing technique in equivalent subkey model.

3.2.5 Complexity Modeling and Objective Function for SIMON

This section integrates all constraint satisfaction models described earlier and then constructs necessary constraints to compute the attack's complexity metrics. Following the framework established in Section Subsection 2.1, we calculate three critical parameters:

- $|k_{in}|$: Involved key bits in upper part
- $|k_{out}|$: Involved key bits in lower part
- $|k_{in} \cap k_{out}|$: Overlapping key bits

These parameters govern the attack's time, memory, and data complexities. Our model computes them through these constraints:

$$\begin{cases} \mathbf{k}_{in} &= \sum_{r=1}^{r_{in}-1} \sum_{i=0}^{n-1} \mathbf{ikb}_r^{eq}[i] \\ \mathbf{k}_{out} &= \sum_{r=0}^{r_{out}-2} \sum_{i=0}^{n-1} \mathbf{ikf}_r^{eq \cup skg}[i] \\ \mathbf{k}_{in \cup out} &= \mathbf{k}_{in} + \mathbf{k}_{out} \\ \mathbf{k}_{in \cap out} &= 0 \end{cases}$$

We implement the complexity formulas from Equation (1) and define the objective function as $\min \mathbf{T}$, where $\mathbf{T}$ is the time complexity variable. This enables our model to:

- Systematically compute all complexity parameters
- Optimize attack efficiency by minimizing $\mathbf{T}$
- Generate provably optimal attacks via the OR-Tools CP solver

Algorithm 4: $\text{CSP}_{out}^{eq\cup skg}$ model to find exact k_{out} through E_{out} for SIMON using selective key guessing in equivalent subkey model

Input: $\text{CSP}_{m.var}$, CSP_{out}^{dp} , CSP_{out}^{isk} and the integer number r_{out}
Output: $\text{CSP}_{out}^{eq\cup skg}$

```

1 Declare an empty CSP model  $\mathcal{M}$ ;
2 /* Declaration of CP variables */
3  $\mathcal{M}.var \leftarrow \{\text{ikf}_r^{eq}[i] \in \{0, 1\} : 0 \leq r \leq r_{out} - 2, 0 \leq i \leq (n - 1)\}$ ;
4  $\mathcal{M}.var \leftarrow \{\text{ikf}_r^{eq\cup skg}[i] \in \{0, 1\} : 1 \leq r \leq r_{out} - 2, 0 \leq i \leq (n - 1)\}$ ;
5  $\mathcal{M}.var \leftarrow \{\widetilde{\text{ky}}_{r,0}^f[i], \widetilde{\text{kz}}_{r,0}^f[i] \in \{0, 1, 2\} : 1 \leq r \leq r_{out} - 2, 0 \leq i \leq (n - 1)\}$ ;
6  $\mathcal{M}.var \leftarrow \{\widehat{\text{ky}}_{r,0}^f[i], \widehat{\text{kz}}_{r,0}^f[i] \in \{0, 1, 2\} : 1 \leq r \leq r_{out} - 2, 0 \leq i \leq (n - 1)\}$ ;
7  $\mathcal{M}.var \leftarrow \{\widetilde{\text{ky}}_{r,0}^f[i], \widehat{\text{kz}}_{r,0}^f[i] \in \{0, 1\} : 1 \leq r \leq r_{out} - 2, 0 \leq i \leq (n - 1)\}$ ;
8  $\mathcal{M}.var \leftarrow \{\widehat{\text{ky}}_{r,0}^f[i], \widetilde{\text{kz}}_{r,0}^f[i] \in \{0, 1\} : 1 \leq r \leq r_{out} - 2, 0 \leq i \leq (n - 1)\}$ ;
9 /* Involved key in equivalent subkey model */
10 for  $r = 0, \dots, r_{out} - 2, i = 0, \dots, (n - 1)$  do
11    $\mathcal{M}.con \leftarrow (\text{if } \widetilde{\text{ky}}_{r,0}^f[i] = \widetilde{\text{kz}}_{r,0}^f[i] = 0 \text{ then } \text{ikf}_r^{eq}[i] = 0 \text{ else } \text{ikf}_r^{eq}[i] = 1)$ ;
12 end
13 /* Updated involved key bits using selective key guessing in equivalent subkey model */
14 for  $r = 1, \dots, r_{out} - 2, i = 0, \dots, (n - 1)$  do
15    $\mathcal{M}.con \leftarrow \text{SKG}_1(\text{dy}_{r,0}^f[i], \text{dz}_{r,0}^f[i], \text{ky}_{r,0}^f[i], \text{kz}_{r,0}^f[i], \widetilde{\text{ky}}_{r,0}^f[i], \widetilde{\text{kz}}_{r,0}^f[i])$ ;
16 end
17 for  $r = 1, \dots, r_{out} - 2$  do
18    $\mathcal{M}.con \leftarrow (\widetilde{\text{ky}}_{r,0}^f = \widetilde{\text{ky}}_{r,0}^f \ggg 8) \cup (\widetilde{\text{kz}}_{r,0}^f = \widetilde{\text{kz}}_{r,0}^f \ggg 1)$ ;
19 end
20 for  $r = 1, \dots, r_{out} - 2, i = 0, \dots, (n - 1)$  do
21    $\mathcal{M}.con \leftarrow \text{SKG}_2(\widetilde{\text{ky}}_{r,0}^f[i], \widetilde{\text{kz}}_{r,0}^f[i], \widetilde{\text{kz}}_{r,0}^f[i + 8], \widetilde{\text{ky}}_{r,0}^f[i + 1], \widehat{\text{ky}}_{r,0}^f[i], \widehat{\text{kz}}_{r,0}^f[i])$ ;
22 end
23 for  $r = 1, \dots, r_{out} - 2$  do
24    $\mathcal{M}.con \leftarrow (\widehat{\text{ky}}_{r,0}^f = \widehat{\text{ky}}_{r,0}^f \ggg 8) \cup (\widehat{\text{kz}}_{r,0}^f = \widehat{\text{kz}}_{r,0}^f \ggg 1)$ ;
25 end
26 for  $r = 1, \dots, r_{out} - 2, i = 0, \dots, (n - 1)$  do
27    $\mathcal{M}.con \leftarrow (\text{if } \widehat{\text{ky}}_{r,0}^f[i] = \widehat{\text{kz}}_{r,0}^f[i] = 0 \text{ then } \text{ikf}_r^{eq\cup skg}[i] = 0 \text{ else } \text{ikf}_r^{eq\cup skg}[i] = 1)$ ;
28 end
29 for  $i = 0, \dots, (n - 1)$  do
30    $\mathcal{M}.con \leftarrow (\text{ikf}_0^{eq\cup skg}[i] = \text{ikf}_0^{eq}[i])$ ;
31 end
32 return  $\mathcal{M}$ ;

```

4 Applications to AndRX Ciphers

Our automated model is applied to develop differential MITM attacks targeting AndRX ciphers, which rely on three core operations: **AND** ($x \odot y$), **Rotation** ($x \lll n$ or $x \ggg n$), and **XOR** ($x \oplus y$). In particular, we focus on analyzing all variants of SIMON and Simeck within this framework. For detailed specifications of SIMON and Simeck, please refer to Subsection 2.3. The experiments are conducted using a standard laptop equipped with an Intel Core i5-8250U CPU (1.6GHz, 8 cores) and 8GB of RAM. To model and solve the constraint programming (CP) problems involved, we utilize MiniZinc [NSB⁺07]. To the best of our knowledge, this is the first work to present a bit-wise CP model for identifying differential MITM attacks on AndRX ciphers. Table 1 provides an overview of the differential MITM attacks we developed on Simon and Simeck. Our results indicate the differential MITM attack offers improved performance over classical differential, original MITM, and DS-MITM attacks for many, but not all, versions of SIMON and Simeck. Hence, we refrain from declaring it the strongest attack universally applicable. The key advancement presented here is the creation of an efficient, bit-wise automation tool

specifically for finding optimized differential MITM attacks on these cipher families. Our differential MITM attack methodology for AndRX ciphers follows a two-stage framework:

- *Core Attack Construction:* We first deploy our novel bit-wise automated model to generate differential MITM attacks against AndRX variants.
- *Complexity Optimization:* The core attacks discovered through automation are then extended via *parallel partitioning* and *data reduction with imposed conditions* techniques, culminating in comprehensive complexity calculations for the final attacks.

It is noteworthy that the integration of parallel partitioning and data reduction with the imposed conditions technique within our bitwise automated model remains an open problem.

Advantages of Our Automated Approach This component emphasizes the benefits of our automated approach for constructing differential MITM attacks on AndRX ciphers. The primary advantage is its ability to formulate a unified optimization problem that takes all attack parameters into account, allowing the tool to generate an optimized differential MITM attack for AndRX ciphers. In contrast, manual methods often involve exploring multiple differential trails within a constrained search space before selecting the best option. Another significant benefit is that, while our tool is applied to AndRX ciphers, it can easily be adapted to other bit-oriented ciphers to construct differential MITM attacks. Additionally, the tool offers the flexibility to integrate with various state-of-the-art solvers, including CP, SMT, and MILP solvers, further enhancing its efficiency and applicability.

4.1 Application to SIMON

We implement the differential MITM attack against all SIMON variants in single-key scenarios using our novel, bit-wise automated model. First, the differential distinguishers that serve as the foundation for our efficient and optimized differential MITM attacks on all variants of SIMON are detailed in Table 4 and Table 5. The parameters of our automated differential MITM attack in SIMON are detailed in Table 6, including input parameters (the length of E_{in} as r_{in} , the length of E_m as r_m , the length of E_{out} as r_{out}), and output parameters (the probability of differential distinguisher 2^{-p} , the number of involved subkeys in E_{in} as $|k_{in}|$, the number of involved subkeys in E_{out} as $|k_{out}|$, the total number of involved subkeys as $|k_{in} \cup k_{out}|$), crucial for generating the complexities of the differential MITM attack, as shown in Subsection 2.1. Furthermore, we incorporate two more optimization techniques (parallel partition and data reduction with imposed condition) into the core attack we discover using our automated tool for all versions of SIMON, and the attack summary can be found in Table 7.

All figures related to the core differential MITM attacks on SIMON variants are provided in Appendix B. Our automated tool takes three integer parameters (r_{in}, r_m, r_{out}) and generates an optimized differential MITM attack for $r_{tot} = r_{in} + r_m + r_{out}$ rounds of target SIMON variants. As detailed in Section 3, the tool incorporates two key optimizations: the *equivalent subkey technique* and the *selective key guessing technique*. However, two additional enhancements to the general differential MITM framework – the parallel partitioning technique and the data reduction with imposed conditions technique (discussed in Subsection 2.2) – lie beyond the tool’s current scope. Consequently, the differential MITM attack produced by the tool can be further enhanced for specific SIMON versions by manually applying these two techniques. Critically, the integration of parallel partitioning and data reduction with conditions into the automated framework remains an open research problem.

To provide a clear and detailed application example, we first outline a differential MITM

attack on 22-round SIMON-32-64, constructed using our automated tool. We present this attack and analyze its complexity. Building upon this 22-round foundation, we then develop a 23-round differential MITM attack on SIMON-32-64, achieving improved time, data, and memory complexities. This improvement is attained by applying the parallel partitioning technique and the data reduction with imposed conditions technique. A detailed description of both attacks follows.

4.1.1 A Detailed Description of differential MITM attack on 22-round SIMON-32-64

We utilize our bit-wise CP model to identify differential MITM attacks on all variants of SIMON and Simeck. Since our analytical approach is uniform across all versions of these ciphers, we provide a detailed example of a differential MITM attack on SIMON-32-64, discovered through our model. The attack parameters for the remaining versions will be discussed in the subsequent section.

The differential MITM attack on 22 rounds of SIMON-32-64 is illustrated in Figure 5. Specifically, Figure 5a and Figure 5b show the attacks using the original and equivalent subkey models, respectively. Now, our 22-round differential MITM attack on SIMON-32-64 based on the 14-round differential distinguisher with probability 2^{-p} , where $p = 30.81$. As previously discussed, the differences Δ_x and Δ_y propagate through E_{in}^{-1} to the plaintexts and through E_{out} to the ciphertexts, respectively, with probability 1. Here, E_{in} and E_{out} represent the additional $r_{in} = 4$ and $r_{out} = 4$ rounds surrounding E_m of $r_m = 14$ rounds. Active and unknown bit differences are denoted by  and , respectively. The input difference Δ_x has an active bit at $R_4[12]$, while the output difference Δ_y has an active bit at $L_{18}[4]$, with other bits showing inactive differences. Using the CP models CSP_{out}^{dp} and CSP_{in}^{dp} , we identify the state bits with active  and unknown  differences in both E_{in} and E_{out} .

Proceeding to the critical phase, we systematically identify specific state bits requiring either value or difference determination to construct the necessary plaintext pair $(P, \tilde{P})$ satisfying $E_{in}(P) \oplus E_{in}(\tilde{P}) = \Delta_x$ and ciphertext pair $(C, \tilde{C})$ satisfying $E_{out}^{-1}(C) \oplus E_{out}^{-1}(\tilde{C}) = \Delta_y$. These requirements are visually encoded in Figure 5, where red markers () denote bits needing value determination (modeled by $kx = 1$) and black markers () indicate bits requiring difference determination (modeled by $kdx = 1$). Furthermore, Figure 5a displays all involved E_{in} and E_{out} subkey bits under the original model, distinctly marked by  to illustrate their distribution. In contrast, Figure 5b displays the subkey bits after applying the selective key guessing technique in the equivalent subkey model, represented by , with unnecessary key bits indicated by . From Figure 5b, we get the attack parameters $|k_{in}| = 25$, $|k_{out}| = 21$, and $|k_{in} \cup k_{out}| = 46$. And from Figure 5a, we obtain $|k_{in}| = 27$, $|k_{out}| = 27$, and $|k_{in} \cup k_{out}| = 54$. The total number of involved subkey bits decreases from 54 in the original model to 46 in the equivalent model, showing that the equivalent subkey technique reduces the number of subkeys needed for key recovery attacks.

Finally, we can derive the complexities of the differential MITM attack by applying the complexity formula outlined in Subsection 2.1. This can be incorporated into our framework using the constraints presented in Equation 1. We discuss this in more detail. The data complexity follows the expression $\min(2^n, 2^{p+\min(|k_{in}|, |k_{out}|)})$. For this attack configuration, where the distinguisher probability equals 2^{-p} with $p = 30.81$, and $\min(|k_{in}|, |k_{out}|) = 21$. This evaluates to $\min(2^{32}, 2^{30.81+21}) = 2^{32}$. Time complexity is governed by: $2^p \times (2^{|k_{in}|} + 2^{|k_{out}|}) + 2^{|k_{in}|+|k_{out}|-|k_{in} \cap k_{out}|-n+p} + 2^{k-n+p}$. Substituting our parameters yields $2^{55.81} + 2^{51.81} + 2^{44.81} + 2^{62.81}$, dominated by the largest term ($\approx 2^{62.81}$). Memory complexity adheres to $2^{\min(|k_{in}|, |k_{out}|, |k_{in} \cap k_{out}|)}$. Given $|k_{in} \cap k_{out}| = 0$ in this instance, this simplifies to 2^{21} . Therefore, the total complexities of our attack are

as follows:

$$\begin{cases} \text{Time Complexity} : 2^{62.81} \\ \text{Data Complexity} : 2^{32} \\ \text{Memory Complexity} : 2^{21} \end{cases}$$

4.1.2 A Detailed Description of differential MITM attack on 23-round SIMON-32-64

We present a 22-round differential meet-in-the-middle (MITM) attack on SIMON-32-64, based on a 14-round optimal differential with probability $2^{-30.81}$. In this attack, the upper part involves 25 subkey bits ($|k_{in}| = 25$), while the lower part covers 21 subkey bits ($|k_{out}| = 21$). We extend our 22-round differential MITM attack on SIMON-32/64 to 23 rounds without increasing the overall complexity by applying the parallel partitioning technique outlined in Subsubsection 2.2.1. Two main factors enable this extension: the linearity of SIMON's key schedule and the fact that subkeys are XORed exclusively with the right half of the state. The detailed attack steps are as follows:

- **Step 1.** We request the encryption of the whole codebook.
- **Step 2.** We choose 2^{16} plaintext-ciphertext pairs $(L_0^\ell \parallel R_0^\ell, L_{23}^\ell \parallel R_{23}^\ell)$ (where $1 \leq \ell \leq 2^{16}$) such that $R_{23}^\ell = C$ (constant). In other words, we choose 2^{16} ciphertexts $L_{23}^\ell \parallel R_{23}^\ell = C$ and ask the decryption oracle to generate the corresponding 2^{16} plaintexts of the form $L_0^\ell \parallel R_0^\ell$. Here, we exploit the fact that the original round subkey is only applied to the right half of each internal state.
- **Step 3.** For each plaintext $L_0^\ell \parallel R_0^\ell$ from the structure, and each value i of k_{in} , we compute $\tilde{L}_0^{\ell,i} \parallel \tilde{R}_0^{\ell,i}$ which satisfy the following condition:

$$E_{in}(L_0^\ell \parallel R_0^\ell) \oplus E_{in}(\tilde{L}_0^{\ell,i} \parallel \tilde{R}_0^{\ell,i}) = \underbrace{00 \cdots 0}_{16 \text{ times}} \parallel \underbrace{00 \cdots 0}_{11 \text{ times}} 1 \underbrace{00 \cdots 0}_{3 \text{ times}}$$

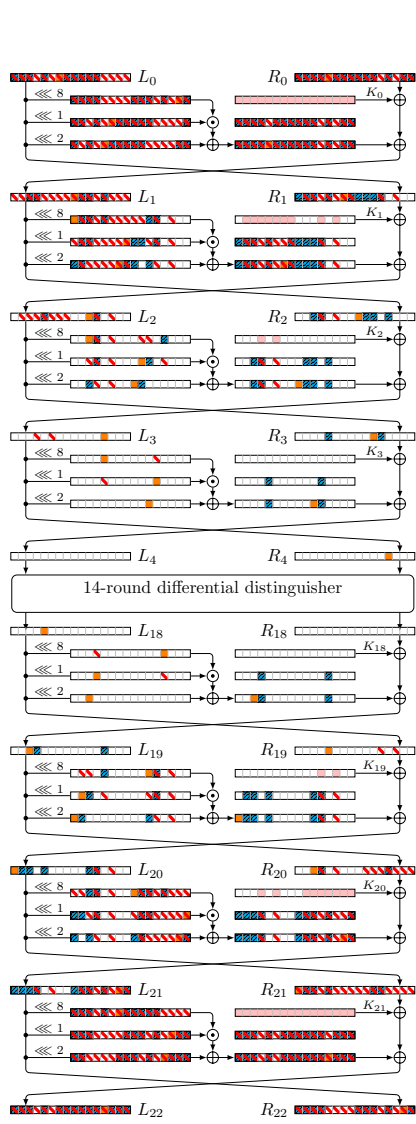
In this context, E_{in} denotes the upper part of the 22-round differential MITM attack, comprising $r_{in} = 4$ rounds. Now, for each plaintext of the form $\tilde{L}_0^{\ell,i} \parallel \tilde{R}_0^{\ell,i}$, we compute ciphertext $\tilde{L}_{23}^{\ell,i} \parallel \tilde{R}_{23}^{\ell,i}$, and subsequently construct the tuples of the form $(L_{23}^\ell \parallel R_{23}^\ell, \tilde{L}_{23}^{\ell,i} \parallel \tilde{R}_{23}^{\ell,i}, i)$. It is noteworthy that the tuples are constructed for all the 2^{16} ciphertexts $L_{23}^\ell \parallel R_{23}^\ell$. Hence, if you store them in a hash table, the memory complexity in this step is $2^{25+16} = 2^{41}$ 89 bit words, as each tuple consists of $32 + 32 + 25 = 89$ bits.

- **Step 4.** Similarly, from 2^{16} many ciphertexts of the form $(L_{23}^\ell \parallel R_{23}^\ell = C)$, we compute 2^{16} such states $(C = L_{22}^\ell \parallel R_{22}^\ell)$ (one for each possible value of K_{22}). Therefore, for each value j of k_{out} , and each state $L_{22}^\ell \parallel R_{22}^\ell$, we compute all possible tuples of the form $(L_{22}^\ell \parallel R_{22}^\ell, \tilde{L}_{22}^{\ell,j} \parallel \tilde{R}_{22}^{\ell,j}, j)$, each of which satisfy the following condition:

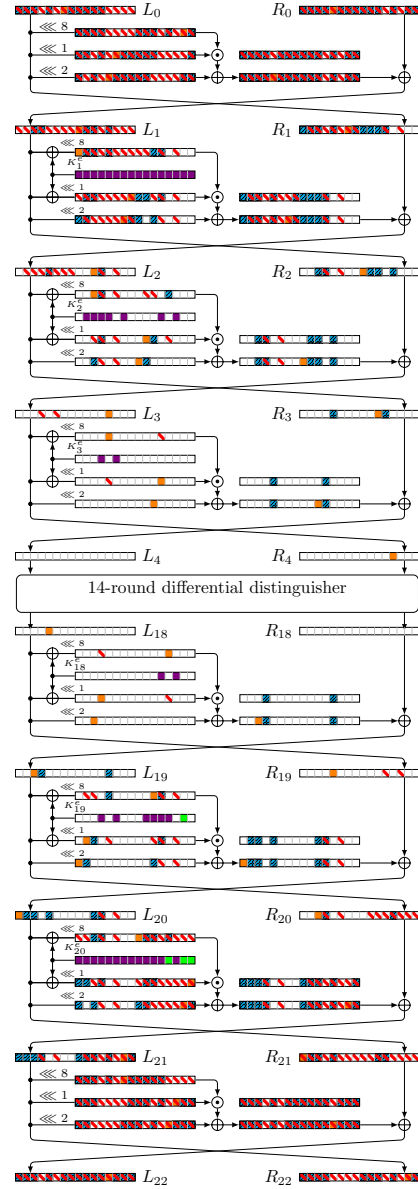
$$E_{out}^{-1}(L_{22}^\ell \parallel R_{22}^\ell) \oplus E_{out}^{-1}(\tilde{L}_{22}^{\ell,j} \parallel \tilde{R}_{22}^{\ell,j}) = \underbrace{00 \cdots 0}_{4 \text{ times}} 1 \underbrace{00 \cdots 0}_{11 \text{ times}} \parallel \underbrace{00 \cdots 0}_{16 \text{ times}}$$

Here E_{out} denotes the lower part of the 22-round differential MITM attack. Hence, if you store these tuples in a hash table, the memory complexity will be $2^{21+16} = 2^{37}$ 85 bit words, as each tuple consists of $32 + 32 + 21 = 85$ bits.

- **Step 5.** Our goal is to match the two lists of tuples $(L_{23}^\ell \parallel R_{23}^\ell, \tilde{L}_{23}^{\ell,i} \parallel \tilde{R}_{23}^{\ell,i}, i)$ and $(L_{22}^\ell \parallel R_{22}^\ell, \tilde{L}_{22}^{\ell,j} \parallel \tilde{R}_{22}^{\ell,j}, j)$ for each ℓ, i, j . Typically, this is done by storing one list in a hash table and then checking for matches against elements from the other list. To optimize memory usage, we choose to store the tuples $(L_{22}^\ell \parallel R_{22}^\ell, \tilde{L}_{22}^{\ell,j} \parallel \tilde{R}_{22}^{\ell,j}, j)$ in the



(a) Original Subkey.



(b) Equivalent Subkey.

 Difference required
  Value required
 1
  any
  Key bits skipped by SKG technique
 Involved subkey bits (original)
  Involved subkey bits (equivalent)

Figure 5: Differential MITM attack on 22-round SIMON32-64 (Original subkey vs Equivalent subkey).

hash table, and then check for matches with the tuples $(L_{23}^\ell \parallel R_{23}^\ell, \tilde{L}_{23}^{\ell,i} \parallel \tilde{R}_{23}^{\ell,i}, i)$. As a result, the memory complexity of this procedure is 2^{37} words of 85 bits each, or more precisely $2^{43.409}$ bits. Finally, the match is now performed on three possibilities:

- From $L_{22}^\ell \parallel R_{22}^\ell$, and $L_{23}^\ell \parallel R_{23}^\ell$, exploiting the round function structure of SIMON, we have the following condition:

$$L_{23}^\ell = R_{22}^\ell \oplus F(L_{22}^\ell) \oplus K_{22} = R_{22}^\ell \oplus F(C) \oplus K_{22}$$

It is important to note that, it is a 16-bit filter. Exploiting the linear nature of SIMON's key schedule, subkey K_{22} expresses linearly as:

$$K_{22} = f(k_{in}) \oplus g(k_{out}) \oplus h(k_{re})$$

where f, g, h denote public linear mappings. Here, k_{re} constitutes additional key materials ($|k_{re}| = 18$), essential for complete master key derivation, computed as $64 - |k_{in} \cup k_{out}| = 64 - 46 = 18$. Therefore, the condition is as follows:

$$L_{23}^\ell = R_{22}^\ell \oplus F(C) \oplus K_{22} = R_{22}^\ell \oplus F(C) \oplus f(k_{in}) \oplus g(k_{out}) \oplus h(k_{re})$$

- Additionally, for the associated ciphertexts of the form $\tilde{L}_{23}^{\ell,i} \parallel \tilde{R}_{23}^{\ell,i}$ and $\tilde{L}_{22}^{\ell,j} \parallel \tilde{R}_{22}^{\ell,j}$, we have the following condition:

$$\begin{cases} \tilde{L}_{22}^{\ell,j} = \tilde{R}_{23}^{\ell,i} \\ \tilde{L}_{23}^{\ell,i} = \tilde{R}_{22}^{\ell,j} \oplus \tilde{L}_{22}^{\ell,j} \oplus K_{22} = \tilde{R}_{22}^{\ell,j} \oplus \tilde{L}_{22}^{\ell,j} \oplus f(k_{in}) \oplus g(k_{out}) \oplus h(k_{re}) \end{cases}$$

This represents a 32-bit filter.

- **Step 6.** We repeat from **Step 2** until the right key is recovered.

To guarantee inclusion of a valid differential pair, the attack must be repeated until the cumulative pairs cover the differential's probability. With 2^{16} pairs generated per iteration (**Step 2**) and a differential probability of $2^{-30.81}$, the required number of repetitions is $2^{14.81}$. In this attack strategy, the data complexity amounts to 2^{32} , as access to the complete codebook is required. Furthermore, from **Step 5**, it is evident that the memory requirement is approximately $2^{43.409}$ bits. As outlined in **Step 3**, we can generate around 2^{41} tuples, and from **Step 4**, we obtain about 2^{37} tuples. Moving to **Step 5**, the remaining key bits in k_{re} are guessed, and the filtering process involves a total of 48 bits (32 from one filter and 16 from another). Considering 2^{16} chosen plaintext-ciphertext pairs (P^ℓ, C^ℓ) , the expected number of matching tuples is $2^{41+37+18-48} = 2^{48}$. Since this process must be repeated $2^{14.81}$ times, the resulting overall time complexity is $2^{48+14.81} = 2^{62.81}$.

Now, an intriguing question is whether the data and memory complexities for our 23-round SIMON-32-64 attack can be further reduced using the data reduction with imposed conditions technique (Section **Subsubsection 2.2.2**). This approach requires fixing x bits in both plaintexts P and $\tilde{P}$, where x must simultaneously satisfy:

$$\begin{cases} p + x \leq n - x \\ 2^{p+x}(2^{|k_{in}|} + 2^{|k_{out}|}) < 2^k \end{cases}$$

Here k represents the total key size, and n is the block size. Applying our attack parameters ($p = 30.81$, $n = 32$, $k = 64$, $|k_{in}| = 25$, $|k_{out}| = 21$) yields a theoretical maximum $x \leq 0.595$. However, fractional bit fixation is cryptographically meaningless in practice. We therefore conclude that data-time tradeoff improvements via this technique are unattainable for this

Table 4: The single differential characteristics to construct differential MITM attack in SIMON.

Cipher	Round	Input active bits	Output active bits	Probability
SIMON-32-64	11	$x_{0,0}[0], x_{0,1}[0], x_{0,1}[14]$	$y_{11,0}[0], y_{11,0}[12], y_{11,1}[14]$	2^{-31}
SIMON-48-72	15	$x_{0,0}[22],$ $x_{0,1}[0], x_{0,1}[14],$ $x_{0,1}[20]$	$y_{15,0}[0], y_{15,0}[10], y_{15,0}[16],$ $y_{15,0}[20], y_{15,1}[18]$	2^{-46}
SIMON-48-96	15	$x_{0,0}[11], x_{0,1}[9], x_{0,1}[13]$	$y_{15,0}[5], y_{15,0}[6], y_{15,0}[9],$ $y_{15,0}[13], y_{15,1}[7]$	2^{-46}
SIMON-64-96	18	$x_{0,0}[7], x_{0,1}[5], x_{0,1}[31]$	$y_{18,0}[7]$	2^{-62}
SIMON-64-128	18	$x_{0,0}[9], x_{0,1}[7]$	$y_{18,0}[9]$	2^{-62}
SIMON-96-96	26	$x_{0,0}[47], x_{0,1}[1], x_{0,1}[5],$ $x_{0,1}[45]$	$y_{26,0}[3], y_{26,0}[43], y_{26,1}[1],$ $y_{26,1}[5], y_{26,1}[45]$	2^{-86}
SIMON-96-144	27	$x_{0,0}[47], x_{0,1}[1], x_{0,1}[5],$ $x_{0,1}[45]$	$y_{27,0}[5], y_{27,0}[41], y_{27,0}[45],$ $y_{27,1}[3], y_{27,1}[43]$	2^{-90}
SIMON-128-128	33	$x_{0,0}[63], x_{0,1}[1], x_{0,1}[61]$	$y_{33,0}[1], y_{33,1}[63]$	2^{-122}
SIMON-128-192	36	$x_{0,0}[63], x_{0,1}[1], x_{0,1}[61]$	$y_{36,0}[63], y_{36,1}[1]$	2^{-126}
SIMON-128-256	36	$x_{0,0}[63], x_{0,1}[1], x_{0,1}[61]$	$y_{36,0}[63], y_{36,1}[1]$	2^{-126}

Table 5: The optimal differentials for SIMON [LLW17].

Block Size	Round	Input active bits	Output active bits	Probability
32 [†]	14	$x_{0,1}[12]$	$y_{14,0}[4]$	$2^{-30.81}$
48 [†]	17	$x_{0,0}[16],$ $x_{0,1}[22], x_{0,1}[18],$ $x_{0,1}[14]$	$y_{17,0}[22], y_{17,0}[18],$ $y_{17,0}[14],$ $y_{17,1}[16]$	$2^{-46.38}$
64	23	$x_{0,0}[31], x_{0,1}[29], x_{0,1}[1]$	$y_{23,0}[29], y_{23,0}[25], y_{23,0}[1],$ $y_{23,1}[27]$	$2^{-61.93}$
96	31	$x_{0,0}[33],$ $x_{0,1}[47], x_{0,1}[39],$ $x_{0,1}[35], x_{0,1}[31]$	$y_{31,0}[47], y_{31,0}[39],$ $y_{31,0}[35],$ $y_{31,1}[45], y_{31,1}[37]$	$2^{-95.34}$
128	41	$x_{0,0}[57],$ $x_{0,1}[63], x_{0,1}[59],$ $x_{0,1}[55]$	$y_{41,0}[63], y_{41,0}[59],$ $y_{41,0}[55],$ $y_{41,1}[57]$	$2^{-123.74}$

[†]: The differentials are taken from [KLT15]. Except for these, all other differentials are the same as in [LLW17].

specific 23-round attack. The final complexity figures for our differential MITM attack on SIMON-32-64 consequently remain unchanged:

$$\begin{cases} \text{Time Complexity : } 2^{62.81} \\ \text{Data Complexity : } 2^{32} \\ \text{Memory Complexity : } 2^{43.409} \end{cases}$$

4.1.3 Comprehensive Summary: Differential MITM Attacks on All SIMON Variants

We deploy our automated tool to discover differential MITM attacks across all SIMON variants. For each cipher configuration, we develop two distinct attack variants:

- An attack leveraging a single optimal differential characteristic

Table 6: Summary of our differential MITM attack (without parallel partitioning technique) parameters for **SIMON**.

Cipher	$(r_{in}, r_m, r_{out}, r_{tot})$	p	$ k_{in} $	$ k_{out} $	$ k_{in} \cup k_{out} $	Data	Time	Memory	Figure
SIMON-32-64*	(4, 11, 4, 19)	31	30	30	60	2^{32}	2^{63}	2^{30}	Figure 7
SIMON-32-64†	(4, 14, 4, 22)	30.81	25	21	46	2^{32}	$2^{62.81}$	2^{21}	Figure 8
SIMON-48-72*	(3, 15, 3, 21)	46	24	23	47	2^{48}	2^{70}	2^{23}	Figure 9
SIMON-48-72†	(2, 17, 3, 22)	46.32	6	23	29	2^{48}	$2^{70.32}$	2^6	Figure 10
SIMON-48-96*	(4, 15, 4, 23)	46	44	48	92	2^{48}	2^{94}	2^{44}	Figure 11
SIMON-48-96†	(3, 17, 4, 24)	46.32	27	46	73	2^{48}	$2^{94.32}$	2^{27}	Figure 12
SIMON-64-96*	(3, 18, 3, 24)	62	22	10	32	2^{64}	2^{94}	2^{10}	Figure 13
SIMON-64-96†	(3, 23, 3, 29)	61.93	21	26	47	2^{64}	$2^{93.93}$	2^{21}	Figure 14
SIMON-64-128*	(4, 18, 5, 27)	62	43	58	101	2^{64}	2^{126}	2^{43}	Figure 15
SIMON-64-128†	(4, 23, 4, 31)	61.93	52	55	107	2^{64}	$2^{125.93}$	2^{52}	Figure 16
SIMON-96-96*	(2, 26, 2, 30)	86	6	4	10	2^{90}	2^{92}	2^4	Figure 18
SIMON-96-144*	(3, 27, 3, 33)	90	32	25	57	2^{96}	2^{138}	2^{25}	Figure 17
SIMON-96-144†	(3, 31, 3, 37)	95.34	39	25	64	2^{96}	$2^{143.34}$	2^{25}	Figure 22
SIMON-128-128*	(2, 33, 2, 37)	122	4	2	6	2^{124}	2^{126}	2^2	Figure 19
SIMON-128-192*	(4, 36, 4, 44)	126	60	37	97	2^{128}	2^{190}	2^{37}	Figure 20
SIMON-128-192†	(3, 41, 4, 48)	123.74	32	64	96	2^{128}	$2^{187.74}$	2^{32}	Figure 23
SIMON-128-256*	(5, 36, 6, 47)	126	117	120	237	2^{128}	2^{254}	2^{117}	Figure 21
SIMON-128-256†	(4, 41, 5, 50)	123.74	78	119	197	2^{128}	$2^{251.74}$	2^{78}	Figure 24

r_m : The length of the distinguisher. r_{in} : The length of the extended backward direction. r_{out} : The length of the extended forward direction. r_{tot} : Total number of attacked rounds. p : The probability of the distinguisher is 2^{-p} . $|k_{in}|$: The number of involved subkey bits in E_{in} . $|k_{out}|$: The number of involved subkey bits in E_{out} .

*: Based on a single differential trail illustrated in Table 4.

†: Based on optimal differentials in **SIMON** described in Table 5.

- An attack utilizing optimal differentials (composite trails)

Crucially, the differential MITM attacks employing optimal differentials achieve the maximum round count for every **SIMON** version, demonstrating the framework’s extended reach. We document the single optimal differential characteristics in Table 4 and the optimal differential trails in Table 5, both utilized in our differential MITM attacks across **SIMON** variants. Finally, the summary of our differential attack parameters, which we get from our automated tool, is demonstrated in Table 6 for each version of **SIMON**. Building on our differential MITM framework, we now integrate two enhancements—the parallel partitioning technique and data reduction with imposed conditions technique, to develop improved attacks across all **SIMON** variants. The complete specifications of these advanced attacks appear in Table 7. Critical improvements are evident when comparing this new table with Table 6:

- *One-round extension*: Parallel partitioning universally extends our prior attacks (from Table 6) by one additional round (which is highlighted using **red** color in Table 7)
- *Sub-codebook efficiency*: Data reduction with imposed conditions enables non-full-codebook attacks for most **SIMON** variants—a significant practical advancement (which is depicted using **yellow** colour in Table 7)

These techniques collectively establish new state-of-the-art results for some of the variants of **SIMON**, demonstrating both increased cryptanalytic reach and enhanced efficiency over the previous approaches (classical differential attack, original MITM attack, and DS-MITM attack). Most importantly, we present:

Table 7: Summary of our differential MITM attack (incorporating parallel partitioning technique and reducing data with imposed conditions technique) parameters for SIMON.

Cipher	# R	p	x	Data	Time	Memory
SIMON-32-64*	20	31	-	2^{32}	2^{63}	$2^{52.55}$
SIMON-32-64†	23	30.81	-	2^{32}	$2^{62.81}$	$2^{43.409}$
SIMON-48-72*	22	46	1	2^{47}	2^{70}	$2^{52.571}$
SIMON-48-72†	23	46.32	-	2^{48}	$2^{70.32}$	$2^{36.286}$
SIMON-48-96*	24	46	1	2^{47}	2^{94}	$2^{74.128}$
SIMON-48-96†	25	46.32	-	2^{48}	$2^{94.32}$	$2^{57.946}$
SIMON-64-96*	25	62	1	2^{63}	2^{94}	$2^{48.107}$
SIMON-64-96†	30	61.93	1	2^{63}	$2^{93.93}$	$2^{59.22}$
SIMON-64-128*	28	62	1	2^{63}	2^{126}	$2^{81.419}$
SIMON-64-128†	32	61.93	1	2^{63}	$2^{125.93}$	$2^{90.493}$
SIMON-96-96*	31	86	5	2^{85}	2^{92}	$2^{54.615}$
SIMON-96-144*	34	90	3	2^{93}	2^{138}	$2^{77.763}$
SIMON-96-144†	38	95.34	-	2^{96}	$2^{143.34}$	$2^{80.763}$
SIMON-128-128*	38	122	3	2^{121}	2^{126}	$2^{71.012}$
SIMON-128-192*	45	126	1	2^{127}	2^{190}	$2^{108.197}$
SIMON-128-192†	49	123.74	2	2^{126}	$2^{187.74}$	$2^{102.170}$
SIMON-128-256*	48	126	1	2^{127}	2^{254}	$2^{188.542}$
SIMON-128-256†	51	123.74	2	2^{126}	$2^{251.74}$	$2^{148.542}$

R : The total number of attack rounds. x : The number of bits we can fix in both plaintext P and $\tilde{P}$ in reducing data using imposed conditions technique. p : The probability of the distinguisher is 2^{-p} .

*: Based on a single differential trail illustrated in Table 4.

†: Based on optimal differentials in SIMON described in Table 5.

- *23-round differential MITM attack on SIMON-32-64*: Extends a 22-round core attack (automatically generated via our tool; see Figure 8 in Appendix B)
- *25-round differential MITM attack on SIMON-48-96*: Extends a 24-round core attack (automatically generated via our tool; see Figure 12 in Appendix B)
- *32-round differential MITM attack on SIMON-64-128*: Extends a 31-round core attack (automatically generated via our tool; see Figure 16 in Appendix B)
- *38-round differential MITM attack on SIMON-96-144*: Extends a 37-round core attack (automatically generated via our tool; see Figure 22 in Appendix B)

Each of these attacks advances beyond the best existing results in three categories: classical differential, classical MITM, and $\mathcal{DS}$ -MITM attacks. Moreover, we achieve attacks on 23 rounds of SIMON-48-72, 30 rounds of SIMON-64-96, and 51 rounds of SIMON-128-256. While these results surpass all previous MITM-based approaches, they match the maximum round counts attained by prior differential-only attacks for these variants. Table 1 provides a detailed comparison with state-of-the-art results across all three attack categories, demonstrating the efficiency gains of our approach.

Remark 1. This analysis demonstrates the critical importance of integrating equivalent subkeys with selective key guessing in differential MITM attacks. When applying our auto-

mated framework to SIMON-128-256, initial modeling without equivalent subkeys revealed no feasible 50-round attack. Specifically, extending a 41-round differential distinguisher (probability $2^{-123.74}$) with 4 forward and 5 backward rounds produced attack parameters as follows:

$$|k_{in}| = 81, |k_{out}| = 142, |k_{in} \cup k_{out}| = 223$$

Now, utilizing the complexity framework established in [Subsection 2.1](#), the computed time complexity of strictly greater than 2^{256} exceeds practical bounds, confirming the infeasibility of a 50-round differential MITM attack on SIMON-128-256 under the original subkey model. To overcome this limitation, we implement our novel integration of equivalent subkey technique with selective key guessing, which successfully enables the feasible differential MITM attack on 50-round SIMON-128-256. The complete attack structure is presented in [Figure 24](#) ([Appendix B](#)). More precisely, we find out $|k_{in}| = 78$, $|k_{out}| = 119$, and $|k_{in} \cup k_{out}| = 197$, leading to the following attack complexities:

$$\begin{cases} \text{Time Complexity : } 2^{251.74} \\ \text{Data Complexity : } 2^{128} \\ \text{Memory Complexity : } 2^{78} \end{cases}$$

Further enhancement through parallel partitioning extended this attack to 51 rounds while maintaining identical time ($2^{251.74}$) and data (2^{128}) complexities, though with increased memory requirement ($2^{150.381}$). Subsequent application of data reduction with imposed conditions yielded the non-full-codebook attack on 51 rounds, reducing data complexity to 2^{126} while adjusting memory to $2^{148.542}$, all without compromising time complexity. Hence, we present a 51-round differential MITM attack on SIMON-128-256 with the complexities as below:

$$\begin{cases} \text{Time Complexity : } 2^{251.74} \\ \text{Data Complexity : } 2^{126} \\ \text{Memory Complexity : } 2^{148.542} \end{cases}$$

Table 8: The single differential characteristics to construct differential MITM attack in Simeck.

Cipher	Round	Input active bits	Output active bits	Probability
Simeck-32-64	12	$x_{0,0}[0], x_{0,1}[15]$	$y_{12,0}[0], y_{12,0}[14], y_{12,1}[15]$	2^{-27}
Simeck-48-96	18	$x_{0,0}[8], x_{0,1}[3], x_{0,1}[7]$	$y_{18,0}[8]$	2^{-46}
Simeck-64-128	24	$x_{0,0}[13],$ $x_{0,1}[3],$ $x_{0,1}[12],$ $x_{0,1}[14], x_{0,1}[16]$	$y_{24,0}[13],$ $y_{24,1}[16]$ $y_{24,1}[14],$	2^{-62}

Table 9: The optimal differentials for Simeck [\[LLW17\]](#)

Block Size	Round	Input active bits	Output active bits	Probability
32	14	$x_{0,0}[15],$ $x_{0,1}[14],$ $x_{0,1}[0]$	$y_{14,0}[13], y_{14,1}[14], y_{14,1}[0]$	$2^{-31.64}$
48	21	$x_{0,0}[23], x_{0,1}[22], x_{0,1}[0]$	$y_{21,0}[22], y_{21,0}[0], y_{21,1}[23]$	$2^{-45.28}$
64	27	$x_{0,1}[31], x_{0,1}[27]$	$y_{27,0}[31],$ $y_{27,1}[30]$ $y_{27,0}[29],$	$2^{-61.49}$

4.2 Application to Simeck

Employing our constraint programming (CP) model, we develop differential meet-in-the-middle attacks for all Simeck variants under single-key settings. Our methodology follows a two-phase approach:

1. *Core Attack Generation*: Using our novel bit-wise automated framework (Section 3), we construct optimized differential MITM attacks. Key parameters for these foundational attacks—including rounds, time complexity, and memory requirements—are cataloged in Table 10. The underlying differential distinguishers enabling these efficient attacks appear in Table 8 and Table 9. All figures related to the core differential MITM attacks on Simeck variants are provided in Appendix B.
2. *Complexity Refinement*: We enhance these core attacks through *data reduction with imposed conditions*, substantially improving complexity metrics. The optimized parameters post-application are comprehensively detailed in Table 11.

A critical constraint arises from Simeck’s non-linear key schedule: this structure prevents effective application of parallel partitioning within our differential MITM framework. Consequently, our optimizations focus exclusively on data reduction with the imposed conditions technique for this cipher family.

Additionally, we propose two distinct core attack constructions for all Simeck variants. First, we develop differential MITM attacks employing single optimal differential characteristics. More significantly, we present differential MITM attacks utilizing optimal differentials – strategic combinations of multiple differential characteristics. Crucially, this latter approach consistently achieves the maximum attackable rounds for every Simeck variant. Our most impactful results emerge through the application of data reduction techniques to these core attack constructions. For Simeck-48-96, we establish a 29-round non-full-codebook differential MITM attack, surpassing the previous best differential attack by one round – a cryptanalytic advancement achieved while simultaneously reducing data requirements. For Simeck-64-128, we develop a 35-round non-full-codebook attack matching the maximum rounds of prior differential attacks. Table 1 illustrates the summary of our best differential MITM attacks on Simeck, along with the comparison with previous best known differential attacks on Simeck.

Table 10: Summary of our differential MITM attack (without reducing data with imposed conditions technique) parameters for Simeck.

Cipher	$(r_{in}, r_m, r_{out}, r_{tot})$	p	$ k_{in} $	$ k_{out} $	$ k_{in} \cup k_{out} $	Data	Time	Memory	Figure
Simeck-32-64*	(4, 12, 4, 20)	27	21	31	52	2^{32}	2^{59}	2^{21}	Figure 27
Simeck-32-64†	(3, 14, 4, 21)	31.64	20	26	46	2^{32}	$2^{63.64}$	2^{20}	Figure 26
Simeck-48-96*	(4, 18, 5, 27)	46	36	45	81	2^{48}	2^{94}	2^{36}	Figure 29
Simeck-48-96†	(4, 21, 4, 29)	45.28	40	37	77	2^{48}	93.28	2^{37}	Figure 28
Simeck-64-128*	(4, 24, 5, 33)	62	62	64	126	2^{64}	2^{126}	2^{62}	Figure 31
Simeck-64-128†	(4, 27, 4, 35)	61.49	41	40	81	2^{64}	$2^{125.49}$	2^{40}	Figure 30

r_m : The length of the distinguisher. r_{in} : The length of the extended backward direction. r_{out} : The length of the extended forward direction. r_{tot} : Total number of attacked rounds. p : The probability of the distinguisher is 2^{-p} . $|k_{in}|$: The number of involved subkey bits in E_{in} . $|k_{out}|$: The number of involved subkey bits in E_{out} .

*: Based on single differential trail illustrated in Table 8.

†: Based on optimal differentials in Simeck described in Table 9.

Table 11: Summary of our differential MITM attack (incorporating reducing data with imposed conditions technique) parameters for Simeck.

Cipher	# R	p	x	Data	Time	Memory
Simeck-32-64 [*]	20	27	2	2^{30}	2^{59}	2^{19}
Simeck-32-64 [†]	21	31.64	-	2^{32}	$2^{63.64}$	2^{20}
Simeck-48-96 [*]	27	46	1	2^{47}	2^{94}	2^{35}
Simeck-48-96 [†]	29	45.28	1	2^{47}	$2^{93.28}$	2^{36}
Simeck-64-128 [*]	33	62	1	2^{63}	2^{126}	2^{61}
Simeck-64-128 [†]	35	61.49	1	2^{63}	$2^{125.49}$	2^{39}

R : The total number of attack rounds. x : The number of bits we can fix in both plaintext P and $\tilde{P}$ in reducing data using imposed conditions technique. p : The probability of the distinguisher is 2^{-p} .

^{*}: Based on a single differential trail illustrated in Table 8.

[†]: Based on optimal differentials in SIMON described in Table 9.

5 Conclusion and Future Work

This paper introduces, for the first time, an efficient automated tool to find an optimized differential MITM attack framework for bit-oriented ciphers, with a focus on AndRX ciphers. First, we develop a bit-wise CP model to identify optimal differential trails in AndRX ciphers. Then, we integrate this CP model to detect differential distinguishers and combine it with a CP model for key recovery, resulting in a unified approach for identifying optimized differential MITM attacks. Furthermore, based on the structural properties in AndRX ciphers, we incorporate two optimizing techniques, such as *equivalent subkey* and *selective key guessing*, into the differential MITM framework for AndRX ciphers, enhancing the key recovery process.

To demonstrate the effectiveness of our methods, we apply them to all versions of SIMON and Simeck. Our results show that, across all SIMON variants, the differential MITM attack provides significant improvements over existing approaches such as standard MITM and $\mathcal{DS}$ -MITM. Moreover, for certain variants of both SIMON and Simeck, the differential MITM attack even outperforms the best-known differential attacks. Our work also opens new research directions. While we provide a method to apply bit-wise CP models for differential MITM attacks on AndRX designs, future research could explore developing a similar CP model for ARX designs. Finally, the key recovery models we propose may serve as a foundation for further optimization techniques in the differential MITM attack framework. It is important to note that our automated tool does not currently incorporate the parallel partitioning technique, which we leave as a direction for future work.

References

- [AFK⁺08] Jean-Philippe Aumasson, Simon Fischer, Shahram Khazaei, Willi Meier, and Christian Rechberger. New features of latin dances: Analysis of salsa, chacha, and rumba. In Kaisa Nyberg, editor, *Fast Software Encryption*, pages 470–488, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [AKM⁺24] Zahra Ahmadian, Akram Khalesi, Dounia M’Foukh, Hossein Moghimi, and María Naya-Plasencia. Improved differential meet-in-the-middle cryptanalysis. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EURO-CRYPT 2024*, pages 280–309, Cham, 2024. Springer Nature Switzerland.

- [ALLW15] Farzaneh Abed, Eik List, Stefan Lucks, and Jakob Wenzel. Differential cryptanalysis of round-reduced simon and speck. In Carlos Cid and Christian Rechberger, editors, *Fast Software Encryption*, pages 525–545, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg. doi:10.1007/978-3-662-46706-0_27.
- [AS09] Kazumaro Aoki and Yu Sasaki. Meet-in-the-middle preimage attacks against reduced sha-0 and sha-1. In Shai Halevi, editor, *Advances in Cryptology - CRYPTO 2009*, pages 70–89, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.
- [BBS99] Eli Biham, Alex Biryukov, and Adi Shamir. Miss in the middle attacks on idea and khufu. In Lars Knudsen, editor, *Fast Software Encryption*, pages 124–138, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [BDD⁺23] Christina Boura, Nicolas David, Patrick Derbez, Gregor Leander, and María Naya-Plasencia. Differential Meet-In-The-Middle Cryptanalysis. In *43rd Annual International Cryptology Conference, CRYPTO 2023*, volume 14083 of *Lecture Notes in Computer Science*, pages 240–272, Santa Barbara, United States, August 2023. Springer Nature Switzerland. URL: <https://hal.science/hal-04276899>, doi:10.1007/978-3-031-38548-3_9.
- [BDK01] Eli Biham, Orr Dunkelman, and Nathan Keller. The rectangle attack — rectangling the serpent. In Birgit Pfitzmann, editor, *Advances in Cryptology — EUROCRYPT 2001*, pages 340–357, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [Bei16] Christof Beierle. Pen and paper arguments for SIMON and simon-like designs. In Vassilis Zikas and Roberto De Prisco, editors, *Security and Cryptography for Networks - 10th International Conference, SCN 2016, Amalfi, Italy, August 31 - September 2, 2016, Proceedings*, volume 9841 of *Lecture Notes in Computer Science*, pages 431–446. Springer, 2016. doi:10.1007/978-3-319-44618-9_23.
- [BKR11] Andrey Bogdanov, Dmitry Khovratovich, and Christian Rechberger. Biclique cryptanalysis of the full aes. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, pages 344–371, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [BPP⁺17] Subhadeep Banik, Sumit Kumar Pandey, Thomas Peyrin, Yu Sasaki, Siang Meng Sim, and Yosuke Todo. GIFT: A small present - towards reaching the limit of lightweight encryption. In Wieland Fischer and Naofumi Homma, editors, *Cryptographic Hardware and Embedded Systems - CHES 2017 - 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*, volume 10529 of *Lecture Notes in Computer Science*, pages 321–345. Springer, 2017. doi:10.1007/978-3-319-66787-4_16.
- [BRV15] Alex Biryukov, Arnab Roy, and Vesselin Velichkov. Differential analysis of block ciphers simon and speck. In Carlos Cid and Christian Rechberger, editors, *Fast Software Encryption*, pages 546–570, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg. doi:10.1007/978-3-662-46706-0_28.
- [BS91] Eli Biham and Adi Shamir. Differential cryptanalysis of des-like cryptosystems. In Alfred J. Menezes and Scott A. Vanstone, editors, *Advances in Cryptology-CRYPTO’ 90*, pages 2–21, Berlin, Heidelberg, 1991. Springer Berlin Heidelberg.

- [BS93] Eli Biham and Adi Shamir. Differential cryptanalysis of the full 16-round des. In Ernest F. Brickell, editor, *Advances in Cryptology — CRYPTO’ 92*, pages 487–496, Berlin, Heidelberg, 1993. Springer Berlin Heidelberg.
- [BSS⁺15] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The SIMON and SPECK lightweight block ciphers. In *DAC 2015*, pages 175:1–175:6. ACM, 2015. doi:[10.1145/2744769.2747946](https://doi.org/10.1145/2744769.2747946).
- [CHNE24] Debasmita Chakraborty, Hosein Hadipour, Phuong Hoa Nguyen, and Maria Eichlseder. Finding complete impossible differential attacks on andrx ciphers and efficient distinguishers for ARX designs. *IACR Trans. Symmetric Cryptol.*, 2024(3):84–176, 2024. URL: <https://doi.org/10.46586/tosc.v2024.i3.84-176>, doi:[10.46586/TOSC.V2024.I3.84-176](https://doi.org/10.46586/TOSC.V2024.I3.84-176).
- [CNPV13] Anne Canteaut, María Naya-Plasencia, and Bastien Vayssière. Sieve-in-the-middle: Improved mitm attacks. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013*, pages 222–240, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [DF16] Patrick Derbez and Pierre-Alain Fouque. Automatic search of meet-in-the-middle and impossible differential attacks. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II*, volume 9815 of *Lecture Notes in Computer Science*, pages 157–184. Springer, 2016. doi:[10.1007/978-3-662-53008-5_6](https://doi.org/10.1007/978-3-662-53008-5_6).
- [DH77] W. Diffie and M.E. Hellman. Special feature exhaustive cryptanalysis of the nbs data encryption standard. *Computer*, 10(6):74–84, 1977. doi:[10.1109/C-M.1977.217750](https://doi.org/10.1109/C-M.1977.217750).
- [DSP07] Orr Dunkelman, Gautham Sekar, and Bart Preneel. Improved meet-in-the-middle attacks on reduced-round des. In K. Srinathan, C. Pandu Rangan, and Moti Yung, editors, *Progress in Cryptology – INDOCRYPT 2007*, pages 86–100, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
- [GLRW10] Jian Guo, San Ling, Christian Rechberger, and Huaxiong Wang. Advanced meet-in-the-middle preimage attacks: First results on full tiger, and improved results on md4 and sha-2. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, pages 56–75, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [HDE24] Hosein Hadipour, Patrick Derbez, and Maria Eichlseder. Revisiting differential-linear attacks via a boomerang perspective with application to AES, ascon, CLEFIA, SKINNY, PRESENT, KNOT, TWINE, WARP, LBlock, simeck, and SERPENT. IACR Cryptology ePrint Archive, Report 2016/689, 2024. URL: <https://eprint.iacr.org/2024/255>.
- [HE22] Hosein Hadipour and Maria Eichlseder. Autoguess: A tool for finding guess-and-determine attacks and key bridges. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022*, volume 13269 of *LNCS*, pages 230–250. Springer, 2022. doi:[10.1007/978-3-031-09234-3_12](https://doi.org/10.1007/978-3-031-09234-3_12).
- [HGSE24] Hosein Hadipour, Simon Gerhalter, Sadegh Sadeghi, and Maria Eichlseder. Improved search for integral, impossible differential and zero-correlation attacks application to ascon, forkskinny, skinny, mantis, PRESENT and

- qarmav2. *IACR Trans. Symmetric Cryptol.*, 2024(1):234–325, 2024. doi:
10.46586/TOSC.V2024.I1.234–325.
- [HNE22] Hosein Hadipour, Marcel Nageler, and Maria Eichlseder. Throwing boomerangs into feistel structures: Application to CLEFIA, WARP, LBlock, LBlock-s and TWINE. *IACR Trans. Symmetric Cryptol.*, 2022(3):271–302, 2022. doi:10.46586/TOSC.V2022.I3.271–302.
- [HSE23] Hosein Hadipour, Sadegh Sadeghi, and Maria Eichlseder. Finding the impossible: Automated search for full impossible-differential, zero-correlation, and integral attacks. In *EUROCRYPT 2023*, volume 14007 of *LNCS*, pages 128–157. Springer, 2023. doi:10.1007/978-3-031-30634-1_5.
- [IS13a] Takanori Isobe and Kyoji Shibutani. All subkeys recovery attack on block ciphers: Extending meet-in-the-middle approach. In Lars R. Knudsen and Huapeng Wu, editors, *Selected Areas in Cryptography*, pages 202–221, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [IS13b] Takanori Isobe and Kyoji Shibutani. Generic key recovery attack on feistel scheme. 8269:464–485, 2013. doi:10.1007/978-3-642-42033-7_24.
- [KLT15] Stefan Kölbl, Gregor Leander, and Tyge Tiessen. Observations on the simon block cipher family. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology – CRYPTO 2015*, pages 161–185, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg. doi:10.1007/978-3-662-47989-6_8.
- [KMNP10] Simon Knellwolf, Willi Meier, and María Naya-Plasencia. Conditional differential cryptanalysis of nlfsr-based cryptosystems. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, pages 130–145, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [Knu95] Lars R. Knudsen. Truncated and higher order differentials. In Bart Preneel, editor, *Fast Software Encryption*, pages 196–211, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.
- [KR16] Stefan Kölbl and Arnab Roy. A brief comparison of simon and simeck. In Andrey Bogdanov, editor, *Lightweight Cryptography for Security and Privacy - 5th International Workshop, LightSec 2016, Aksaray, Turkey, September 21-22, 2016, Revised Selected Papers*, volume 10098 of *Lecture Notes in Computer Science*, pages 69–88. Springer, 2016. doi:10.1007/978-3-319-55714-4_6.
- [LH94] Susan K. Langford and Martin E. Hellman. Differential-linear cryptanalysis. In Yvo G. Desmedt, editor, *Advances in Cryptology — CRYPTO '94*, pages 17–25, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.
- [LKKD08] Jiqiang Lu, Jongsung Kim, Nathan Keller, and Orr Dunkelman. Improving the efficiency of impossible differential cryptanalysis of reduced camellia and misty1. In Tal Malkin, editor, *Topics in Cryptology – CT-RSA 2008*, pages 370–386, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [LLW17] Zhengbin Liu, Yongqiang Li, and Mingsheng Wang. Optimal differential trails in simon-like ciphers. *IACR Trans. Symmetric Cryptol.*, 2017(1):358–379, 2017. URL: <https://doi.org/10.13154/tosc.v2017.i1.358-379>, doi:10.13154/TOSC.V2017.I1.358–379.
- [LSG⁺23] Yin Lv, Danping Shi, Yi Guo, Qiu Chen, Lei Hu, and Zihui Guo. Automatic Demirci-Selçuk meet-in-the-middle attack on SIMON. *Comput. J.*, 66(12):3052–3068, 2023. doi:10.1093/COMJNL/BXAC149.

- [Mat94a] Mitsuru Matsui. Linear cryptanalysis method for des cipher. In Tor Helleseth, editor, *Advances in Cryptology — EUROCRYPT '93*, pages 386–397, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.
- [Mat94b] Mitsuru Matsui. On correlation between the order of s-boxes and the strength of DES. In Alfredo De Santis, editor, *Advances in Cryptology - EUROCRYPT '94, Workshop on the Theory and Application of Cryptographic Techniques, Perugia, Italy, May 9-12, 1994, Proceedings*, volume 950 of *Lecture Notes in Computer Science*, pages 366–375. Springer, 1994. URL: <https://doi.org/10.1007/BFb0053451>, doi:10.1007/BFb0053451.
- [MP13] Nicky Mouha and Bart Preneel. A proof that the arx cipher salsa20 is secure against differential cryptanalysis. *IACR Cryptology ePrint Archive*, 2013:328, 2013. URL: <http://dblp.uni-trier.de/db/journals/iacr/iacr2013.html#MouhaP13>.
- [MWGP12] Nicky Mouha, Qingju Wang, Dawu Gu, and Bart Preneel. Differential and linear cryptanalysis using mixed-integer linear programming. In Chuan-Kun Wu, Moti Yung, and Dongdai Lin, editors, *Information Security and Cryptology*, pages 57–76, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [NSB⁺07] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. Minizinc: Towards a standard CP modelling language. In Christian Bessiere, editor, *CP 2007*, volume 4741 of *LNCS*, pages 529–543. Springer, 2007. doi:10.1007/978-3-540-74970-7_38.
- [QHS16] Kexin Qiao, Lei Hu, and Siwei Sun. Differential analysis on simeck and SIMON with dynamic key-guessing techniques. In Olivier Camp, Steven Furnell, and Paolo Mori, editors, *ICISSP 2016*, volume 691 of *Communications in Computer and Information Science*, pages 64–85. Springer, 2016. doi:10.1007/978-3-319-54433-5_5.
- [SB18] Sadegh Sadeghi and Nasour Bagheri. Improved zero-correlation and impossible differential cryptanalysis of reduced-round SIMECK block cipher. *IET Inf. Secur.*, 12(4):314–325, 2018. doi:10.1049/IET-IFS.2016.0590.
- [SFW15] Ling Sun, Kai Fu, and Meiqin Wang. Improved zero-correlation cryptanalysis on SIMON. In *Inscrypt*, volume 9589 of *LNCS*, pages 125–143. Springer, 2015.
- [SHMS14] Ling Song, Lei Hu, Bingke Ma, and Danping Shi. Match box meet-in-the-middle attacks on the SIMON family of block ciphers. In Thomas Eisenbarth and Erdinç Öztürk, editors, *LightSec 2014*, volume 8898 of *LNCS*, pages 140–151. Springer, 2014. doi:10.1007/978-3-319-16363-5_9.
- [SHW⁺14] Siwei Sun, Lei Hu, Peng Wang, Kexin Qiao, Xiaoshuang Ma, and Ling Song. Automatic security evaluation and (related-key) differential characteristic search: Application to simon, present, lblock, des(l) and other bit-oriented block ciphers. In Palash Sarkar and Tetsu Iwata, editors, *Advances in Cryptology – ASIACRYPT 2014*, pages 158–178, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.
- [SSD⁺18] Danping Shi, Siwei Sun, Patrick Derbez, Yosuke Todo, Bing Sun, and Lei Hu. Programming the demirci-selçuk meet-in-the-middle attack with constraints. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018*, pages 3–34, Cham, 2018. Springer International Publishing.

- [ST17] Yu Sasaki and Yosuke Todo. New impossible differential search tool from design and cryptanalysis aspects. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017*, pages 185–215, Cham, 2017. Springer International Publishing.
- [SYL23] Ling Song, Qianqian Yang, and Huimin Liu. Revisiting the differential meet-in-the-middle cryptanalysis. Cryptology ePrint Archive, Paper 2023/1302, 2023. URL: <https://eprint.iacr.org/2023/1302>.
- [Wag99] David Wagner. The boomerang attack. In *Fast Software Encryption, 6th International Workshop, FSE ’99, Rome, Italy, March 24-26, 1999, Proceedings*, volume 1636 of *Lecture Notes in Computer Science*, pages 156–170. Springer, 1999. doi:10.1007/3-540-48519-8_12.
- [WWHL18] Xuzi Wang, Baofeng Wu, Lin Hou, and Dongdai Lin. Automatic search for related-key differential trails in simon-like block ciphers based on MILP. In Liqun Chen, Mark Manulis, and Steve A. Schneider, editors, *Information Security - 21st International Conference, ISC 2018, Guildford, UK, September 9-12, 2018, Proceedings*, volume 11060 of *Lecture Notes in Computer Science*, pages 116–131. Springer, 2018. doi:10.1007/978-3-319-99136-8_7.
- [WWJZ18] Ning Wang, Xiaoyun Wang, Keting Jia, and Jingyuan Zhao. Differential attacks on reduced SIMON versions with dynamic key-guessing techniques. *Sci. China Inf. Sci.*, 61(9):098103:1–098103:3, 2018. doi:10.1007/S11432-017-9231-5.
- [XZBL16] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying milp method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology – ASIACRYPT 2016*, pages 648–678, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
- [YZS⁺15] Gangqiang Yang, Bo Zhu, Valentin Suder, Mark D. Aagaard, and Guang Gong. The simeck family of lightweight block ciphers. In *CHES 2015*, volume 9293 of *LNCN*, pages 307–329. Springer, 2015. doi:10.1007/978-3-662-48324-4_16.
- [ZGHL18] Kai Zhang, Jie Guan, Bin Hu, and Dongdai Lin. Security evaluation on simeck against zero-correlation linear cryptanalysis. *IET Inf. Secur.*, 12(1):87–93, 2018. doi:10.1049/IET-IFS.2016.0503.
- [ZLW⁺23] Kai Zhang, Xuejia Lai, Lei Wang, Jie Guan, and Bin Hu. A revisited security evaluation of simeck family ciphers against impossible differential cryptanalysis. *Sci. China Inf. Sci.*, 66(3), 2023. doi:10.1007/S11432-022-3466-X.

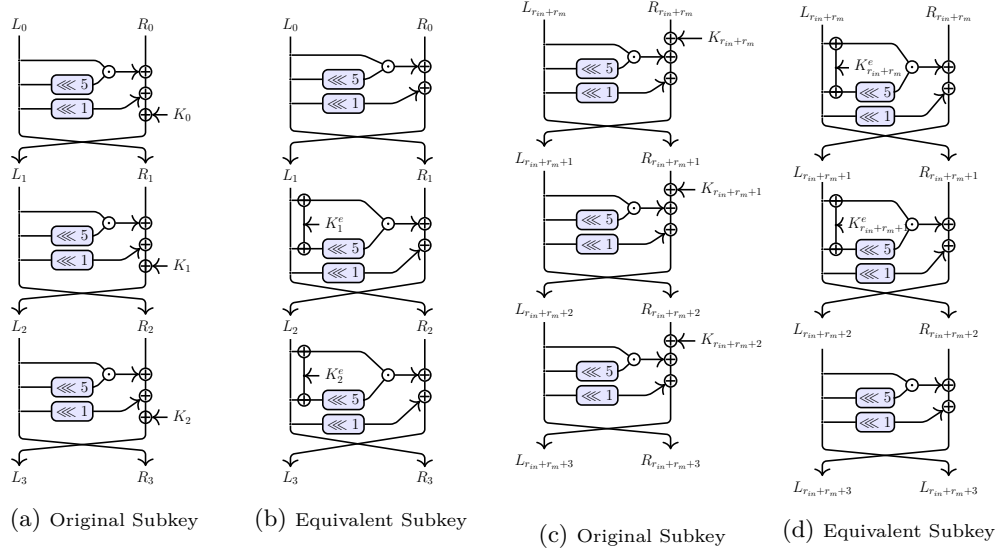


Figure 6: Relationship between original subkey and equivalent subkey in Simeck

A Relationship between original subkey and equivalent subkey in Simeck

The relationships between equivalent subkeys and original subkeys in Simeck Figure 6 are as follows:

$$\begin{cases}
 K_1^e = K_0 \\
 K_2^e = (K_1^e \lll 1) \oplus K_1 \\
 K_3^e = K_1^e \oplus (K_2^e \lll 1) \oplus K_2 \\
 K_4^e = K_2^e \oplus (K_3^e \lll 1) \oplus K_3 \\
 \vdots \\
 K_{r_{in}-1}^e = K_{r_{in}-3}^e \oplus K_{r_{in}-2} \\
 \quad \quad \quad \oplus (K_{r_{in}-2}^e \lll 1) \\
 K_{r_{in}}^e = K_{r_{in}-2}^e \oplus K_{r_{in}-1} \\
 \quad \quad \quad \oplus (K_{r_{in}-1}^e \lll 1)
 \end{cases}
 \quad
 \begin{cases}
 K_{r_{tot}-2}^e = K_{r_{tot}-1} \\
 K_{r_{tot}-3}^e = (K_{r_{tot}-2}^e \lll 1) \oplus K_{r_{tot}-2} \\
 K_{r_{tot}-4}^e = K_{r_{tot}-2}^e \oplus K_{r_{tot}-3} \\
 \quad \quad \quad \oplus (K_{r_{tot}-3}^e \lll 1) \\
 K_{r_{tot}-5}^e = K_{r_{tot}-3}^e \oplus K_{r_{tot}-4} \\
 \quad \quad \quad \oplus (K_{r_{tot}-4}^e \lll 1) \\
 \vdots \\
 K_{r_{in}+r_m}^e = K_{r_{in}+r_m+2}^e \oplus K_{r_{in}+r_m+1} \\
 \quad \quad \quad \oplus (K_{r_{in}+r_m+1}^e \lll 1) \\
 K_{r_{in}+r_m-1}^e = K_{r_{in}+r_m+1}^e \oplus K_{r_{in}+r_m} \\
 \quad \quad \quad \oplus (K_{r_{in}+r_m}^e \lll 1)
 \end{cases}$$

B Figures Related to Differential MITM attacks on SIMON and Simeck

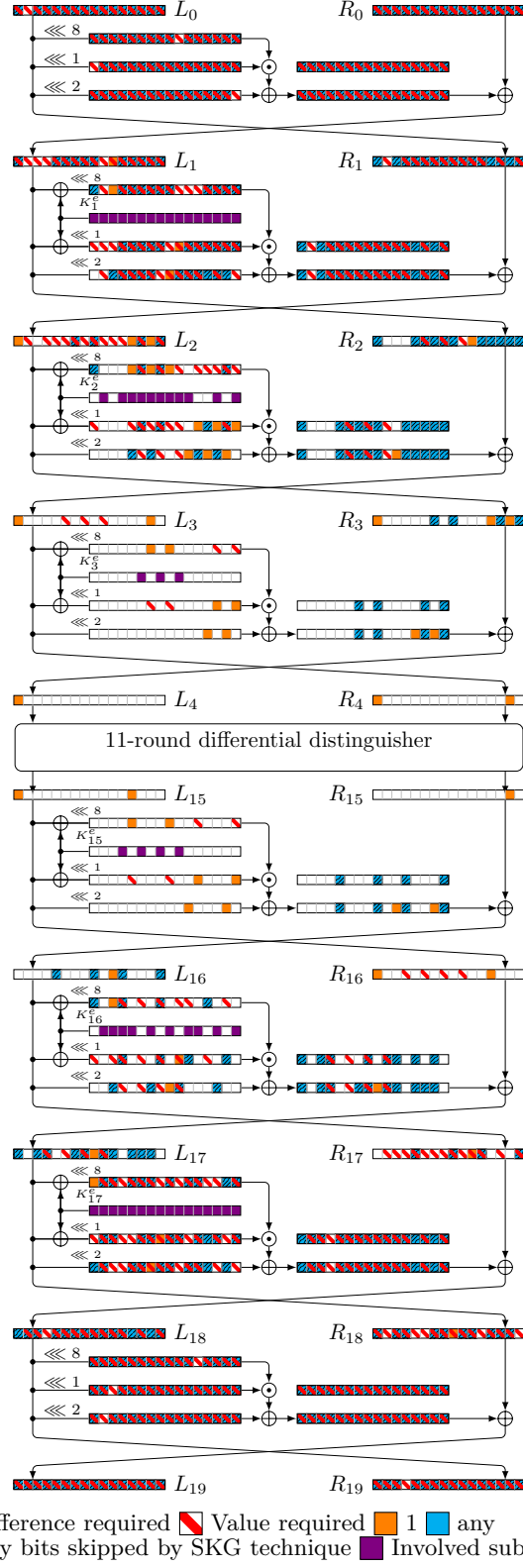


Figure 7: Differential MITM attack on 19-round SIMON-32-64.

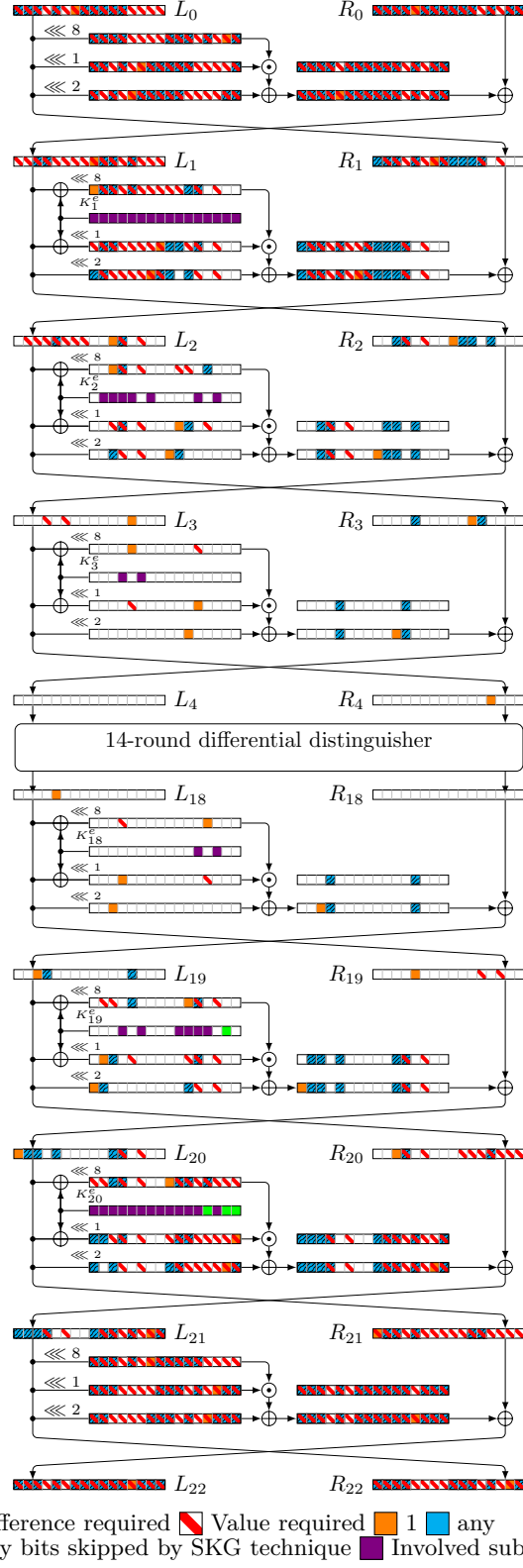


Figure 8: Differential MITM attack on 22-round SIMON-32-64.

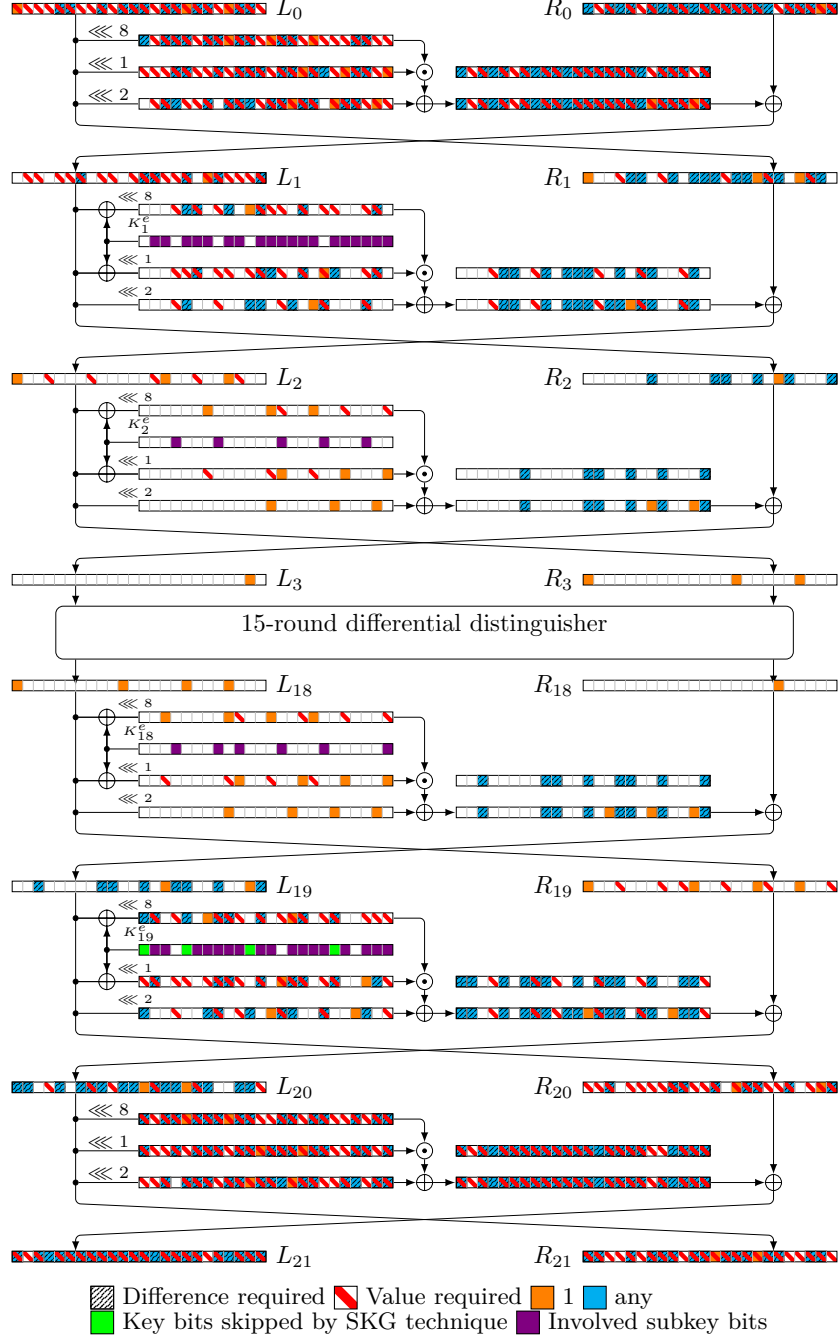


Figure 9: Differential MITM attack on 21-round SIMON-48-72.

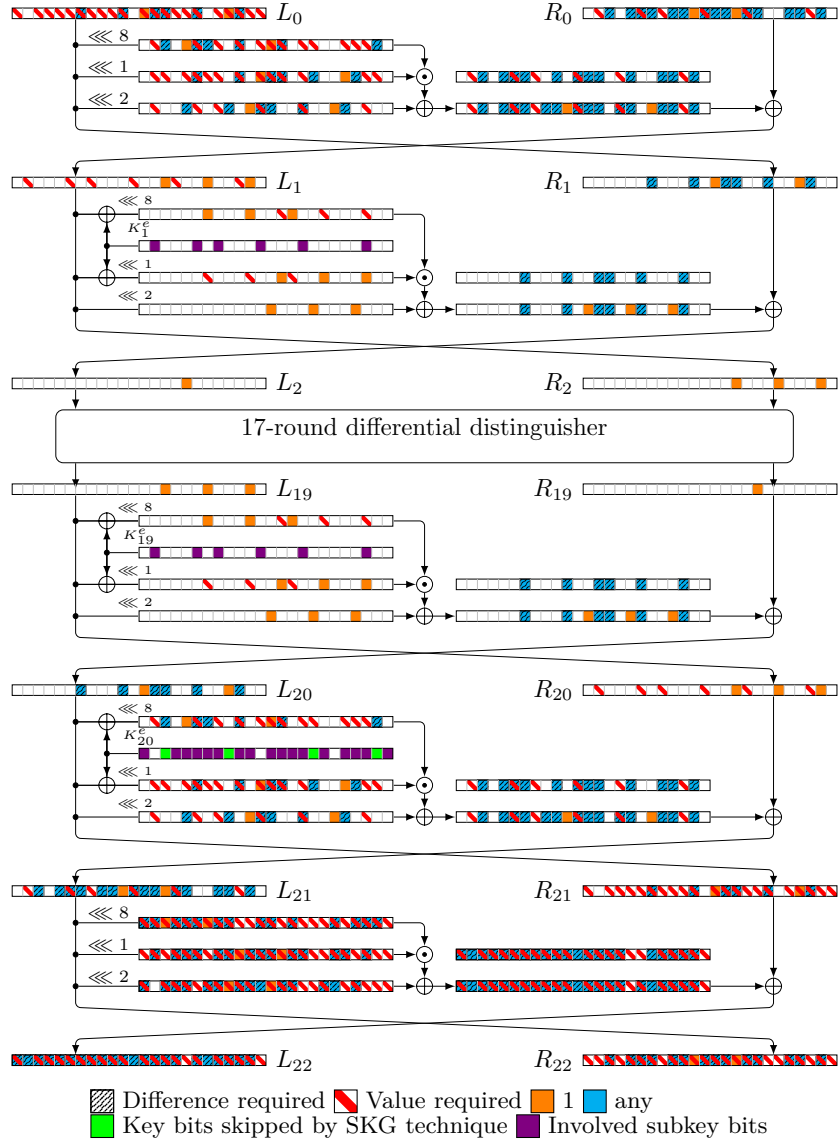


Figure 10: Differential MITM attack on 22-round SIMON-48-72.

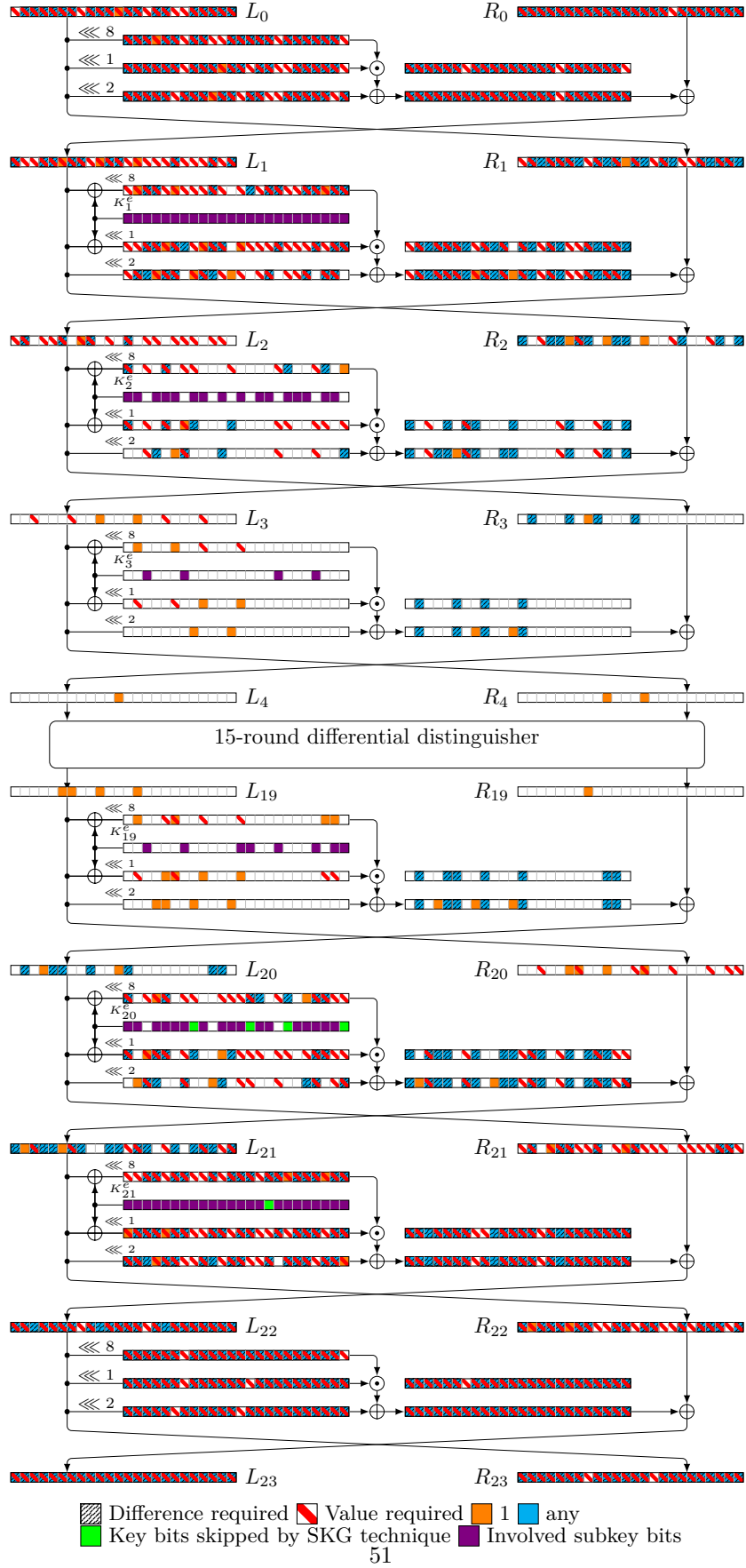


Figure 11: Differential MITM attack on 23-round SIMON-48-96.

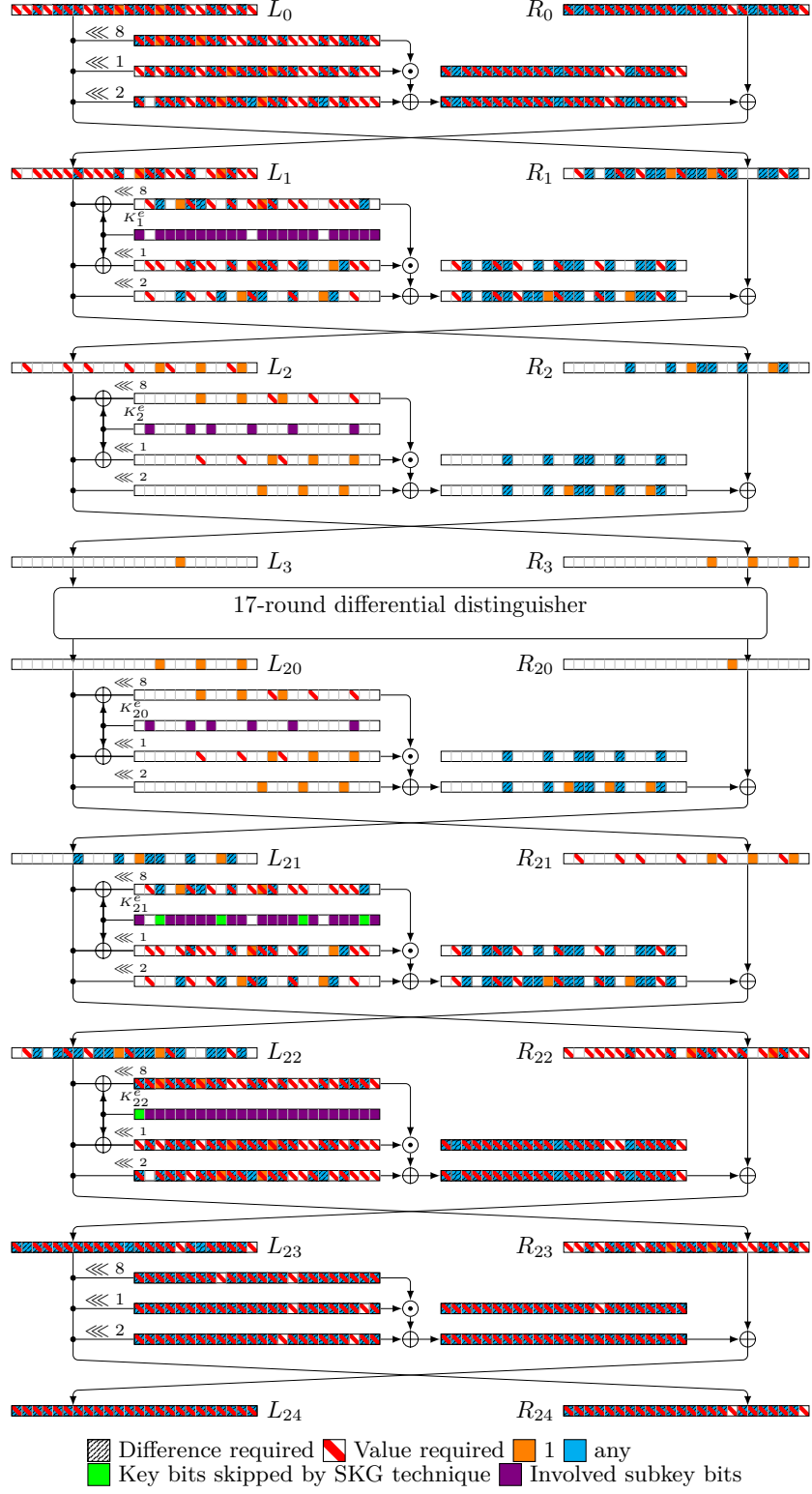


Figure 12: Differential MITM attack on 24-round SIMON-48-96.

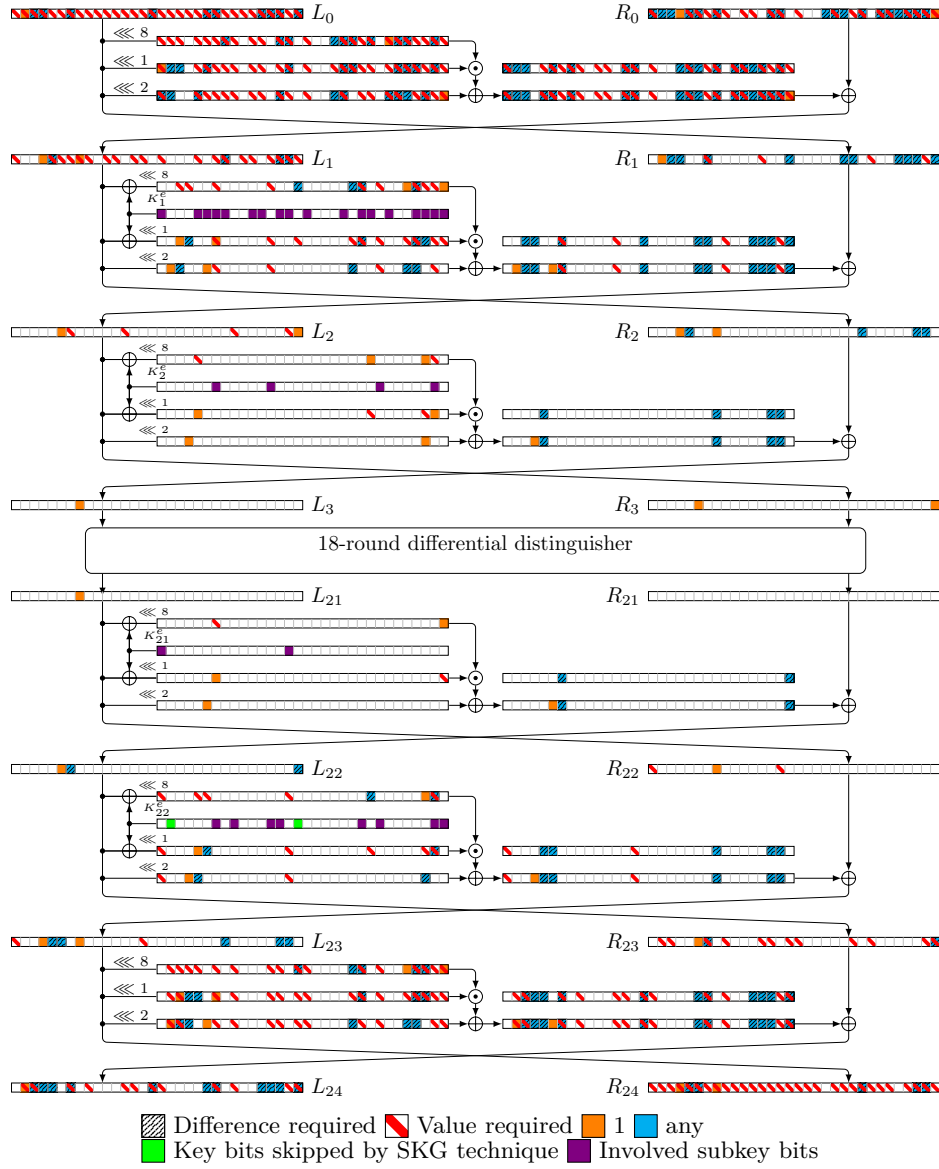


Figure 13: Differential MITM attack on 24-round SIMON-64-96.

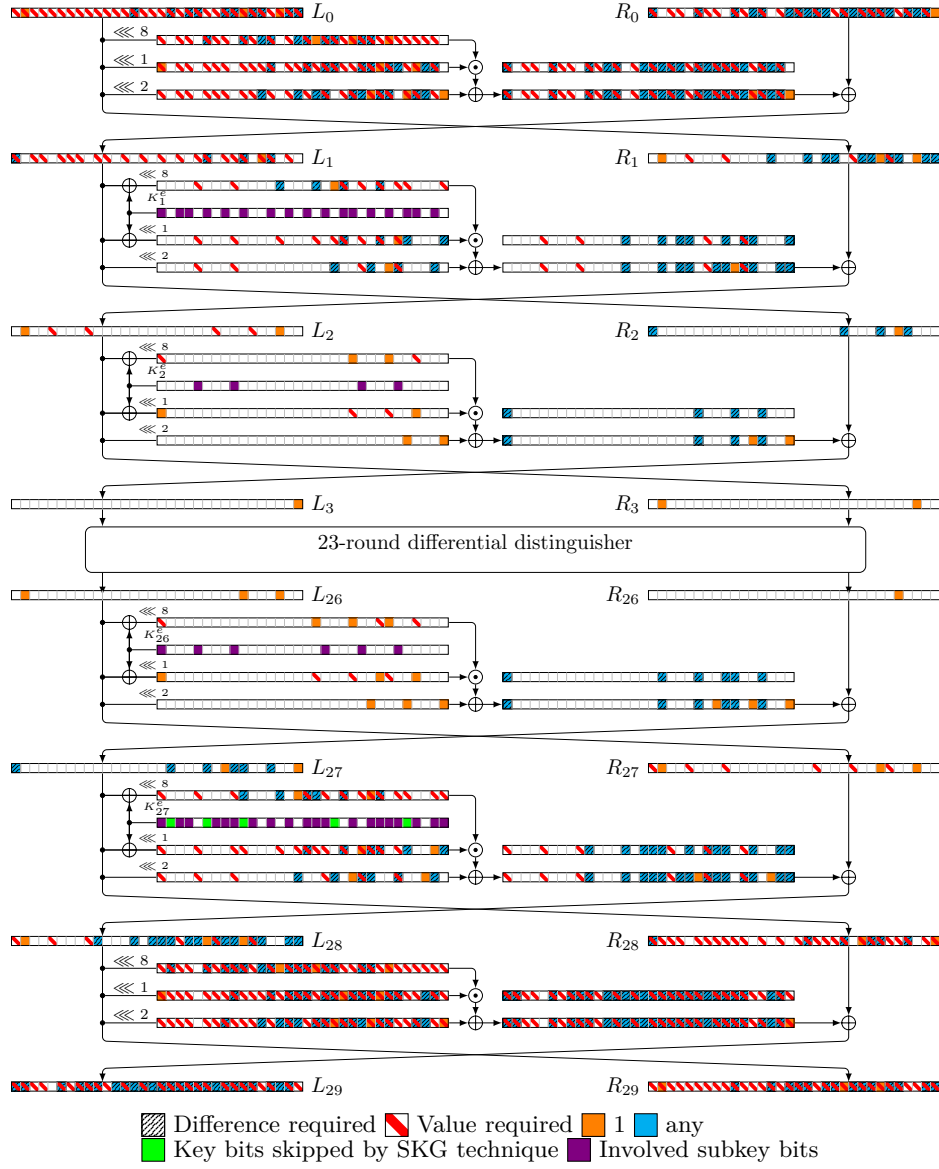


Figure 14: Differential MITM attack on 29-round SIMON-64-96.

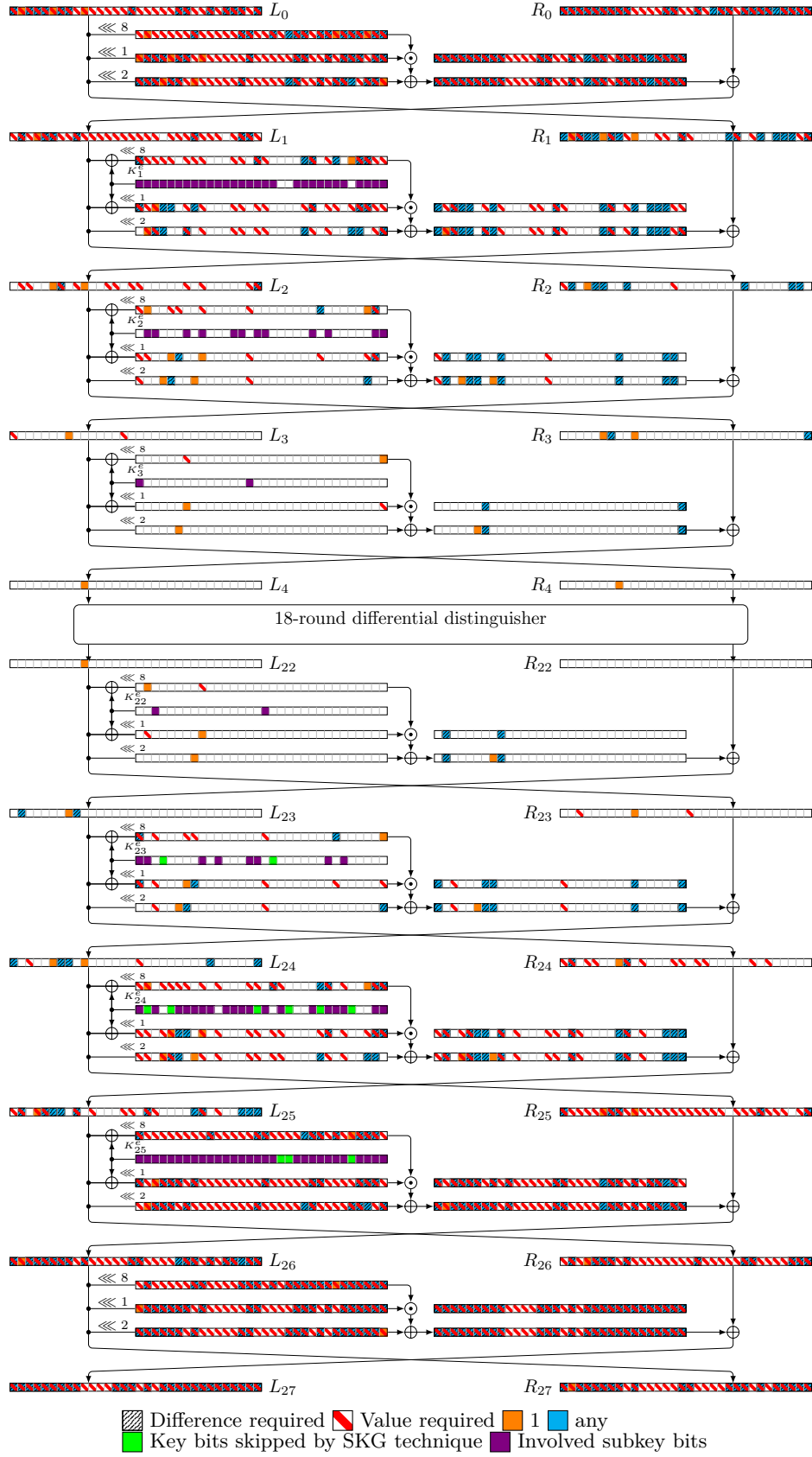


Figure 15: Differential MITM attack on 27-round SIMON-64-128.

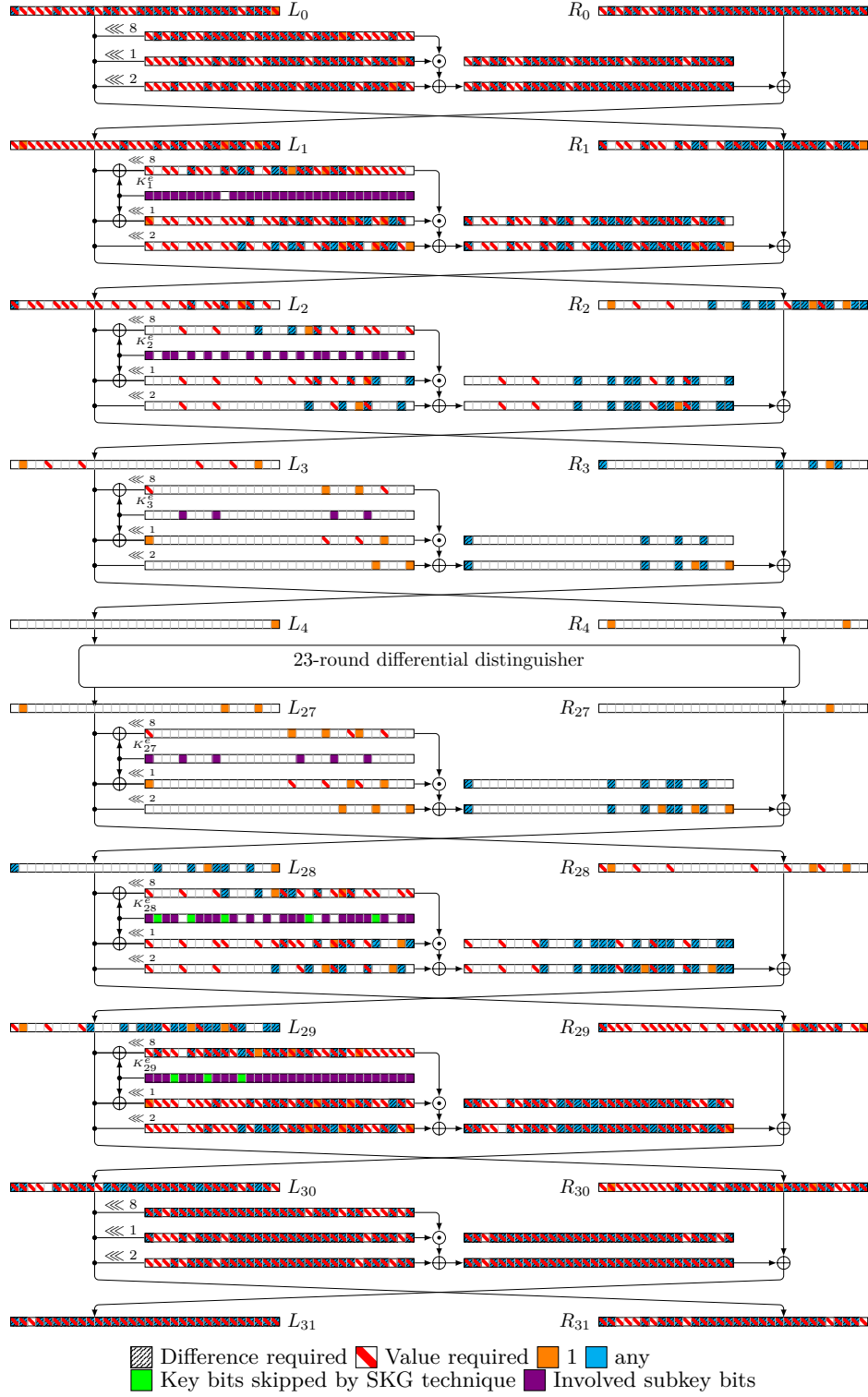


Figure 16: Differential MITM attack on 31-round SIMON-64-128.

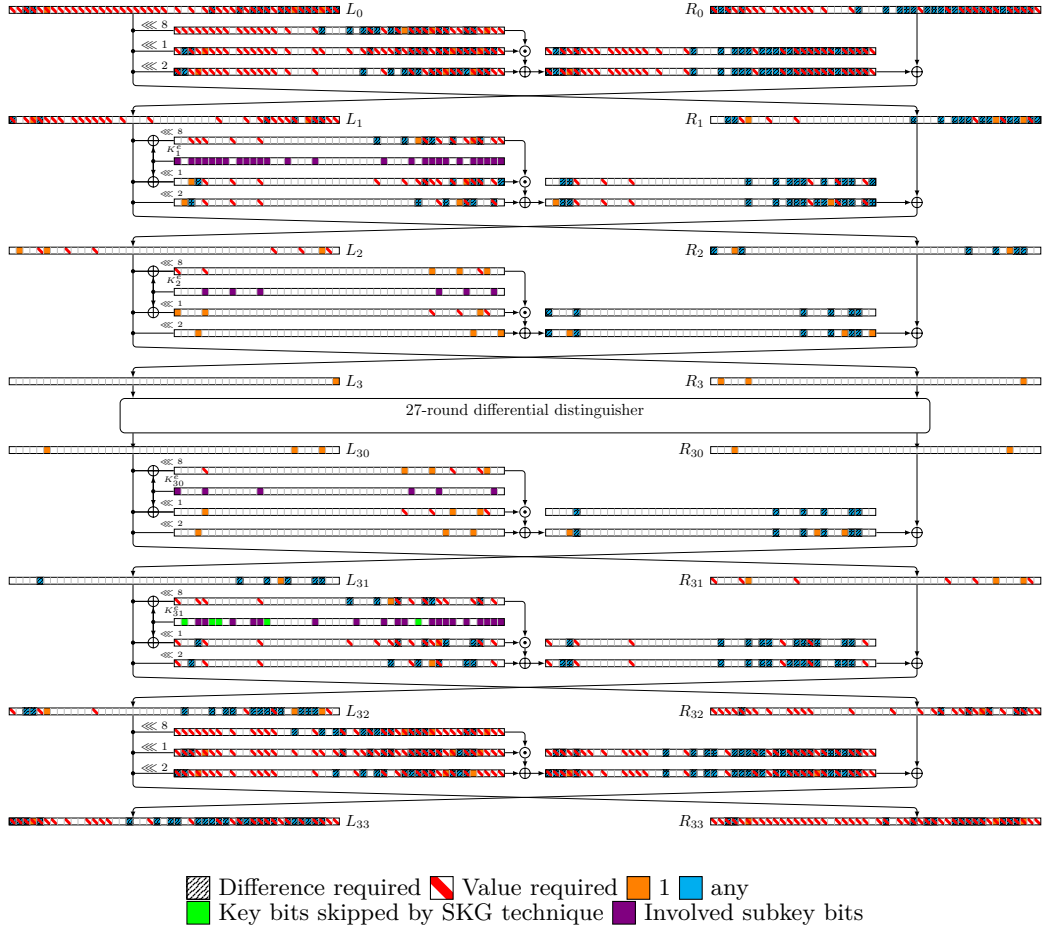


Figure 17: Differential MITM attack on 33-round SIMON-96-144.

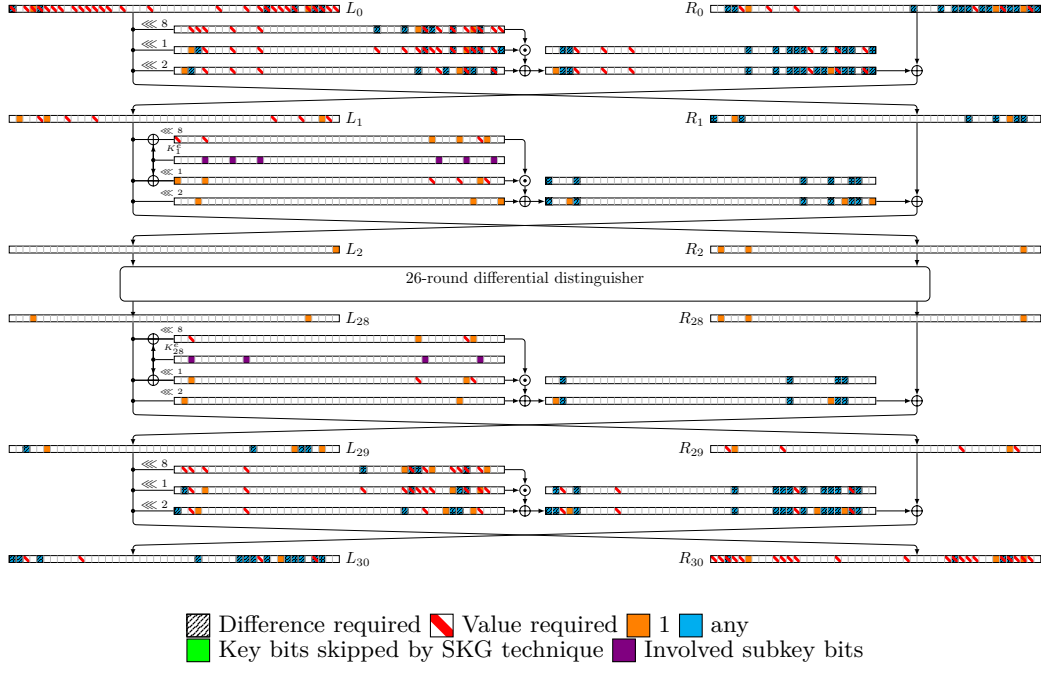


Figure 18: Differential MITM attack on 30-round SIMON-96-96.

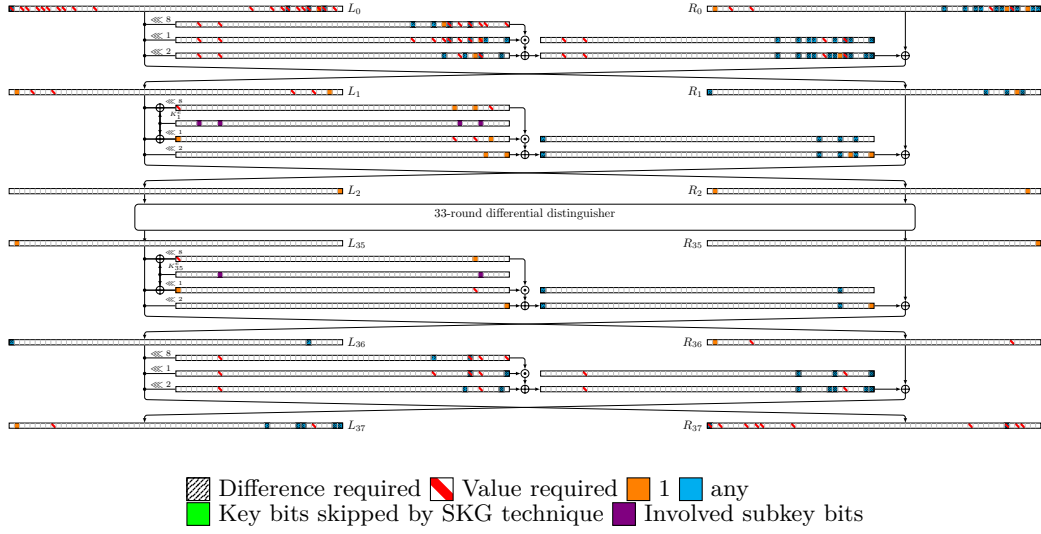


Figure 19: Differential MITM attack on 37-round SIMON-128-128.

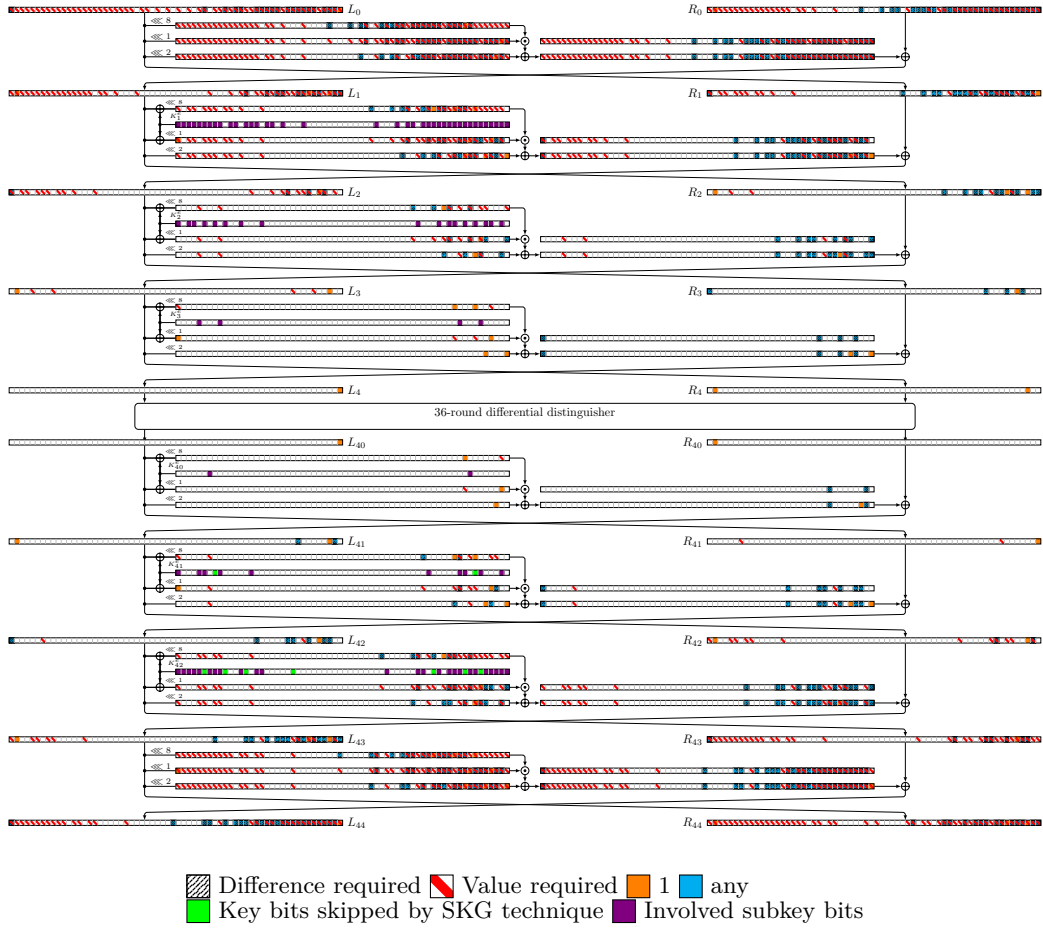


Figure 20: Differential MITM attack on 44-round SIMON-128-192.

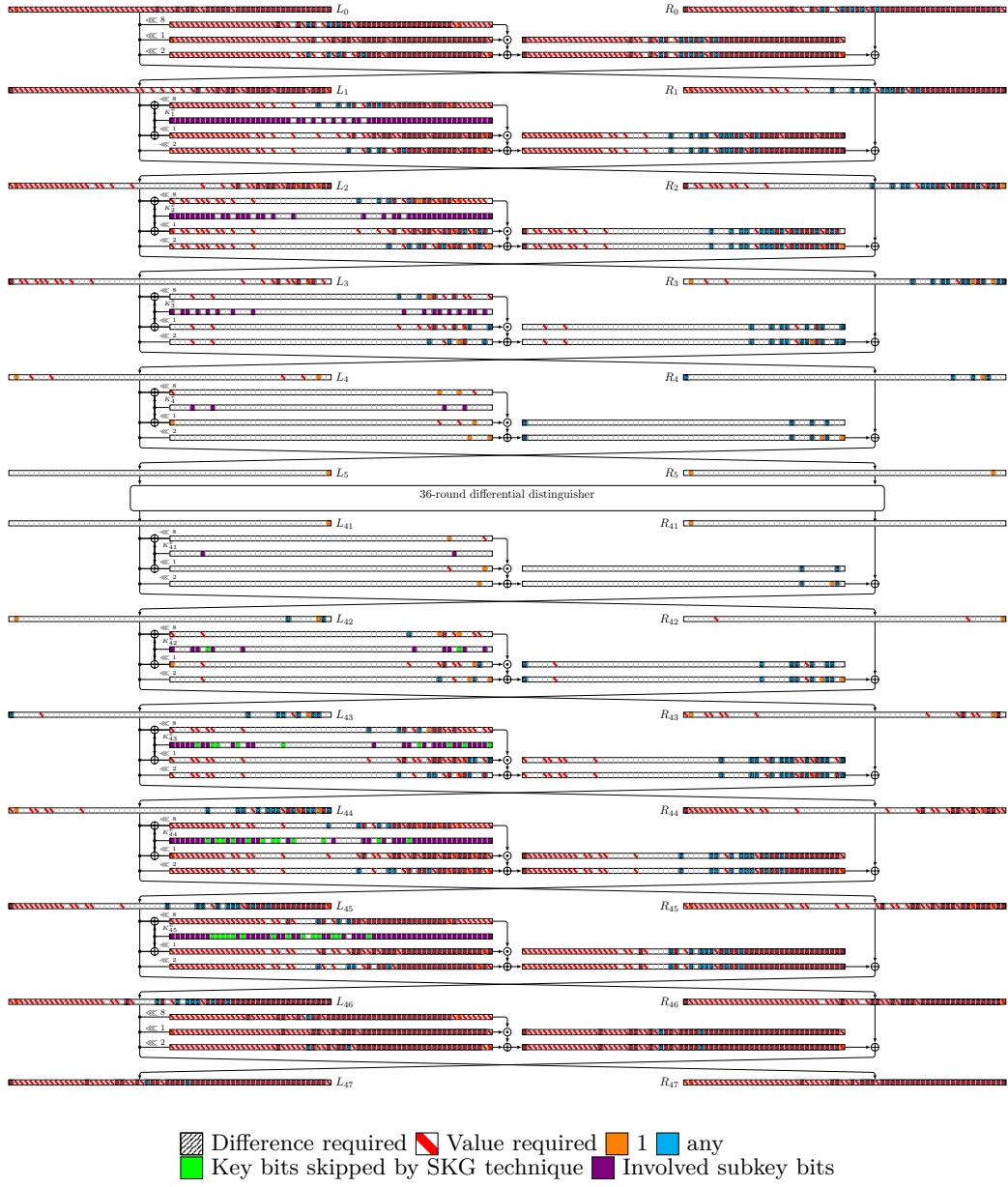


Figure 21: Differential MITM attack on 47-round SIMON-128-256.

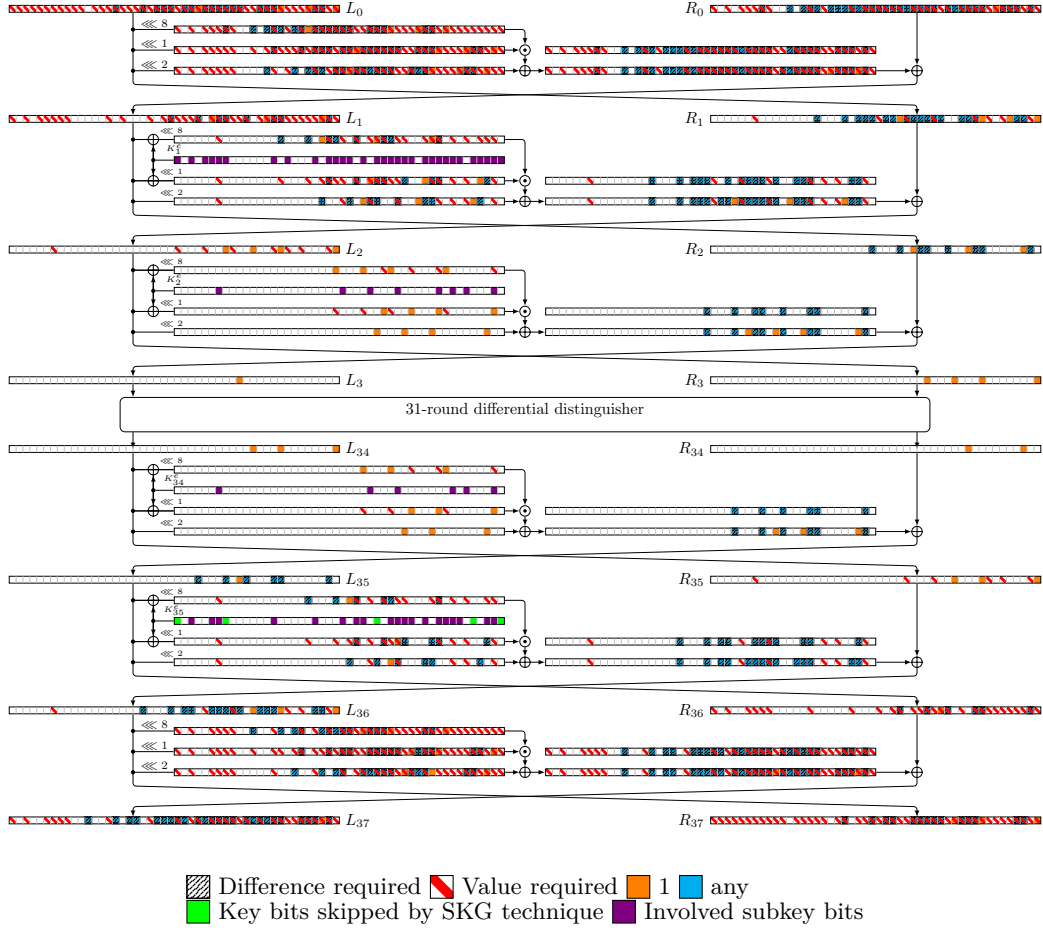


Figure 22: Differential MITM attack on 37-round SIMON-96-144.

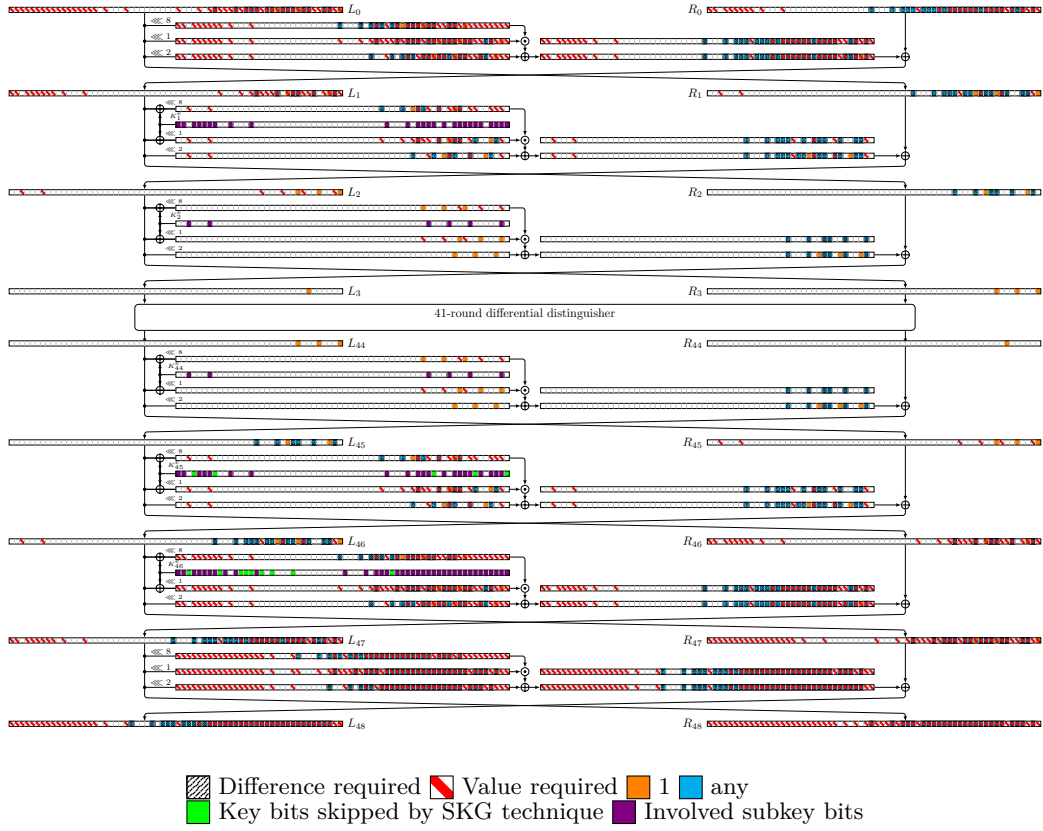


Figure 23: Differential MITM attack on 48-round SIMON-128-192.

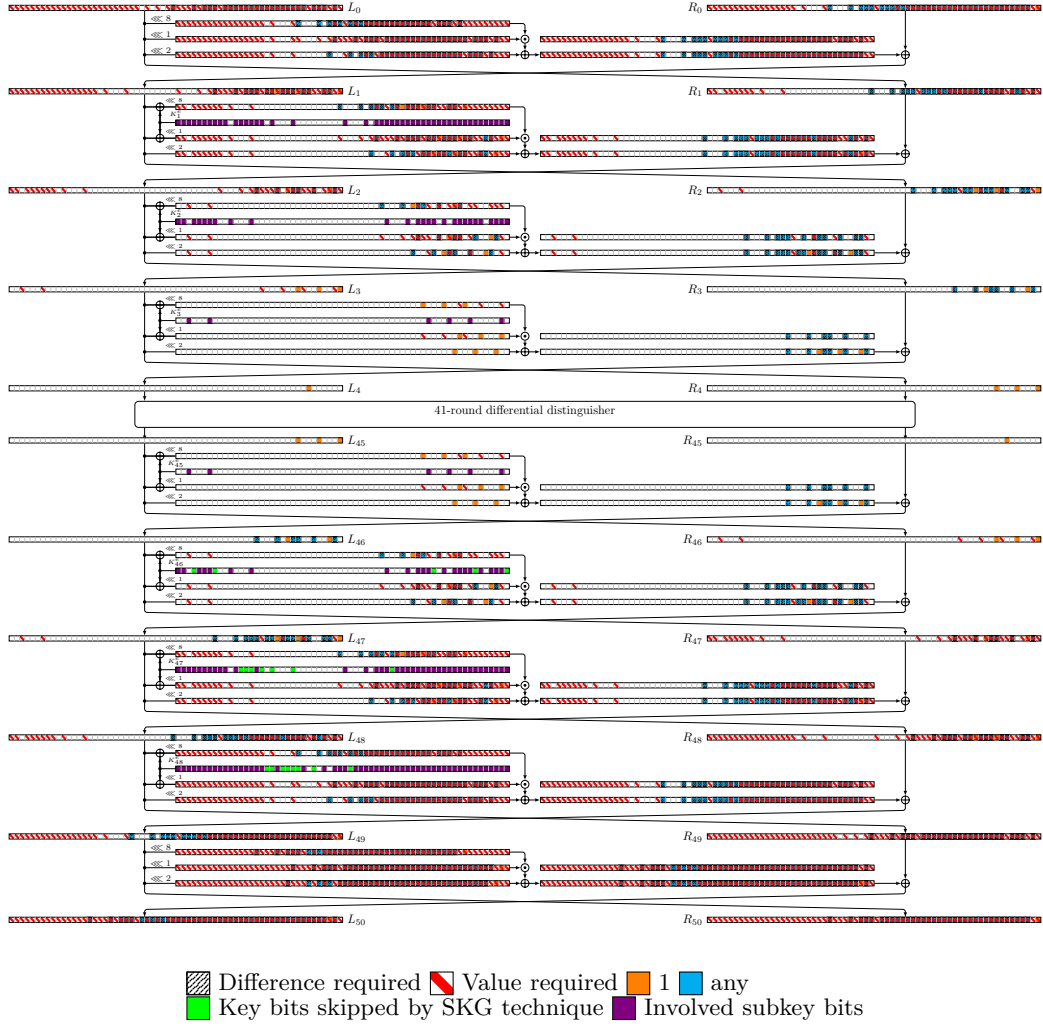
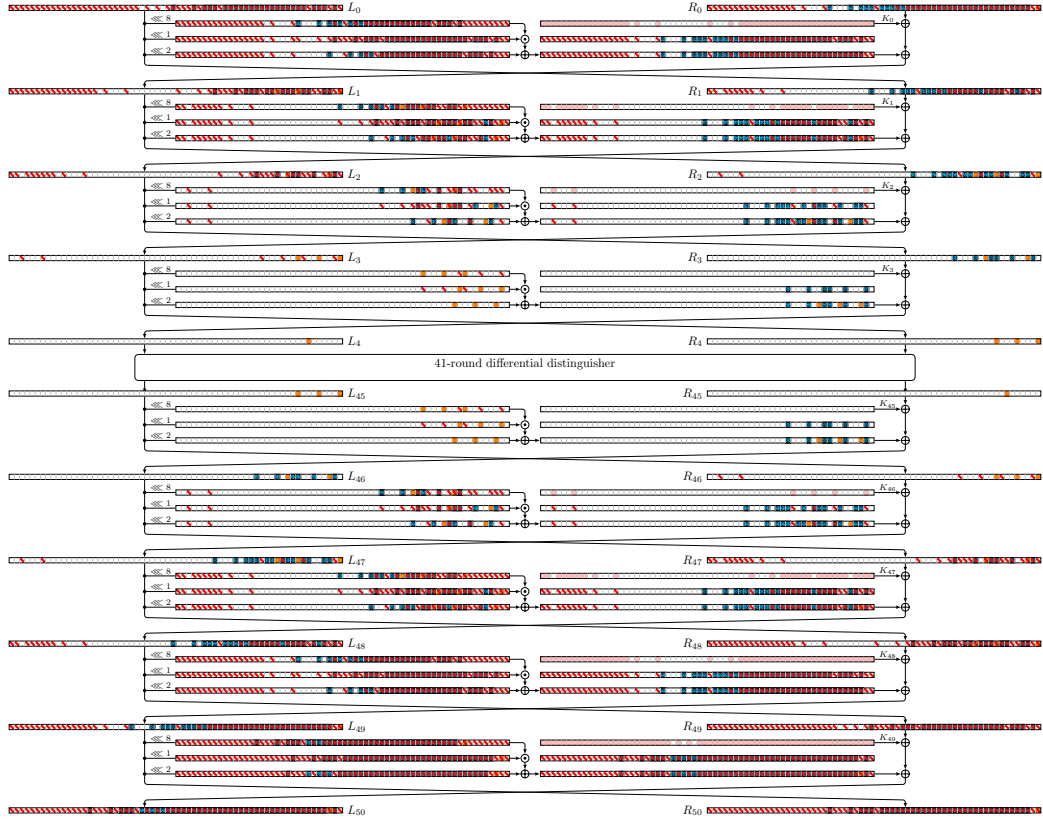


Figure 24: Differential MITM attack on 50-round SIMON-128-256.



Difference required
 Value required
 1
 any
 Involved subkey bits

Figure 25: Differential MITM attack on 50-round SIMON-128-256 original subkey.

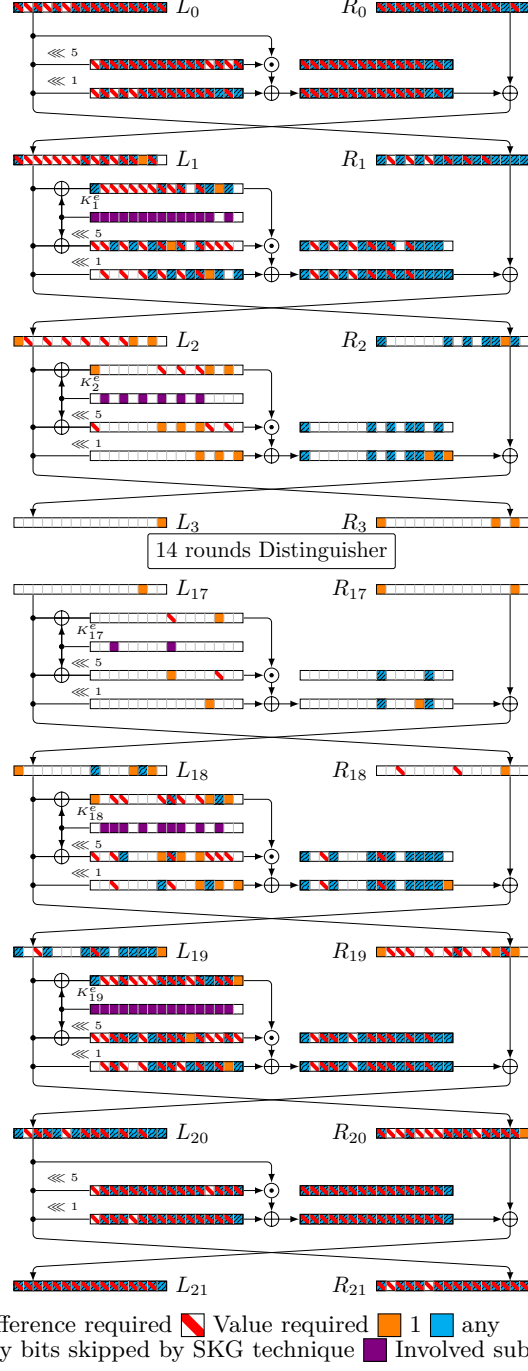


Figure 26: Differential MITM attack on 21-round Simeck-32-64.

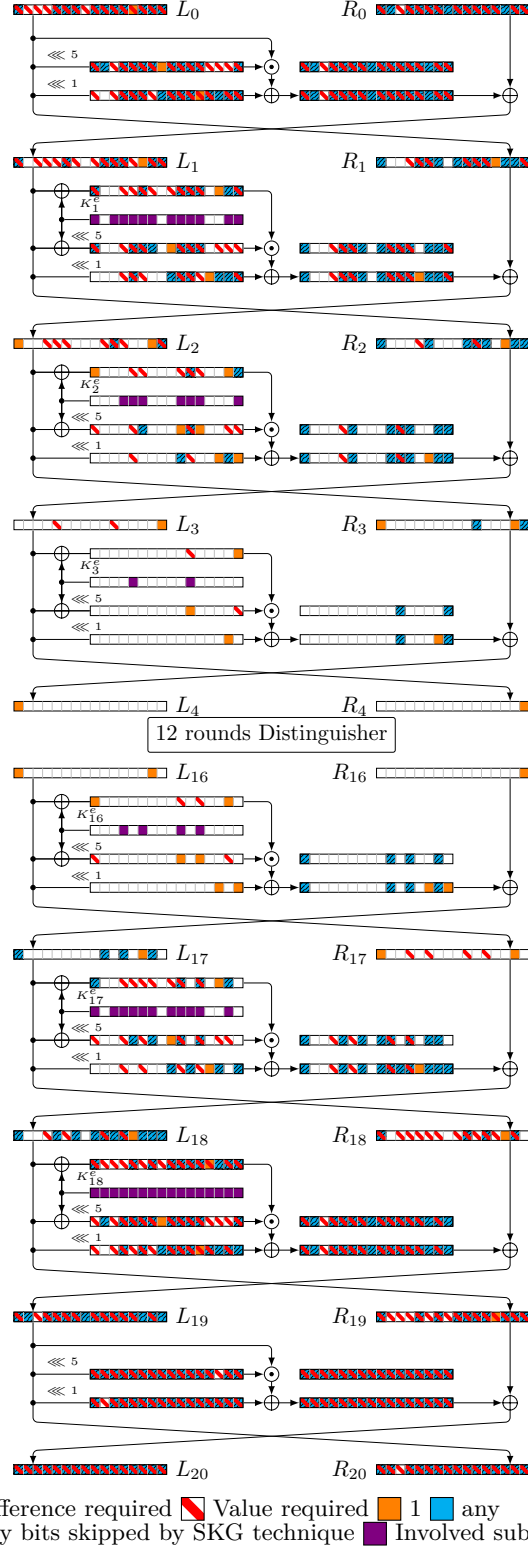


Figure 27: Differential MITM attack on 20-round Simeck-32-64.

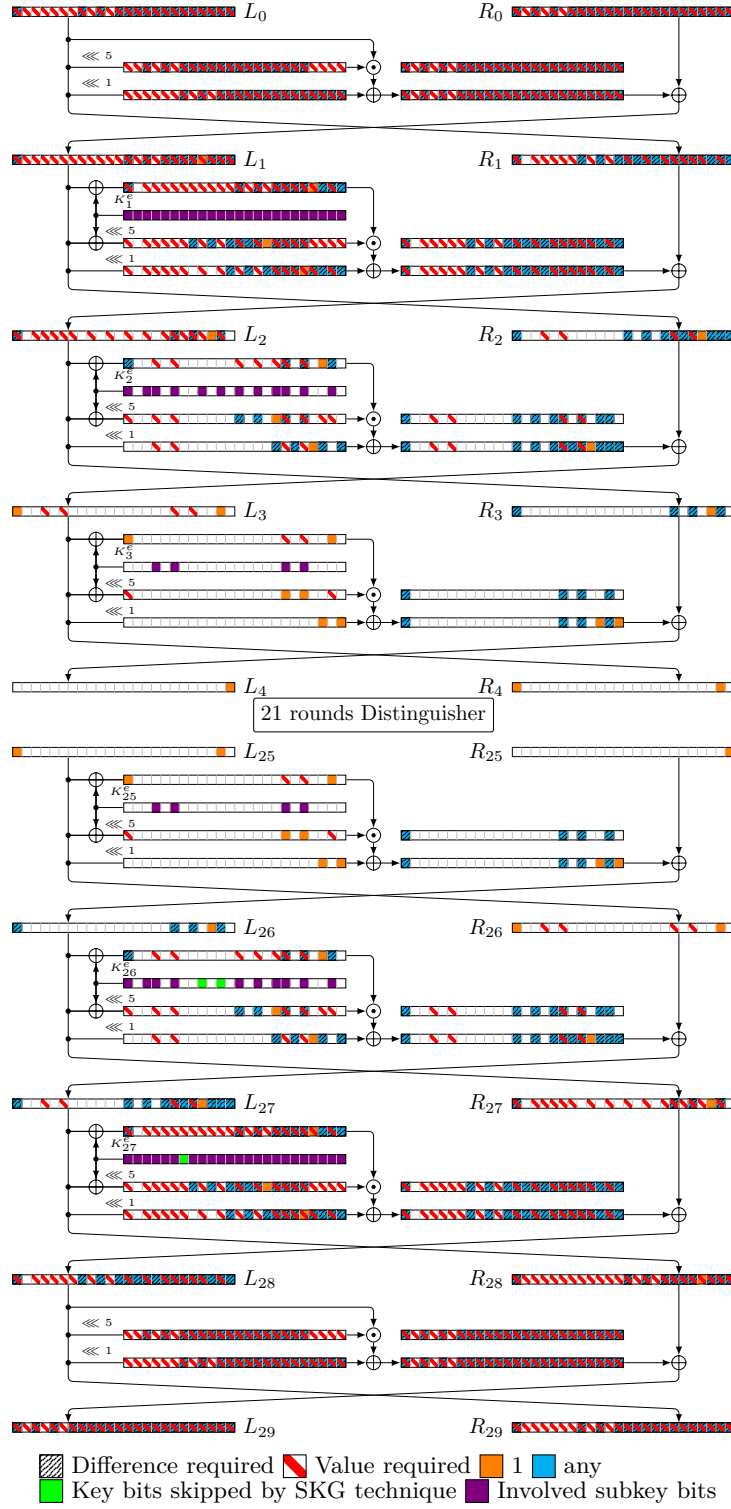


Figure 28: Differential MITM attack on 29-round Simeck-48-96.

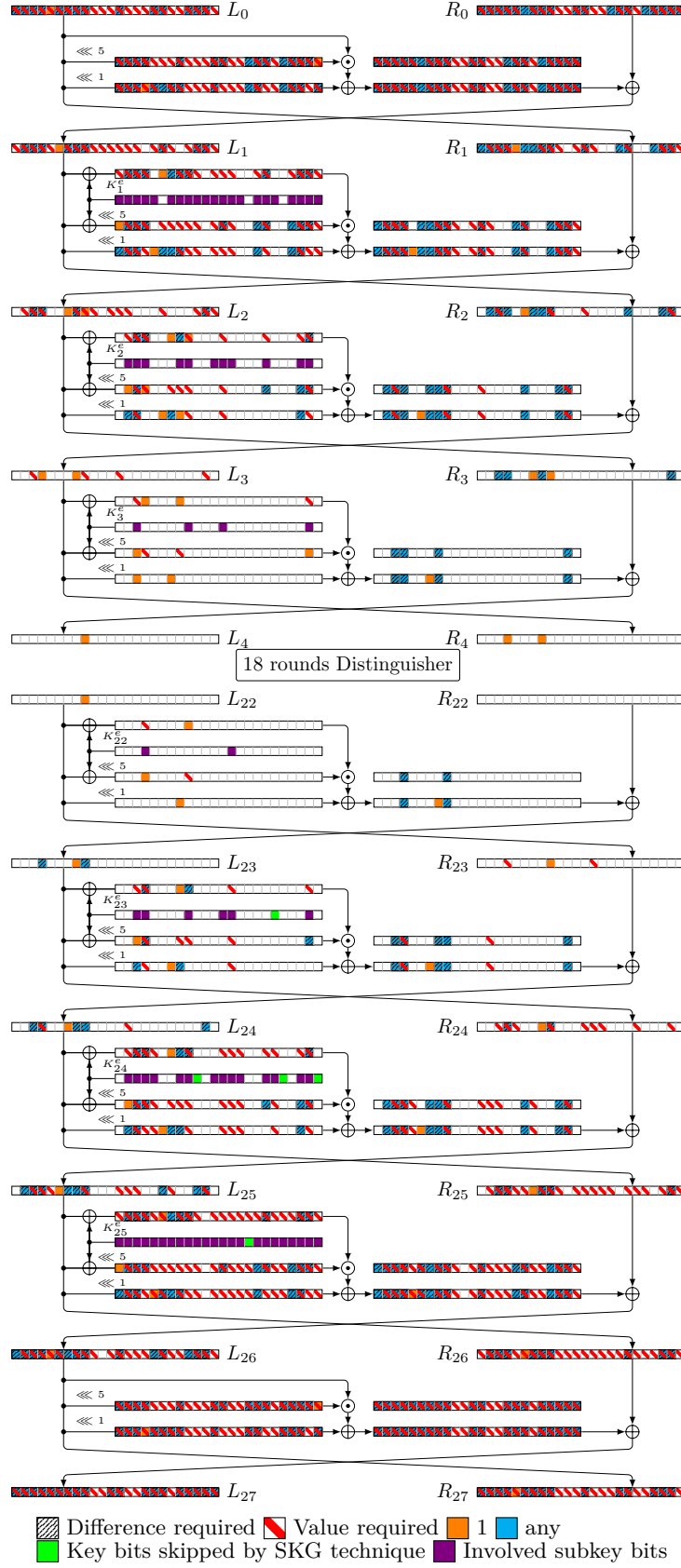


Figure 29: Differential MITM attack on 27-round Simeck-48-96.

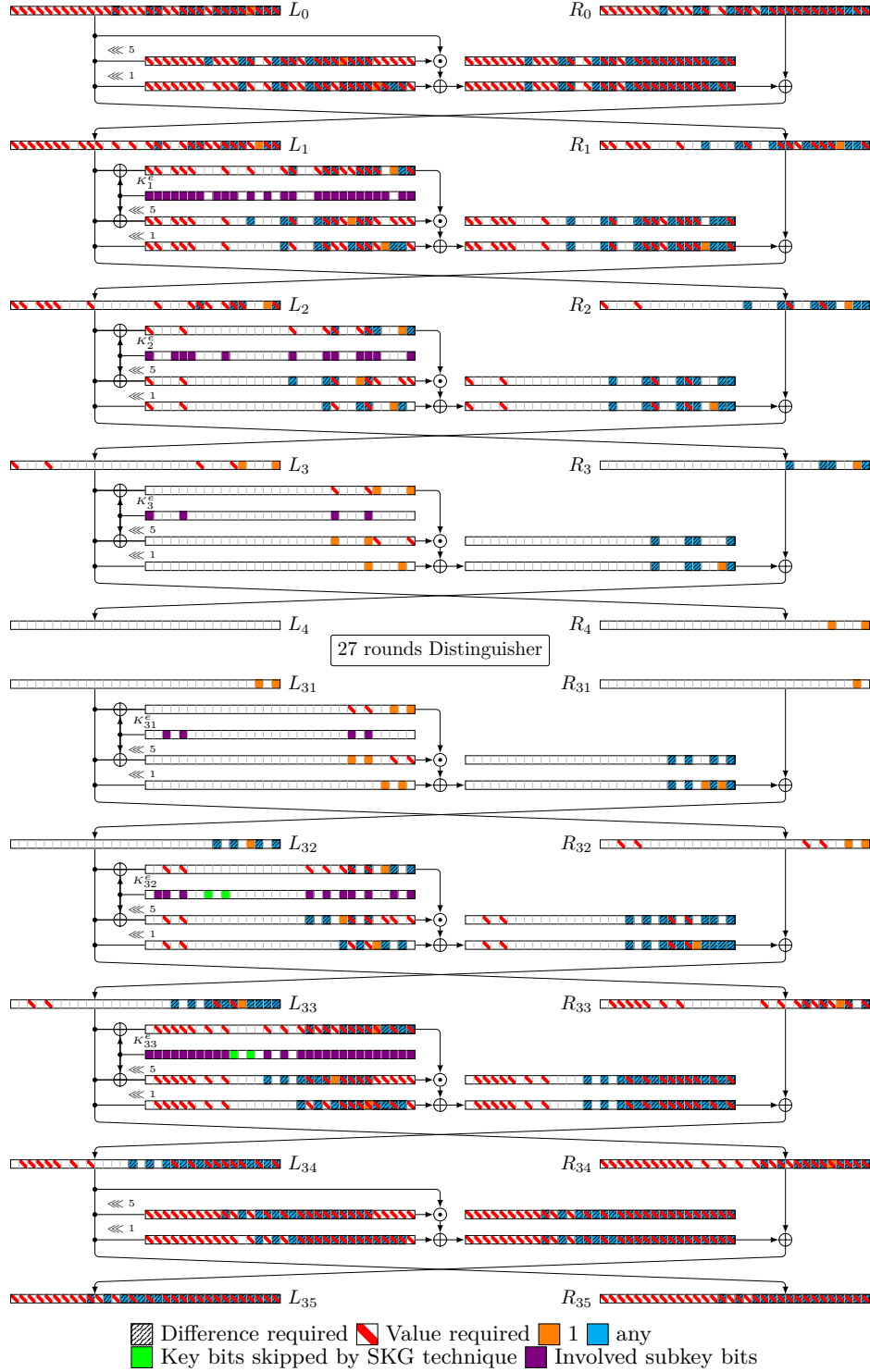


Figure 30: Differential MITM attack on 35-round Simeck-64-128.

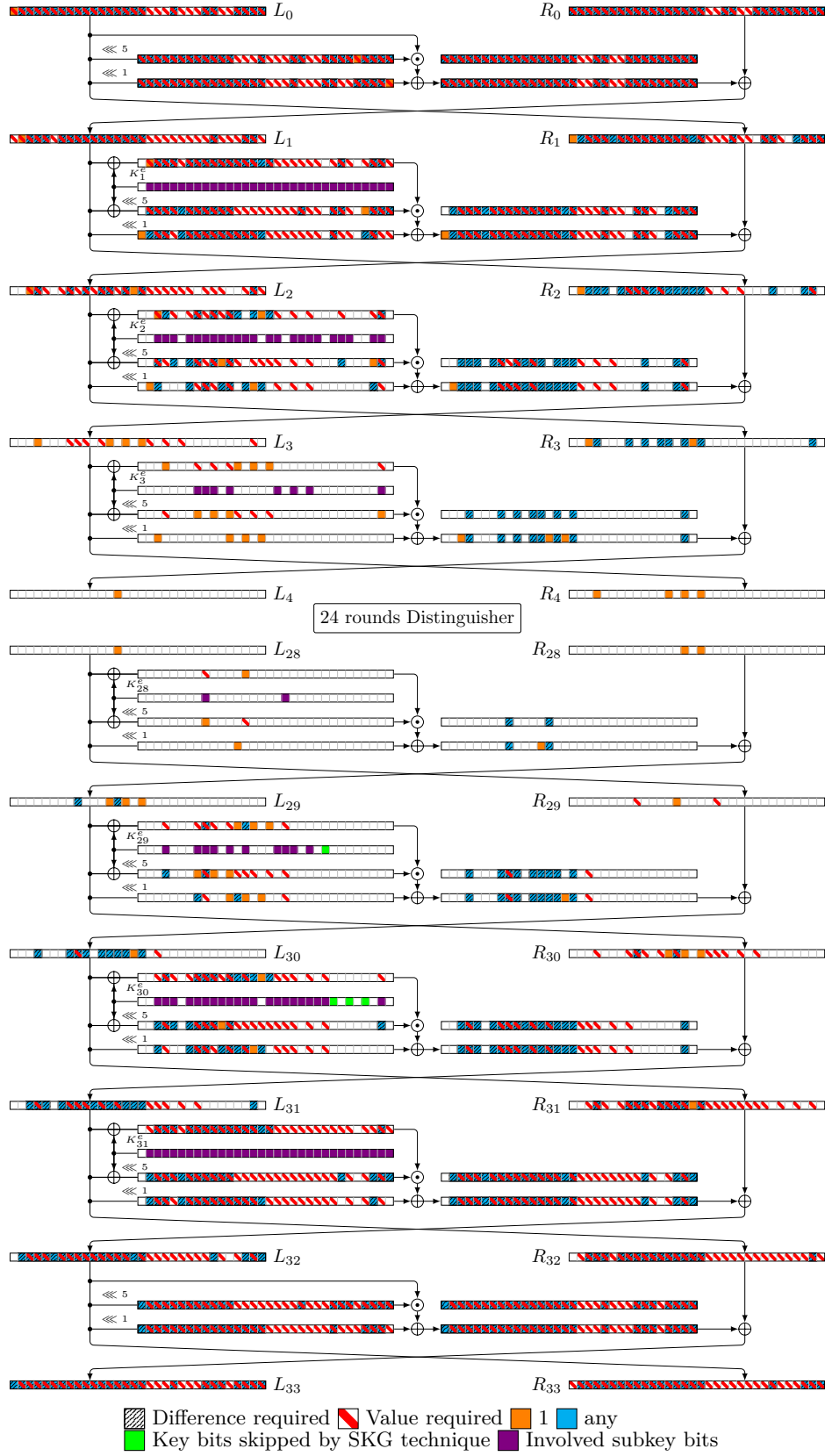


Figure 31: Differential MITM attack on 33-round Simeck-64-128.